

Lecture Notes: Algorithms, Sequential and Parallel

Lecture Notes - Student Version[†]

Kyle Burke

April 21, 2026

This work is licensed under a Creative Commons
“Attribution 4.0 International” license.



Contents

0 Preliminaries	3
0.1 Programming Languages Used	4
0.2 Pre-requisites	4
0.3 Acknowledgements	4
1 Introduction	5
1.1 What is an Algorithm?	5
1.2 Fundamentals of Problem Solving	12
1.3 Problem Types	15
1.4 Fundamental Data Structures	15
2 Analysis Fundamentals	16
2.1 Analysis Framework	16
2.2 Asymptotic Notation	20
2.3 Data Structures Review	39

[†]Created with lectureNotes.sty, which is available at: <https://github.com/paithan/LaTeX-LectureNotes>.

2.4	Iterative Analysis	55
2.5	Recursive Analysis	82
3	Brute Force	89
3.1	Brute Force Sorting	91
3.2	Sequential Search	95
3.3	Geometric Algorithms	100
3.4	Exhaustive Search	104
3.5	Graph Searching	105
4	Decrease and Conquer	114
4.1	Decrease-and-Conquer Sorting	127
4.2	Topological Sorting	130
4.3	Generating Combinatorial Objects	130
4.4	Constant-Factor Decr & Conq Algos	130
4.5	Variable-Size Decr & Conq Algos	134
5	Divide and Conquer	135
5.1	MergeSort	143
5.2	QuickSort	150
5.3	Binary Trees	159
5.4	Large Numbers and Matrices	159
5.5	Geometric Div & Conq Algos	159
6	Transform and Conquer	159
6.1	Presorting	162
6.2	Gaussian Elimination	165
6.3	Balanced Search Trees	165
6.4	Heap Sort	175
6.5	Horner's Rule	179
6.6	Problem Reduction	179
7	Time and Space Tradeoffs	187
7.1	Sorting and Counting	187
7.2	String Matching	187
7.3	Hashing	187
7.4	B-Trees	198
8	Dynamic Programming	199

9 Greedy Algorithms	204
9.1 Prim's Algorithm	206
9.2 Kruskal's Algorithm	209
9.3 Borůvka's Algorithm	211
9.4 Dijkstra's Shortest Path Algorithm	215
10 Iterative Improvement	221
11 Algorithmic Limitations	221
11.1 Lower Bounds	221
11.2 Decision Trees	222
11.3 P vs NP	223
11.4 Using Hardness (Cryptography)	236
12 Constant-Time Max Finding	246
A Answers to Exercises	252

0 Preliminaries

These notes are very incomplete. They are in a tough spot. I am working on it, I promise!

These notes were originally designed to accompany “Introduction to the Design and Analysis of Algorithms” by Anany Levitin [3]¹. I have made some changes (and will continue to make changes). Those are mostly:

- I cover some basic parallel concepts and how using more processors can help speed up programs. (This is mostly on a conceptual level instead of actual parallel programming code samples.)
- I don't exactly follow Levitin's sections for the beginning of the course, so those are different.

Included in here are separate homework assignments, some with the solutions available in the back.

¹<http://www.csc.villanova.edu/~levitin/>

0.1 Programming Languages Used

Code samples are included in here in Java and in algorithmic notation. (I reserve the right to add Python snippets as I see fit.) I originally taught this course using Chapel², so I include two additional keywords to the algorithmic notation that closely model two Chapel keywords.

0.2 Pre-requisites

These notes assume you have a basic understanding of common data structures: arrays, dynamic arrays³, linked lists, stacks, queues, priority queues, sets, binary trees, and graphs. We will avoid the details of most of these, but later sections we'll dig deep into:

- Heaps, in section 6.4.
- Self-balancing Binary Search Trees in section 6.3.
- Hash tables in section 7.3.

This course also assumes you have a basic understanding of discrete mathematics, including: logical expressions and truth tables, set theory, functions, and summation notation.

0.3 Acknowledgements

Thank you to all the students who have taken this course with me over the years, including as CS 275 at Wittenberg and CSC 3380 at Florida Southern! Further appreciation to:

- Brenda Alvarez for pointing out a mistake in a solution and a mistake in an algorithm in the text.
- Jose Venecia for pointing out a mistake in an algorithm.
- Haley Cahoon for pointing out issues with multiple exercises.
- Cade Butler for pointing out an issue with an exercise.

²<https://chapel-lang.org/>

³e.g. Python's lists and Java's ArrayLists.

- Riley Peters, who found multiple errors in exercise answers.
- Bridget Martinez, who found errors in listed running times.
- Anderson Morris, who found errors in five exercises across multiple sections and errors in the text in many places, including at critical points in algorithms.
- Brendan Whitmire for finding mistakes in multiple algorithms.
- Erynn Prado for helping me add an explainer of a common term.

1 Introduction

Welcome to Algorithms!

⟨ Go over Syllabus. ⟩

Computer Science is: Algorithms, algorithms and algorithms!
Anything that uses algorithms is Computer science, even if no computers are being used.

1.1 What is an Algorithm?

Q: What is an algorithm?

A:

Q: Which is more important: the speed of computers or the efficiency of an algorithm?

A:

Sometimes it's better to have a slow human use a fast algorithm than have a fast computer use a slow algorithm!

Example: Consider adding the numbers 1 to n , where $n = 1,000,000$?

Q: How many steps For a computer to brute Force this?

A:

Q: Recall: $\sum_{i=1}^n i = \frac{n(n+1)}{2}$. How many steps to calculate using this?

A:

Q: How many times faster than me does the computer need to be?

A:

Q: What If $n = 3 \times 10^{18}$? (Exascale)

A:

You want your algorithms to scale well For big problems.
Also, some algorithms don't use computers at all!

Q: Some algorithms don't even use computers. What is PCR?

A:

Q: How can we specify algorithms?

A:

The most rigid I'll ask for in written homework is algorithmic

notation.

Algorithm: PCR

Input: d , a strand of DNA to copy; n , the desired number of copies of d

Output: $k \geq n$ copies of d

1: Add d to a solution of proteins.

2: $k \leftarrow 1$

3: while $k < n$ do

4: Heat solution.

5: Cool solution.

6: $k \leftarrow 2k$

7: end while

8: Remove k copies of d from the solution.

Q: How long does it take to run the PCR algorithm?

A:

Q:

How do you find your friend buried in an avalanche? You and your friend were prepared: have standard beacon and receiver. Turn it on and it beeps. If you're in range of it. Your other tools: flags to plant and a shovel to dig them out once you're on top of them. You can walk in a straight line, make right-angle turns and find midpoints between two flags. Show the video: <http://www.youtube.com/watch?v=6C2eWRvZgKU>

A:

Q: How do you know this works?

A:

Q: How long does it take (in terms of the beacon's signal distance, r) in the worst case?

A:

Q: Another example: what is the best way to navigate Meijer? If you have a list of groceries to purchase? If you have 2 people, can you do it twice as fast? Can you do it *nearly* twice as fast?

Q: What is $\text{GCD}(84, 48)$?

A:

Q: How did you find that? What is an algorithm For $\text{GCD}(x, y)$?

A:

Q: Why can't we binary search?

A:

```
Algorithm: GCD(x, y) (Euclid's Algorithm)
Output:  $x$ , the greatest common divisor of  $x$  and  $y$ .
  while  $y \neq 0$  do
     $z \leftarrow y$ 
     $y \leftarrow x \bmod y$ 
     $x \leftarrow z$ 
  end while
```

1.2 Fundamentals of Problem Solving

Q: What's the first thing you need to do to start writing an algorithm to solve instances of a problem?

A: Understand the problem!

Q: What's next?

A: Know the capabilities of your machine!

Q: What does RAM stand For in this class? Hint: different than in organization.

A: Random Access Machine.

Q: If we are using the RAM model For computers, what does that mean?

A: A computer with one processor where each memory access takes the same amount of time (constant).

Q: What is the PRAM model?

A:

Q: How does it differ?

A:

Q: We've got our machine. What do we do next?

A:

Q: Next?

A:

Q: Next?

A:

Q: Next?

A:

Q: Is this the usual order in software development?

A:

Q: Why?

A:

1.3 Problem Types

Skipped... but still homework! :)

1.4 Fundamental Data Structures

Q: Which are the data structures I hope you're comfortable with?

A:

As mentioned in the introduction, I won't be getting in to the

details here. We do review some of the basic details in section 2.3.

2 Analysis Fundamentals

Q: What are the resources we might have to spend on solving our problems?

A:

2.1 Analysis Framework

We'll cover:

- Time efficiency (lots!)
- Space efficiency (much less)

Q: What about processors? "Processor Efficiency"?

A:

Q: What is the other parameter used to defined the efficiencies?

A:

Q: Examples?

A:

Q: What are different ways to measure algorithm efficiency?

A:

Q: Always focus on Worst-Case complexity?

A:

Q: Other cases?

A:

Q: Amortized example: If I have a sorted list, L , of n integers, what's the fastest way to tell if some integer, x is in the list?

A:

Q: What if I have two numbers, x_0 and x_1 ? How long to determine which, if any, are in L ?

A:

Q: What if you have n of them: $x_0, \dots, x_{n-1}$, also sorted?

A:

Q: Can we do better?

A:

↔ *HOMEWORK* : 2.1: Suggested: 1, 4b, 7, 10; Graded: 3 (best and worst cases), 4a, 8. For 8, let $v(n)$ be each function. Then write each answer as: $v(n) = \dots$ in terms of $v(n)$. ↔

2.2 Asymptotic Notation

I expect you have already seen a little bit of bit- O notation (or big- Θ) in a data structures course. (If not, don't worry... keep going!) Those are both examples of *asymptotic notation* because they help us rationalize about how functions grow as the input gets very large.

Q:

What are the steps we use when categorizing a function using asymptotic notation? (This mostly applies to polynomials, but that's fine to start with.)

A:

- Remove all coefficients (but not signs) from all terms. For example, $8n^4 + 5n^3 + 80 \rightarrow n^4 + n^3 + 1$.
- Drop any terms that are “dominated” by other terms. (Dominated just means that another term grows faster.) For example, $n^4 + n^3 + 1 \rightarrow n^4$.

This kind of simplification makes sense in computer science because it may be hard to nail down the exact coefficients of running time formulas:

- The actual hardware might affect different coefficients of terms in different ways, completely independent of the algorithm.
- The operating system might affect how the code runs and is prioritized among other jobs.
- The compiler may make decisions that change coefficients of terms in an attempt to optimize the code. In addition, it might change how many assembly-level steps are needed for each high-level step.

Because of all these reasons, doing the simplification gives us a “good enough” way of finding the big differences between different running times.

We want an actual formal mathematical way of expressing this categorization. We want to be able to say things like:

1. All formulas with a biggest term of n^4 are similar. E.g.: $8n^4 + 5n^3 + 80 \approx n^4$.
2. All formulas with a biggest term of n^4 are less than those with a biggest term of n^5 . E.g.: $8n^4 + 5n^3 + 80 < n^5$.
3. All formulas with a biggest term of n^4 are greater than those with a biggest term of n^3 . E.g.: $8n^4 + 5n^3 + 80 > n^3$.

Unfortunately, those symbols don't really make sense for formulas, so we have to use sets and set comparisons instead. We'll define big- O so that these are all true:

1. $O(8n^4 + 5n^3 + 80) = O(n^4)$
2. $O(8n^4 + 5n^3 + 80) \subsetneq O(n^5)$.
3. $O(8n^4 + 5n^3 + 80) \supsetneq O(n^3)$.

These will be sets of formulas, so we'll be able to say:

$$8n^4 + 5n^3 + 80 \in O(n^4)$$

Big- O is going to act like our " $\leq$ ", so these two things are also going to be true:

- $8n^4 + 5n^3 + 80 \in O(n^5)$, but
- $8n^4 + 5n^3 + 80 \notin O(n^3)$

We're also going to have big- Ω and big- Θ , which will work like this:

- Big- Ω will be like $\geq$, so:
 - $8n^4 + 5n^3 + 80 \in \Omega(n^4)$, and
 - $8n^4 + 5n^3 + 80 \notin \Omega(n^5)$, and
 - $8n^4 + 5n^3 + 80 \in \Omega(n^3)$
- Big- Θ will be like $=$, so:
 - $8n^4 + 5n^3 + 80 \in \Theta(n^4)$, and
 - $8n^4 + 5n^3 + 80 \notin \Theta(n^5)$, and
 - $8n^4 + 5n^3 + 80 \notin \Theta(n^3)$

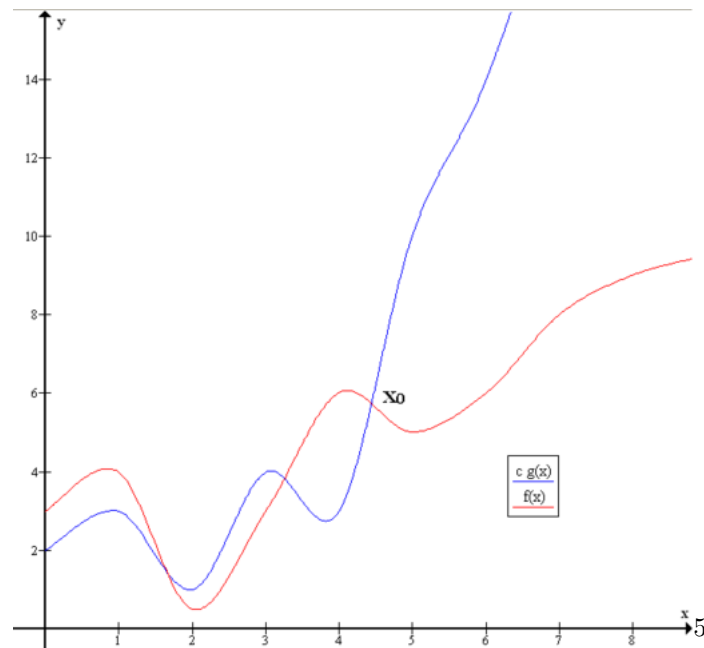
Q: Okay, what's the actual rigorous definition of the “Big- O ” set?

A: $O(g(n))$ is the set of all functions, $f(n)$, such that:

$$\exists c > 0, n_0 \geq 0 : \forall n \geq n_0 : f(n) \leq c \cdot g(n)$$

4

⁴It is common to say n “not” instead of n “zero” for n_0 . Gotta save that syllable!



In the above picture, assuming the functions never intersect again, it seems like $\forall n \geq 5 : f(n) \leq c \cdot g(n)$. The “assuming the functions never intersect again” isn’t enough, though. We need to show that end behavior by proving it using the language above.

Q: How hard is it to prove that a function is in the Big- O of another function?

A: It’s not too bad... the trick is to make the proof as completely obvious as possible.

⁵Taken from the Wikipedia page on Big- O : https://en.wikipedia.org/wiki/Big-O_notation. w:it:User:Fede_Reghe - Screenshot from Graph Software, Public Domain, <https://commons.wikimedia.org/w/index.php?curid=12096768>

Q: How?

Two parts:

A:

- Find appropriate c and n_0 that satisfy Big- O definition.
- Prove that they work $\forall n \geq n_0$.

Let's try to do the second part first.

Q: How do we show that?

We're going to show that, $\forall n \geq n_0 : f(n) \leq c \cdot g(n)$. We can do this by showing a sequence of inequalities of formulas, e.g.:

A:

$$f(n) \square \dots \square \dots \square \dots \square c \cdot g(n)$$

As long as each of those $\square$ (in)equalities are one of either $\leq$, $<$, or $=$.

Important, it looks nicer to write something like that in multiple lines. E.g.:

$$\begin{aligned}
\forall n \geq n_0 : f(n) &\square \dots \\
&\square \dots \\
&\square \dots \\
&\square \dots \\
&= c \cdot g(n)
\end{aligned}$$

We can also put reasons for complicated steps to the right of those steps, e.g.:

$$\begin{aligned}
\forall n \geq n_0 : f(n) &\square \dots, \text{ because blahblahblah} \\
&\square \dots \\
&\square \dots, \text{ because blahblah} \\
&\square \dots, \text{ because blâhaj} \\
&= c \cdot g(n)
\end{aligned}$$

Let's do an example! The $f(n) = 8n^4 + 5n^3 + 80$ from before. We want to show that that's in $O(n^4)$, so $g(n) = n^4$. Here's what the finished proof might look like:

Let $n_0 = 1$ and $c = 93$. Then:

$$\begin{aligned}
\forall n \geq n_0 : f(n) &= 8n^4 + 5n^3 + 80 \\
&\leq 8n^4 + 5n^4 + 80n^4, \text{ because } n \geq 1 \\
&= 93n^4 \\
&= c \cdot g(n)
\end{aligned}$$

Thus, $8n^4 + 5n^3 + 80 \in O(n^4)$.

When I'm going through the steps, I don't start off by guessing c and n_0 . I figure those out at the end (though I got back

and write them up top for the proof). My steps look more like the following. First I set things up:

Let $n_0 = \square$ and $c = \square$. Then:

$$\forall n \geq n_0 : f(n) = 8n^4 + 5n^3 + 80$$

Then I manipulate the formula until I get a multiple of $g(n)$:

Let $n_0 = \square$ and $c = \square$. Then:

$$\begin{aligned} \forall n \geq n_0 : f(n) &= 8n^4 + 5n^3 + 80 \\ &\leq 8n^4 + 5n^4 + 80n^4, \text{ because } n \geq 1 \\ &= 93n^4 \\ &= c \cdot g(n) \end{aligned}$$

Now n_0 needs to be the biggest of all the things I stipulated on the right. (This time there's only one thing, but it can get more complicated.) And then c needs to be the coefficient I got at the end. Plug those back in up top to finish it off:

Let $n_0 = 1$ and $c = 93$. Then:

$$\begin{aligned} \forall n \geq n_0 : f(n) &= 8n^4 + 5n^3 + 80 \\ &\leq 8n^4 + 5n^4 + 80n^4, \text{ because } n \geq 1 \\ &= 93n^4 \\ &= c \cdot g(n) \end{aligned}$$

Thus, $8n^4 + 5n^3 + 80 \in O(n^4)$.

Q: Will $n_0 = 1$ always suffice?

A: No, especially if there are negative terms.

Q: Do any other pair of c and n_0 work?

A:

Q: Do I have to pick the *best* pair that work?

No! It's completely okay to pick a pair that is not "minimal" or anything like that. There are two things you want to accomplish here (and they are very related):

A:

- You want to pick them easily, and
- You want your proof to be *zombie-brain* easy to follow and check.

If possible, do not waste your time optimizing your choice. (Unless you want to spend your time that way.) It is likely to make your proof more complicated.

Q: If it's so easy to pick an n_0 and c that work, why is this such an important concept?

A: Because it's impossible to do when the function isn't in the set. For example, $5n^3 + 10n - 20 \notin O(n^2)$, so there are no such n_0 and c that make it work.

Which of the following contain $20n + 60$?

Q:

- $O(n^3)$
- $O(n^{87})$
- $O(\sqrt{n})$
- $O(1)$

A:

Big-O not the only flavor of asymptotic notation. Also: Big- Ω . Big-O: "Less than or similar" Big- Ω : "Greater than or similar".

- $20n + 60 \in \Omega(n)$, and
- $20n + 60 \in \Omega(1)$, but
- $20n + 60 \notin O(1)$

Q: Rigorous Definition?

A: $f(n) \in \Omega(g(n))$ means:
 $\exists c, n_0 :$
 $\forall n \geq n_0 : f(n) \geq c \cdot g(n)$
 (Can choose $c < 1$ to make $g(n)$ smaller.)

Q: Can we show that $8n^4 + 5n^3 + 80 \in \Omega(n^4)$ now?

A: Yes, it's very similar, but our inequalities are going in the opposite direction ($>$, $\geq$, and $=$ instead). We may also have to use different tricks.

Here's the finished version of a proof that $8n^4 + 5n^3 + 80 \in \Omega(n^4)$:

Let $n_0 = 1$ and $c = 8$. Then:

$$\begin{aligned} \forall n \geq n_0 : f(n) &= 8n^4 + 5n^3 + 80 \\ &\geq 8n^4, \text{ because } n \geq 1 \\ &= c \cdot g(n) \end{aligned}$$

Thus, $8n^4 + 5n^3 + 80 \in \Omega(n^4)$.

(I did the same thing as before and picked my n_0 and c after going through the steps.)

Q: How do you find the c and n_0 here? (E.g. on polynomials?)

A: If all coefficients are positive, it's pretty easy. Just let c be the minimum of 1 and the top coefficient.

For example, to show that $\frac{1}{4}n^2 + 10n + 2 \in \Omega(n^2)$, we should shoot for super simple steps that even a bran muffin could follow: Let $c = \frac{1}{4}$ and $n_0 = 1$. (Again, I chose those after doing the work out below.)

$$\begin{aligned} \forall n \geq n_0 : f(n) &= \frac{1}{4}n^2 + 10n + 2 \\ &> \frac{1}{4}n^2 \text{ Since } n \geq 1 \\ &= c \cdot g(n) \quad \checkmark \end{aligned}$$

Q: What if one of the coefficients is negative? (Note: it won't be the first coefficient.)

A: Use half the c of the prior case, combine the remaining negative coefficients and let n_0 be that sum times $\frac{1}{c}$.

For example, to show that $\frac{1}{5}n^3 - 42n^2 + 8n - 22 \in \Omega(n^3)$, pick $c = \frac{1}{10}$ and $n_0 = 640 = (42 + 22) \times 10$. This is slightly more complicated, but a diseased skunk could still follow these steps:

$$\begin{aligned}
\forall n \geq n_0 : f(n) &= \frac{1}{5}n^3 - 42n^2 + 8n - 22 \\
&> \frac{1}{5}n^3 - 42n^2 + 8n - 22, \text{ since } n \geq 1 \\
&> \frac{1}{5}n^3 - 42n^2 - 22n^2, \text{ since } n \geq 1 \\
&= \frac{1}{5}n^3 - 64n^2 \\
&= \frac{2}{10}n^3 - 64n^2 \\
&= \frac{1}{10}n^3 + \frac{1}{10}n^3 - 64n^2 \\
&= \frac{1}{10}n^3 + \left(\frac{1}{10}n - 64\right)n^2 \\
&= \frac{1}{10}n^3 + \left(\frac{n}{10} - \frac{640}{10}\right)n^2 \\
&= \frac{1}{10}n^3 + \frac{n - 640}{10}n^2 \\
&> \frac{1}{10}n^3, \text{ since } n \geq 640 \\
&= c \cdot g(n) \quad \checkmark
\end{aligned}$$

Which of these contain $20n + 60$?

Q:

- $\Omega(n^{87})$
- $\Omega(n^3)$
- $\Omega(n^2)$
- $\Omega(n)$
- $\Omega(\sqrt{n})$
- $\Omega(1)$

A:

Sometimes more specific: Θ : "Exactly Similar"

Example: $5n^2 \in \Theta(n^2)$ and $\in \Theta(4n^2)$, but not in $\Theta(5n)$ or $\Theta(6n^6)$.

Q: Rigorous?

A:

Also: little- o : “Strictly less than”.

Q: Rigorous?

A:

Little- o used in CS, also Elsewhere to show slow function growth.

Q: Which of these defines equivalence classes? (Of polynomials with all positive coefficients.)

A:

Q: When is problem $P \in O(g(n))$?

A:

Q: Which problems do you know in $O(n)$?

A:

Q: What about in $\Omega(n)$?

Q: What about $O(n^2)$?

A:

There are many nice properties of asymptotic notations!

Theorem! $t(n) + s(n) \in O(\max t(n), s(n))$ (Also true For Θ !)

Q: Proof?

A:

Q: Is the opposite true? $\max t(n), s(n) \in O(t(n) + s(n))$?

A:

If t, s are non-negative-valued functions, then $\max t(n), s(n) \in O(t(n) + s(n))$

Check out proof in book: Page 56

Q: What is the importance of the following theorem?
 $\forall i, j > 1 : \log_i(n) \in O(\log_j(n))$

A:

Often-considered complexities:

- 1 (constant)
- $\log(n)$ (logarithmic)
- n (linear)
- $n \log(n)$
- n^2 (quadratic)
- n^3 (cubic)
- 2^n (exponential)
- n (factorial)

Sometimes, more general:

Polynomial, polylogarithmic, pseudopolynomial. Expand.

Exercises for 2.2**Exercise 0**

Using our asymptotic notation definitions:

- a) Show that $20n^3 + 21n^2 + 22n + 23 \in O(n^3)$
- b) Show that $20n^3 + 21n^2 + 22n + 23 \in \Omega(n^3)$
- c) Use the previous two parts to show that $20n^3 + 21n^2 + 22n + 23 \in \Theta(n^3)$

(Answer 2.2.0 in Appendix)

Exercise 1

Using our asymptotic notation definitions:

- a) Show that $3n^3 - 10n^2 - 5n + 10 \in O(n^3)$
- b) Show that $3n^3 - 10n^2 - 5n + 10 \in \Omega(n^3)$
- c) Use the previous two parts to show that $3n^3 - 10n^2 - 5n + 10 \in \Theta(n^3)$

(Answer 2.2.1 in Appendix)

Exercise 2

Using our asymptotic notation definitions:

- a) Show that $48n^5 \in O(n^5)$
- b) Show that $48n^5 \in \Omega(n^5)$
- c) Use the previous two parts to show that $48n^5 \in \Theta(n^5)$

Exercise 3

Use the definitions of big- O and big- Ω to show that $f(n) = 10n^2 + 18n + 5 \in \Theta(n^2)$.

Exercise 4

Use the definitions of big- O and big- Ω to show that $f(n) = 7n^2 - 10n + 70000 \in \Theta(n^2)$.

Exercise 5

Use the definitions of big- O and big- Ω to show that $f(n) = 12n + \sum_{i=1}^n 17i \in \Theta(g(n))$ for some specific function (that you choose) $g(n)$.

Exercise 6

Simplify each of the following expressions.

a) $\Omega(n^4) \cap \Theta(n^2)$

b) $\Omega(n^4) \cap \Theta(n^5)$

c) $O(n) \cup O(n^3)$

(Answer 2.2.6 in Appendix)

Exercise 7

Simplify each of the following expressions.

a) $\Omega(n^4) \cap O(n^4)$

b) $\Theta(n^4) \cap \Theta(n^3)$

c) $\Omega(n^4) \cup \Omega(n^2)$

Exercise 8

Simplify each of the following expressions.

- a) $O(n^4) \cap \Theta(n^9)$
- b) $\Omega(n^2) \cup \Theta(n^2)$
- c) $\Omega(n) \cap \Omega(n^2) \cap \Omega(n^3) \cap \Omega(n^4)$

2.3 Data Structures Review

In this section, I’m going to assume we’re coding the following data structures in the “proper” way. Just to be certain we’re on the same page, let’s run through the basic data structures. In the end we’ll summarize, looking at **add**, **remove**, **get**, and **contains**, which are different (and sometimes different) for each data structure.

Q: What are the important operations of an array?

A:

- **get(index)**, usually written in algorithmic notation (and code) as $A[i]$
- **set(index, value)**, as $A[i] \leftarrow v$

Q: How long does each of those take to execute?

A: They are both in constant time: $\Theta(1)$.

Q: What is the biggest limitation of arrays?

A: They have a fixed size when created.

Q: What is a Dynamic array?

A: It is similar to an array that can be resized by adding and removing elements.

Q: What are the natural operations of a dynamic array?

A:

- `get(index)`
- `set(index, value)`
- `add(value)`: adds to the end
- `add(index, value)`: adds at given index
- `delete(index)`
- `contains(value)`

Q: How is a dynamic array implemented?

A: It keeps track of an underlying array of elements as well as its logical size. That array may have extra unused cells at its end.

Q: Why does it keep that extra space?

A: This allows it to add new elements without creating a new array underlying array.

Q: Okay, so how does `add` work?

A: If the underlying array is full (the logical size equals the array's size) then it creates a new array of twice the size, and copies all the elements over, then adds the new element. Otherwise, it just adds the new element to the existing array. In both cases, it updates the logical size.

Q: What is the running time of `add` then?

A: It's complicated! I usually cheat by saying: Often $\Theta(1)$, seldom: $\Theta(n)$.

Q: That is cheating! You said we're doing worst-case analysis!

A: Yes! However, here it helps to note that adding a bunch of elements to a dynamic list runs pretty fast!

Q: How long does it take to add $n = 2^k$ elements to a dynamic list that starts empty?

We can write this as the sum of two sums: one that has all the times spent copying over into new arrays, and one with adding the single new elements:

A:

$$\begin{aligned}\sum_{i=1}^n \Theta(1) + \sum_{i=0}^k 2^i &= \Theta(n) + 2^{k+1} - 1 \\ &= \Theta(n) + 2 \cdot 2^k - 1 \\ &= \Theta(n) + \Theta(2^k) \\ &= \Theta(n) + \Theta(n) \\ &= \Theta(n)\end{aligned}$$

This is known as *amortized complexity*: the amount of time it takes to execute an operation many times. Although it can be expensive to add a single element, the average case evens things out so that it isn't that bad. This tells us that the average time complexity is in $\Theta(1)$.

Q: Okay, so what are the running times of each of the dynamic array operations?

A:

- `get(index)`: $\Theta(1)$
- `set(index, value)`: $\Theta(1)$
- `add(value)`: Often $\Theta(1)$, seldom: $\Theta(n)$
- `add(index, value)`: Often $\Theta(n - i)$, seldom: $\Theta(n)$
- `delete(index)`: $\Theta(n - i)$
- `contains(value)`: $\Theta(n)$

Let's move on to Linked Lists!

Q: What are the natural operations of a linked list?

Software design note: It's the same as a dynamic array, because both are used to implement lists.

A:

- `get(index)`
- `set(index, value)`
- `add(value)`: adds to the end
- `add(index, value)`: adds at given index
- `delete(index)`
- `contains(value)`

Q: What is the proper way to code a LinkedList?

A: Doubly-linked, with a wrapper class with pointers to the front and back nodes.

Q: Okay, so what are the running times of each of those linked list operations?

A:

- `get(index)`: $\Theta(\min(i, n - i))$
- `set(index, value)`: $\Theta(\min(i, n - i))$
- `add(value)`: $\Theta(1)$
- `add(index, value)`: $\Theta(\min(i, n - i))$
- `delete(index)`: $\Theta(\min(i, n - i))$
- `contains(value)`: $\Theta(n)$

Stacks!

Q: What are the natural operations of a stack?

A:

- `push(value)`
- `peek()`
- `pop()`

Q: What's the proper way to implement a Stack?

A:

With a linked list.

Q: Then what are the running times of each of those operations?

A:

- `push(value)`: $\Theta(1)$
- `peek()`: $\Theta(1)$
- `pop()`: $\Theta(1)$

Q: How long would the running time be for a `contains` on a Stack?

A: $\Theta(n)$

Queues!

Q: What are the natural operations of queues?

A:

- `add(value)` (AKA `push`, `offer`, `enqueue`)
- `peek()` (AKA `element`, `get`)
- `remove()` (AKA `pop`, `dequeue`)

Q: What's the proper way to implement a queue?

A: Also a linked list.

Q: Then what are the running times of each of those operations?

A:

- `add(value)`: $\Theta(1)$
- `peek()`: $\Theta(1)$
- `remove()`: $\Theta(1)$

Q: How long would the running time be for a `contains` on a Queue?

A: $\Theta(n)$

Priority Queues!

Q: What are the natural operations of priority queues?

A:

- `add(value)`
- `peek()`
- `remove()`

Q: What's the proper way to implement a queue?

A: A heap!

Q: Then what are the running times of each of those operations?

A:

- `add(value)`: $\Theta(\log(n))$
- `peek()`: $\Theta(1)$
- `remove()`: $\Theta(\log(n))$

Sets!

Q: What are the natural operations of sets?

- `add(value)`

A:

- `remove(value)`
- `contains(value)`
- `subset(set) ?`

Q: What are two common ways to implement sets?

A: Self-balancing Binary Search Trees and Hash Tables.

Q: What's the running times for these if we're using a self-balancing BST?

- `add(value)`: $\Theta(\log(n))$

A:

- `remove(value)`: $\Theta(\log(n))$
- `contains(value)`: $\Theta(\log(n))$
- `subset(set)`: $\Theta(m \log(n))$

Q: What's the running times for these if we're using a Hash Table?

A:

- `add(value)`: $\Theta(1)$
- `remove(value)`: $\Theta(1)$
- `contains(value)`: $\Theta(1)$
- `subset(set)`: $\Theta(m)$

Q: So why have you separated them into two separate rows?

Two reasons:

A:

- Space considerations. Hash Tables should be implemented with a very large array, significantly larger than n , e.g. $\Omega(n^2)$. BSTs need exactly n nodes. (For more information on Hash Tables, see section 7.3.)
- We aren't 100% certain that $P \neq NP$. If they are equal, then these operations require $\Theta(n)$ time instead of $\Theta(1)$. (For more information about P vs NP , see section 11.3.)

Let's put it all together!

Teaching note: You may want to transpose this table for

writing on the board.

Operation:	add	remove	get	contains
Dynamic Array (ArrayList)	Often: ? Seldom: ? At index i : ?	?	?	?
Linked List	? ?	? ?	?	?
Stack	?	?	?	?
Queue	? ?	? ?	?	?
Priority Queue	Least Priority: ? o/w: ?	? ?	? ?	? ?
Binary Search Tree	?	?	$\emptyset$	?
Hash Table	?	?	$\emptyset$	?

Q: Let's recall the running times of basic data structure operations! Fill in the above table using big- Θ notation.

A:

This table is not the end-all for determining running times. There are always special cases where running times may be faster. For example, for **contains** on sorted Dynamic Lists, the running time will be $\Theta(\log(n))$ instead of $\Theta(n)$. You must be creative and inquisitive to determine whether there's a faster way to do something.

Exercises for 2.3

Exercise 0

How long does it take to create a new binary search tree of integers and add the number 5 to it n times? (Assume your tree

is able to hold multiple copies of the same value.) Write out the code if that helps you analyze the running time.

(Answer 2.3.0 in Appendix)

Exercise 1

How long does it take to create a new `LinkedList` and add n values to it? Write out the code if that helps you analyze the running time.

Exercise 2

How long does it take to remove all elements from a `Queue` of size n and put into an array that starts with n empty cells? Write out the code if that helps you analyze the running time.

Exercise 3

How long does it take to reverse the elements in a queue of n elements? Write out the code if that helps you analyze the running time.

(Answer 2.3.3 in Appendix)

Exercise 4

How long does it take to reverse the elements in a stack of n elements? Write out the code if that helps you analyze the running time.

Exercise 5

Using big- Θ notation, describe how long the following code takes to execute. `stack` is a `Stack<String>` with n elements.

```
Queue<Integer> queue = new Queue<>(0);
while (!stack.isEmpty()) {
    queue.add(stack.pop());
}
```

(Answer 2.3.5 in Appendix)

Exercise 6

Using big- Θ notation, describe how long the following code takes to execute. `queue` is a `Queue<Integer>` with n elements. `INT_MAX` is a `Comparator<Integer>`.

```
Heap<Integer> maxHeap = new Heap<>(INT_MAX);
while (!queue.isEmpty()) {
    maxHeap.add(queue.remove());
}
```

Exercise 7

Using big- Θ notation, describe how long the following code takes to execute. `stack` is a `Stack<String>` with n elements.

```
ArrayList<String> list = new ArrayList<>(0);
while (!stack.isEmpty()) {
    list.add(0, stack.pop());
}
```

Exercise 8

Using big- Θ notation, describe how long the following code takes to execute. `pq` is a `PriorityQueue<Integer>` with n elements. `INT_MIN` is a `Comparator<Integer>`.

```
BinaryTree<Integer> tree = new AVLTree<>(INT_MIN);  
while (!pq.isEmpty()) {  
    tree.add(pq.remove());  
}
```

Exercise 9

Using big- Θ notation, describe how long the following code takes to execute. `list` is an `ArrayList<Integer>` with n elements, sorted from lowest to highest. `INT_MAX` is a `Comparator<Integer>`.

```
Heap<Integer> maxHeap = new Heap<>(INT_MAX);  
while (!list.isEmpty()) {  
    int num = list.get(list.size() - 1);  
    maxHeap.add(num);  
    list.remove(list.size() - 1);  
}
```

2.4 Iterative Analysis

Important: from this point on, we can always use the shortcut to determine the correct asymptotic notation for any function; we no longer have to prove it from the definition!

How do we determine Time Complexity? Mathematical Analysis.

Example: Analysis

Consider the following algorithm:

Input: array a of length n

Output: pair (k, m)

```

1:  $k \leftarrow 0$ 
2: for  $i \in [0 : n - 1]$  do
3:    $k \leftarrow \max(k, a[i])$ 
4: end for
5:  $m \leftarrow 1000000$ 
6: for  $i \in [0 : n - 1]$  do
7:    $m \leftarrow \max(m, a[i])$ 
8: end for

```

To analyze: break it down and count steps:

$$T_1(n) = 1$$

$$T_{2-4}(n) = 3n$$

$$T_5(n) = 1$$

$$T_{6-8}(n) = 3n$$

$$T_9(n) = 1$$

$$T(n) = T_1(n) + T_{2-4}(n) + T_5(n) + T_{6-8}(n) + T_9(n) = 6n + 3 \in O(n).$$

Q: What if we have multiple processors? What steps might we want to do in parallel?

A: Each run of the for loops. (This is actually a terrible answer, but we're going to run with it.)

(I know, I know, there are a ton of mechanical, procedural, and existential questions we have to discuss about this! We'll get to them in a bit.)

Let's use some new terminology for that. A **Forall** loop means that we're going to run each iteration of the loop concurrently. They may be on different processors.⁶

⁶I am inspired to use this notation from Chapel, Cray's HPC programming language.

Q: How can you create and run a new thread in Java?

There may be newer ways (please feel free to fill me in) but here's a basic way of doing it:

:

```
Thread t = new Thread( () -> {  
    int a = 5;  
    int b = 7;  
    int c = a + b;  
    System.out.println("c: " + c);  
});  
t.start();  
//t and this main thread will both run  
:
```

A:

(The above uses the java Thread class: <https://docs.oracle.com/javase/8/docs/api/java/lang/Thread.html>)

It's likely that there is an older genesis of this term.

Q: What if you want that Thread to interact with your code?

It can't return, but you can interact with constants outside the Thread's scope, like this:

```
:  
final List<Integer> box = new  
ArrayList<>();  
Thread t = new Thread( () -> {  
    int a = 5;  
    int b = 7;  
    int c = a + b;  
    box.add(c);  
});  
t.start();  
//t and this main thread will both run  
:  
System.out.println(box);
```

A:

Q: There's a huge problem with this. What is it?

A: It's not clear that `t` will finish executing before `box` gets printed. It might print an empty list or it might print one with the 12 in it.

Q: What do we have to do to wait for `t` to finish?

We can call the `join` method on `t`:

:

```
final List<Integer> box = new  
ArrayList<>();
```

```
Thread t = new Thread( () -> {
```

```
    int a = 5;
```

```
    int b = 7;
```

```
    int c = a + b;
```

```
    box.add(c);
```

```
});
```

```
t.start();
```

```
//t and this main thread will both run
```

:

```
t.join(); //wait for t to complete
```

```
System.out.println(box);
```

A:

Q: This is only one new thread! How do we do a bunch of threads?

We can make a data structure of Threads. Let's say we wanted to pairwise add two Lists, `listA` and `listB`, together:

```
:
final List<Integer> listC = new
ArrayList<>(listA.length);
List<Thread> threads = new
ArrayList<>();
for (int i = 0; i < listA.size(); i++)
{
    final int j = i; //separate j for
each iteration
    Thread t = new Thread( () -> {
        int a = listA.get(j);
        int b = listB.get(j);
        int c = a + b;
        listC.set(j, c);
    });
    threads.add(t);
    t.start();
}
//all threads running
:
for (int i = 0; i < threads.size();
i++) {
    threads.get(i).join();
}
System.out.println(listC);
```

A:

This is not a perfect implementation of the forall instruction we've introduced, but it's pretty good.

Back to the theory!

Q: What happens to the time complexity of the algorithm at the beginning of this section when we change all the for-loops into forall-loops?

A: It may run faster if there are more processors.

Q: How will we denote that?

A: With a new function, $TP(n, p)$ = the running time on input n with p processors.

Q: What is $TP(n, 1)$ equal to?

A: $T(n)$. In parallel computing we have another name for this: $W(n)$, the *work complexity* of a multi-processor algorithm.

Q: What if we have more processors than our algorithm could ever use? (We write this as $TP(n, \infty)$? Is there a name for that?

A: Yes! That is denoted as $S(n)$, the *step complexity* of an algorithm.

Q: How can we use these to express $TP(n, p)$ for any p ?

There is a wonderful theorem known as *Brent's Theorem*, that tells us:

A:
$$\max\left(\frac{W(n)}{p}, S(n)\right) \leq TP(n, p) \leq \frac{W(n)}{p} + S(n)$$

7

For our uses, it will be helpful to state Brent's Theorem using our asymptotic notation. Both of the following must be true:

$$TP(n, p) \in O\left(\frac{W(n)}{p} + S(n)\right)$$

and

$$\begin{aligned} TP(n, p) &\in \Omega\left(\max\left(\frac{W(n)}{p}, S(n)\right)\right) \\ &= \Omega\left(\frac{W(n)}{p} + S(n)\right) \end{aligned}$$

⁷Brent's Theorem is usually defined as simply: $\frac{W(n)}{p} \leq TP(n, p) \leq \frac{W(n)}{p} + S(n)$, but we include the fact that $S(n) \leq TP(n, p)$ must also be true.

Written another way:

$$\begin{aligned} TP(n, p) &\in O\left(\frac{W(n)}{p} + S(n)\right) \cap \Omega\left(\frac{W(n)}{p} + S(n)\right) \\ &= \Theta\left(\frac{W(n)}{p} + S(n)\right) \end{aligned}$$

Q: After we change the (terrible) algorithm above to use foralls, what is the Work Complexity of it?

It's exactly the same as $T(n)$. If you've already done that, you can skip this part:

A:
$$\begin{aligned} W(n) &= W_1(n) + W_{2-4}(n) + W_5(n) + W_{6-8}(n) + W_9(n) \\ &= 6n + 3 \\ &\in \Theta(n) \end{aligned}$$

Q: What about $S(n)$?

Since all loops are now foralls, and we're assuming we have "infinite" processors, we can treat each forall loop as though it only runs one time:

A:
$$\begin{aligned} S(n) &= S_1(n) + S_{2-4}(n) + S_5(n) + S_{6-8}(n) + S_9(n) \\ &= 1 + 3 + 1 + 3 + 1 \\ &= 9 \\ &= \Theta(1) \end{aligned}$$

Q: Okay, the Work is linear and the Steps is constant. How long would this take if we had n processors?

Use Brent's Theorem!

A:

$$\begin{aligned} TP(n, n) &\in \Theta \left(\frac{W(n)}{n} + S(n) \right) \\ &= \Theta \left(\frac{\Theta(n)}{n} + \Theta(1) \right) \\ &= \Theta(\Theta(1) + \Theta(1)) \\ &= \Theta(\Theta(1)) \\ &= \Theta(1) \end{aligned}$$

Q: Let's do one more example with $\sqrt{n}$ processors.

A:

- | | |
|-----------|--|
| Q: | Should we always carefully count every operation when we analyze the time for an algorithm? |
| A: | No! That kind of analysis just takes way too long. Well, actually... maybe. This is complicated. |

Here are my extended thoughts on this:

- Usually there is one or two important operations that are being executed the most often (or are more expensive than other operations being executed, for one reason or another) and it's best to count those instead of everything.
- If you pick the “wrong” operation, then you may get a wrong answer that is too low. If you're really uncertain, then just go with everything. With practice, it will become easier to pick the most appropriate operation.
- For some problems (e.g. homework and test exercises) you may be asked to count a specific operation. In that case, the choosing part has been done for you. Hooray!
- It is very alluring to just count the depth of the deepest loop. A lot of people do this. Sometimes (often) it's right on. Sometimes it's not, and there's actually something more expensive you need to catch on to.
- Be careful! There are some times when you're dealing with a specific situation in a process and the running time of some function is cheaper than its worst case. There are multiple papers written where someone analyzes a previous researcher's algorithm and finds that the running time is

better than what was stated. (Yet another reason that big- O can be safer to use than big- Θ in some circumstances.)

Q: Let's go back to the first algorithm we "parallelized". Is that a good algorithm?

A:

8

Q: Does it work?

A:

Adding processors is easy! Getting them all to manipulate data is hard. Four PRAM paradigms! $\{E, C\}R\{E, C\}W$

CR is easy; CW is harder. Have to be careful, and less mechanically feasible.

Q: $TP(n, n)$ is an often-considered configuration. Why?

A: It's a common goal. It can signify that a problem is *embarrassingly parallelizable*.

⁸No, parallelized is not a word. Yes, I will use it throughout the semester.

Q: How would you know when a problem is embarrassingly parallelizable?

A: If $TP(n, n) \in \Theta(TP(n)/n)$, or is very close.

Q: Do we ever actually have n processors?

A: Not really. It's almost better to think of as a limit, e.g. $TP(n, n) = \lim_{p \rightarrow n} T(n, p)$

Q: Should we be concerned about reading and writing to the same variable/array cell with multiple threads?

A: Very concerned.

Consider the following algorithm:

Algorithm: **OneAndTwo**

Input: nothing

Output: A , an array with two elements

```
1:  $A \leftarrow [1, 5]$ 
2: for all  $i \in [0, 1]$  do
3:    $A[i] \leftarrow A[(i + 1) \bmod 2]$ 
4: end for
```

Q: What can the output of this algorithm be? (Hint: there are multiple possible answers.)

There are three possibilities:

- A:**
- $[5, 1]$
 - $[1, 1]$
 - $[5, 5]$

Q: How is that possible?

There are two things to understand here:

- A:**
- Each of those statements inside the forall are running in parallel. That means we can't make any assumptions about when they will execute with relation to each other.
 - Each of those statements is a high-level operation that actually breaks down into two (or more, actually) operations:
 - An evaluation of the value fetched from the array, and
 - An assignment back into the array.

(Let's leave it as those two for simplicity. That's enough to create the different outcomes.)

Let's model the two threads as sequences of events:

- Thread $i = 0$: $\underbrace{r_0 \leftarrow A[1]}_{\textcircled{1}} \rightarrow \underbrace{A[0] \leftarrow r_0}_{\textcircled{2}}$
- Thread $i = 1$: $\underbrace{r_1 \leftarrow A[0]}_{\textcircled{3}} \rightarrow \underbrace{A[1] \leftarrow r_1}_{\textcircled{4}}$

(You can think of the r variables as local to that thread or as registers on different CPUs if you want to go even lower-level.)

Q: Assuming no two events happen simultaneously, what are the possible orderings of events ①, ②, ③, and ④?

A:

- ① → ② → ③ → ④
- ① → ③ → ② → ④
- ① → ③ → ④ → ②
- ③ → ① → ② → ④
- ③ → ① → ④ → ②
- ③ → ④ → ① → ②

9

Q: What are the return values of each of those six orderings?

A:

- ① → ② → ③ → ④: [5, 5]
- ① → ③ → ② → ④: [5, 1]
- ① → ③ → ④ → ②: [5, 1]
- ③ → ① → ② → ④: [5, 1]
- ③ → ① → ④ → ②: [5, 1]
- ③ → ④ → ① → ②: [1, 1]

⁹The possibility of simultaneous events is both a practical and philosophical question and beyond the scope of what we can get into deeply in this course!.

Note: we're not making any claims about the probabilities of each of these outcomes.

Q: Is there a name for these kinds of different outcomes?

A: Yes! When the outcome of a program can change due to the timing of threads, this is known as a *race condition*.

Q: In this course, we'll mainly concern ourselves with one (big) heuristic for avoiding race conditions. What is that one heuristic?

A: Don't both read and write from the same place(s) in multiple threads. It's okay if you're reading from (a) different place(s) than you're writing, but if you're reading and writing from the same place(s), then you're putting your code at risk of race conditions.

I like to call that the *Silver Heuristic*: Don't both read and write simultaneously from the same place in multiple threads.

Q: Is there a term for code that cannot cause race conditions?

A: Yes! Code that does not have any race conditions is *thread safe*.

Let's do another one. Consider the following algorithm, which

aims to shift all elements in an array right one cell in parallel.
(The last element is just dropped.)

Algorithm: **BadArrayShift**

Input: array A

Output: A itself

```
1: for all  $i \in [0, \dots, A.length - 2]$  do  
2:    $A[i + 1] \leftarrow A[i]$   
3: end for
```

Q:

What are the possible states of $A = [1, 2, 3, 4]$ after executing **BadArrayShift**(A)?

A:

Q: Can we rewrite the array shift to avoid race conditions *and* still have $TP(n, n) \in \Theta(1)$?

A:

Q: Why does this solve the problem?

A:

Note: In the following algorithm, the circled numbers represent potential places to change a for loop into a forall loop.

Algorithm: **MatrixSort**(A)

Input: array $A[0 : n - 1]$

Output: A itself

```
1: Let  $M[0 : n - 1][0 : n - 1]$ 
2: for ①  $i \in [0 : n - 1]$  do
3:   for ②  $j \in [0 : n - 1]$  do
4:     if  $A[i] > A[j]$  or  $(A[i] = A[j] \text{ and } i > j)$  then
```

```
5:          $M[i][j] \leftarrow 1$ 
6:     else
7:          $M[i][j] \leftarrow 0$ 
8:     end if
9: end for
10: end for
11: Let  $S[0 : n - 1]$ 
12: for ③  $i \in [0 : n - 1]$  do
13:      $S[i] \leftarrow 0$ 
14:     for ④  $j \in [0 : n - 1]$  do
15:          $S[i] \leftarrow S[i] + M[i][j]$ 
16:     end for
17: end for
18: Let  $B[0 : n - 1]$ 
19: for ⑤  $i \in [0 : n - 1]$  do
20:      $B[i] \leftarrow A[i]$ 
21: end for
22: for ⑥  $i \in [0 : n - 1]$  do
23:      $A[S[i]] \leftarrow B[i]$ 
24: end for
```

Q:

What is the running time of this algorithm? Note: whenever I mention running time and don't mention the number of processors, I am always assuming a single processor.

A:

$\Theta(n^2)$. The two “deepest” loops both run n^2 times and don't call any functions inside of them that need separate analysis.

Q: Which for-loops can we safely turn into forall loops without causing race conditions? (Safe means no race conditions are introduced.)

A:

Q: After we parallelize all loops that we safely can, what is $S(n)$?

A: $\Theta(n)$

Q: What is $TP(n, n)$?

A:

Q: What is $TP(n, n^2)$?

A:

Exercises for 2.4

Exercise 0

Consider the algorithm, H, below:

Algorithm: H(n)

Input: integer n

Output: integer s

```
1:  $s \leftarrow 0$ 
2: for  $i \in [1 : n]$  do
3:    $s \leftarrow s + i$ 
4: end for
```

- Find the running time of H using big- Θ notation in terms of n .
- Write out the formula to calculate the return value in terms of the parameter. (Hint: summation!)

- c) That formula is very common and has a nice simplification. If you don't know it, look it up online. Show the simplification and list where you found the simplification of the formula.
- d) Write a new algorithm, $J(n)$, to calculate and return the same value using the simplified formula. Calculate the running time of that formula.

(Answer 2.4.0 in Appendix)

Exercise 1

Consider the algorithm, H, below:

Algorithm: $H(n)$

Input: integer n

Output: integer s

```
1:  $s \leftarrow 0$ 
2: for  $i \in [0 : n]$  do
3:    $s \leftarrow s + 2^i$ 
4: end for
```

- a) Find the running time of H using big- Θ notation in terms of n .
- b) Write out the formula to calculate the return value in terms of the parameter. (Hint: summation!)
- c) That formula is very common and has a nice simplification. If you don't know it, look it up online. Show the simplification and list where you found the simplification of the formula.

- d) Write a new algorithm, $J(n)$, to calculate and return the same value using the simplified formula. Calculate the running time of that formula.

Exercise 2

Consider the algorithm, H, below:

Algorithm: $H(n)$

Input: integer n

Output: integer s

```
1:  $s \leftarrow 0$ 
2: for  $i \in [1 : n]$  do
3:    $s \leftarrow s + i^2$ 
4: end for
```

- a) Find the running time of H using big- Θ notation in terms of n .
- b) Write out the formula to calculate the return value in terms of the parameter. (Hint: summation!)
- c) That formula is very common and has a nice simplification. If you don't know it, look it up online. Show the simplification and list where you found the simplification of the formula.
- d) Write a new algorithm, $J(n)$, to calculate and return the same value using the simplified formula. Calculate the running time of that formula.

Exercise 3

Consider an algorithm with these two parallel running times:

- $TP(n, 1) = 10n^4 + 8n^3 \log(n) + 10n^2$
- $TP(n, \infty) = 121n^2$

Use Brent's Theorem to find $TP(n, n^3)$.
(Answer 2.4.3 in Appendix)

Exercise 4

Consider an algorithm with two parallel running times:

- $TP(n, 1) = n^2$
- $TP(n, \infty) = 10n \log(n)$

Use Brent's Theorem to find $TP(n, n)$.

Exercise 5

Consider an algorithm with two parallel running times:

- $TP(n, 1) = 21n^8$
- $TP(n, \infty) = n^2 \log(\log(n))$

Use Brent's Theorem to find $TP(n, n^6)$.

Exercise 6

Consider the algorithm, H, below:

Algorithm: H(n)

Input: arrays A , B , of same length

Output: arrays A , B

```

1: for all  $i \in [0 : \text{length}(A) - 2]$  do
2:    $A[i + 1] \leftarrow B[i]$ 
3:    $B[i + 1] \leftarrow A[i]$ 
4: end for
5: for all  $i \in [0 : \text{length}(A) - 1]$  do
```

```

6:    $A[i] \leftarrow A[i] - 1$ 
7: end for

```

- a) Run H on the arrays $A = [1, 2, 3]$, $B = [11, 12, 13]$. Are there multiple possibilities for the value the output can have? If so, list at least two of them. Is there a race condition in H on general inputs?
- b) If there's a race condition, rewrite H by changing the loops that cause race conditions to iterative loops instead of parallel loops.
- c) Find $W(n)$ and $S(n)$ of H (after you rewrote, if you had to).
- d) Find $TP(n, n)$.

(Answer 2.4.6 in Appendix)

Exercise 7

Consider the algorithm, H , below:

Algorithm: $H(n)$

Input: arrays A , B , of same length

Output: arrays A , B

```

1:  $n \leftarrow \text{length}(A)$ 
2: for all  $i \in [0 : n - 1]$  do
3:   if  $i \bmod 2 = 0$  then
4:      $A[i] \leftarrow B[i]$ 
5:   else
6:      $B[i] \leftarrow A[i]$ 
7:   end if
8: end for
9: for all  $i \in [0 : \lfloor n/2 \rfloor]$  do

```



```

10:    $A[i] \leftarrow B[n - 1 - i]$ 
11:    $B[i] \leftarrow A[n - 1 - i]$ 
12: end for

```

- a) Run H on the arrays $A = [1, 2, 3]$, $B = [11, 12, 13]$. Are there multiple possibilities for the value the output can have? If so, list at least two of them. Is there a race condition in H in general inputs?
- b) If there's a race condition, rewrite H by changing the loops that cause race conditions to iterative loops instead of parallel loops.
- c) Find $W(n)$ and $S(n)$ of H (after you rewrote, if you had to).
- d) Find $TP(n, n)$.

Exercise 8

Consider the algorithm, H , below:

Algorithm: $H(n)$

Input: arrays A , B , of same length

Output: arrays A , B

```

1:  $n \leftarrow \text{length}(A)$ 
2: for all  $i \in [0 : n - 1]$  do
3:   if  $i \bmod 2 = 0$  then
4:      $A[i] \leftarrow B[i]$ 
5:   else
6:      $B[i] \leftarrow A[i]$ 
7:   end if
8: end for
9: for all  $i \in [0 : n - 1]$  do

```

```
10:    $A[i] \leftarrow A[n - 1 - i]$   
11: end for
```

- a) Run H on the arrays $A = [1, 2, 3]$, $B = [11, 12, 13]$. Are there multiple possibilities for the value the output can have? If so, list at least two of them. Is there a race condition in H ?
- b) If there's a race condition, rewrite H by changing the loops that cause race conditions to iterative loops instead of parallel loops.
- c) Find $W(n)$ and $S(n)$ of H (after you rewrote, if you had to).
- d) Find $TP(n, n)$.

2.5 Recursive Analysis

Q: What do we do when analyzing recursive functions?

A: We can create a *recurrence relation*, a function defined in terms of itself on smaller subcases.

Q: For example, $\text{factorial}(n)$. Write out the factorial function.

Algorithm: **factorial**(n)

Input: $n \geq 0$

Output: x

A:

```
1: if  $n = 0$  then  
2:    $x \leftarrow 1$   
3: else  
4:    $a \leftarrow \text{factorial}(n - 1)$   
5:    $x \leftarrow a \times n$   
6: end if
```

Q: Write a recurrence relation to express $T(n)$, the running time of our factorial function. (In this example, I counted multiplications.)

A:
$$T(n) = \begin{cases} 0, & n = 0 \\ T(n - 1) + 1, & n > 0 \end{cases}$$

Q: How do we solve recurrence relations?

A: A common, general tool is *backwards substitution* to try and indicate a pattern. In backwards substitution, we keep substituting for the next terms, assuming we never reach the base case.

Let's use it on this recurrence:

$$\begin{aligned}
 T(k) &= T(k-1) + 1 \\
 &= [T(k-2) + 1] + 1 \text{ (substitute)} \\
 &= T(k-2) + 2 \text{ (clean it up)} \\
 &= [T(k-3) + 1] + 2 \text{ (substitute)} \\
 &= T(k-3) + 3 \\
 &= [T(k-4) + 1] + 3 \\
 &= T(k-4) + 4
 \end{aligned}$$

Then we want to look for a pattern after i steps. This one looks pretty easy.

$$\begin{aligned}
 T(k) &= T(k-1) + 1 \\
 &= \vdots \\
 &= T(k-4) + 4 \\
 &\vdots \\
 &= T(k-i) + i
 \end{aligned}$$

And then we want to choose the correct i to get to our base case.

$$\begin{aligned}
 T(k) &= T(k-i) + i \\
 &= T(k-k) + k, \text{ when } i = k \\
 &= T(0) + k \\
 &= k
 \end{aligned}$$

Q: Does this prove that $T(k) = k$?

A: No! Our work, though correct, doesn't actually prove that.

Q: How would we prove this?

A: In order to prove this, we would need to use a *proof by induction*, a proof that something is true for a basecase and an incremental implication is also true.

¹⁰

Proof-by-induction that $T(n) = n$:

- Base case: $n = 0$

$$T(0) = 0✓$$

- Recursive case:

Assume $T(k) = k$. (Inductive Hypothesis)

$$\begin{aligned} T(k+1) &= T(k+1-1) + 1 \\ &= T(k) + 1 \\ &= k + 1✓ \end{aligned}$$

That was admittedly an easy one. There'll be harder ones in the future.

¹⁰I have some Induction resources at: <http://kyleburke.info/inductionRoadMap.php>.

Q: Who knows what the Towers of Hanoi are?

A: The *Towers of Hanoi* is a legend that there is a temple in Hanoi, Vietnam, where monks are moving a stack of 64 successively larger golden discs from one dias to another. There is an extra dias where a discs can be placed temporarily. The tricky issue is that a disc cannot be placed upon a smaller disc. Each day, the monks move one disc, and when they are finished moving all the discs, the world will end.

11

Q: How do you move the discs from one dias to another with only one extra dias to use?

A: I don't even know how to explain this without pictures. I highly recommend you look up any of the nice youtube videos about it! (Sorry! Hopefully someday there will be pictures or something here...) Without images, the basic idea is that to move n discs, you first recursively move the stack of $n - 1$ discs to the extra dias, then you move the base disc to the goal dias, then you (recursively again) move the $n - 1$ stack from the extra dias to the end.

¹¹This legend is a complete fabrication that appears to have been invented by French mathematician Édouard Lucas in 1889. More info: https://en.wikipedia.org/wiki/Tower_of_Hanoi#Origins.

Q: How long does it take to move a stack of n discs from the starting disk to the end?

A: We can write a recurrence! $T(n) =$
$$\begin{cases} 0, & n = 0 \\ 2T(n-1) + 1, & n > 0 \end{cases}$$

Q: It looks very similar to the last one! Will the outcome be similar?

A:

Q: How should we try to solve this?

A: Backwards substitution again.

$$\begin{aligned}T(k) &= 2T(k-1) + 1 \\&= 2[2T(k-2) + 1] + 1 \\&= 4T(k-2) + 2 + 1 \\&= 4T(k-2) + 3 \\&= 4[2T(k-3) + 1] + 3 \\&= 8T(k-3) + 7 \\&= 8[2T(k-4) + 1] + 7 \\&= 16T(k-4) + 15 \\&= \vdots \\&= 2^i T(k-i) + 2^i - 1\end{aligned}$$

Q: Okay, then how do we use that to find a closed form?

A: Figure out what i needs to be to hit the base case and do it out. When $i = k$: $T(k) = 2^k T(k-k) + 2^k - 1 = 2^k - 1$

Q: How can we prove that this closed form is correct?

A: Proof by induction!

Proof by induction that $T(n) = 2^n - 1$:

- Base Case: $n = 0$

$$T(0) = 0✓$$

- Recursive Case: Assume $T(k) = 2^k - 1$. Show that it works for $k + 1$:

$$\begin{aligned}T(k + 1) &= 2T(k + 1 - 1) + 1 \\&= 2T(k) + 1 \\&= 2[2^k - 1] + 1 \\&= 2^{k+1} - 2 + 1 \\&= 2^{k+1} - 1✓\end{aligned}$$

We can apply Brent's Theorem to these running times just like we could for non-recursive functions.

Exercises for 2.5

Exercise 0

Although it's clearly impossible, pretend that the Tower of Hanoi monks can perform the two recursive calls in parallel. Based on this fake scenario, what is the recurrence relation for $S(n)$, the number of parallel steps needed to move n disks? Use Brent's Theorem to find the running time with n processors: $TP(n, n)$.

3 Brute Force

Brute Force Techniques are the Nike of algorithm design: "Just do it!" Often, they are relatively easy to code.

Q: Most Basic Algorithm: how many multiplications to find 6^{12} the “brute force” way?

A:

Q: Will we see a faster one?

A: Later on, yes!

Q: What are some problems brute-force techniques are good at solving?

A:

Q: What else is nice about brute-force algorithms?

A:

Q: Which sorting algorithms are Brute-Force type algorithms?

A:

3.1 Brute Force Sorting

Q: Describe the most blatant, brute-force sorting algorithm you can think of?

A:

Okay, let's write out the actual algorithm for SelectionSort:

Algorithm: **SelectionSort**

Input: array A

Output: array A

```
1: for  $i \in [0, \dots, A.length - 2]$  do
2:    $min \leftarrow i$ 
3:   for  $j \in [i + 1, \dots, A.length - 1]$  do
4:     if  $A[j] < A[min]$  then
5:        $min \leftarrow j$ 
6:     end if
7:   end for
8:    $swapA[i] \leftrightarrow A[min]$ 
9: end for
```

Q: What is the running time of SelectionSort?

A:

Q: How can we parallelize it?

A: We cannot safely parallelize either of the loops by turning them into forall loops. There are some mediocre things we can do, but we haven't learned them yet, so we're not going to talk about them yet.

BubbleSort is also a brute-force sorting algorithm:

Algorithm: **BubbleSort**

Input: array A

Output: array A

```
1: for  $i \in [A.length - 2, \dots, 0]$  do
2:   for  $j \in [0, \dots, i]$  do
3:     if  $A[j] > A[j + 1]$  then
4:        $swap A[j] \leftrightarrow A[j + 1]$ 
5:     end if
6:   end for
7: end for
```

Q: What is the running time of BubbleSort?

A:

Q: Can we turn either of the loops into a forall loop?

A:

Note:

- $2 \lfloor \frac{n-1}{2} \rfloor$ is whichever of $n-1$ and $n-2$ is even, and
- $2 \lfloor \frac{n-2}{2} \rfloor + 1$ is whichever of $n-1$ and $n-2$ is odd.

Algorithm: BubbleSort2

Input: array A

Output: array A

```

1:  $n \leftarrow A.length$ 
2: for  $i \in [n-2, \dots, 0]$  do
3:   for all  $j \in [0, 2, 4, \dots, 2 \lfloor \frac{n-1}{2} \rfloor]$  do
4:     if  $A[j] > A[j+1]$  then
5:        $swapA[j] \leftrightarrow A[j+1]$ 
6:     end if
7:   end for
8:   for all  $j \in [1, 3, 5, \dots, 2 \lfloor \frac{n-2}{2} \rfloor + 1]$  do
9:     if  $A[j] > A[j+1]$  then
10:       $swapA[j] \leftrightarrow A[j+1]$ 
11:    end if
12:   end for
13: end for
```

Q: What is the running time of BubbleSort2?

A:

Q: What is $TP(n, n)$?

A:

Exercises for 3.1

Exercise 0

Execute `SelectionSort` on the list $[3, 1, 2]$. Show how the list is modified with each change (i.e. every time a swap is made).

(Answer 3.1.0 in Appendix)

Exercise 1

Execute `SelectionSort` on the list $["E", "X", "A", "M", "P", "L", "E"]$. (Use normal lexicographical comparisons for strings.) Show how the list is modified every time it changes. (This is an extension of exercise 3.1.8 of Levitin, [3].)

Exercise 2

Execute (sequential) `BubbleSort` on the list $[3, 2, 1]$. Show how the list is modified with each change (i.e. every time a swap is made).

(Answer 3.1.2 in Appendix)

Exercise 3

Execute `BubbleSort` on the list $["E", "X", "A", "M", "P", "L", "E"]$. (Use normal lexicographical comparisons for strings.) Show how the list is modified every time it changes. (This is an extension of exercise 3.1.11 of Levitin, [3].)

Exercise 4

Find $TP(n, n^2)$ for the parallelized version of BubbleSort that we wrote.

(Answer 3.1.4 in Appendix)

Exercise 5

Find $TP(n, \sqrt{n})$ for the parallelized version of BubbleSort.

3.2 Sequential Search

Sometimes a sequential search is the best way to look for a value.

Algorithm: `arrayContains(A)`

Input: A , an array of strings of length n , and e , a value

Output: b , whether A contains e

```

1:  $b \leftarrow \text{false}$ 
2: for all  $i \in [0, \dots, n - 1]$  do
3:   if  $A[i] = e$  then
4:      $b \leftarrow \text{true}$ 
5:   end if
6: end for
```

If the entries of A aren't sorted in any way, then this is the fastest way to search.

Q: What is the worst case (not running time) when running `arrayContains`?

A: e is not in A

Q: What is the running time in that case?

A: $\Theta(n)$

Q: Design an algorithm, `is_substring( $s, x$ )` that determines whether x is a substring of s .

A:

Q: What's the worst case complexity of that?

A:

Q: Can we make any parallelize any of the loops?

A:

Q: What is $TP((n, m), n + m)$?

A:

Exercises for 3.2

Exercise 0

Consider an array of strings.

- a) Write an (optimal) algorithm to determine whether any of those strings equals "monkey".

- b) Find $T(n)$, the running time of your algorithm when the array has length n .
- c) Find $TP(n, n)$, the running time of your algorithm with n processors. (Update your algorithm to run in parallel if you need to.)

(Answer 3.2.0 in Appendix)

Exercise 1

Consider an 2-D array of strings.

- a) Write an (optimal) algorithm to determine whether any of those strings equals "monkey".
- b) Find $T(n)$, the running time of your algorithm when the array has size $n \times \frac{n}{2}$.
- c) Find $TP(n, n)$, the running time of your algorithm with n processors. (Update your algorithm to run in parallel if you need to.)

Exercise 2

- a) Write a sequential search algorithm to determine whether a `Stack<Integer>` of size n has any neighboring numbers that are equal. (Your algorithm has to return the stack to it's original state when it's done.)
- b) Find $T(n)$, the running time of your algorithm when the stack has a size of n .
- c) Find $TP(n, n)$, the running time of your algorithm with n processors. (Update your algorithm to run in parallel if you need to.)

(Answer 3.2.2 in Appendix)

Exercise 3

- a) Write an optimal algorithm to determine whether a `Queue<String>` has two elements that are equal (anywhere). (If you modify the Queue, your algorithm has to return it to its original state when it's done. Hint: use a separate data structure while searching.)
- b) Find $T(n)$, the running time of your algorithm if the queue has n elements.
- c) Find $TP(n, n)$, the running time of your algorithm with n processors. (Update your algorithm to run faster with n processors if it helps it run faster.)

Exercise 4

- a) Write a non-optimal brute force algorithm to determine whether a `Queue<String>` and a `Stack<String>` share any elements. Be sure that the two data structures are in their original state when your algorithm completes.
- b) Find $T((n, m))$, the running time of your algorithm if the queue has n elements and the stack has m .
- c) Find $TP((n, m), n+m)$, the running time of your algorithm with $n+m$ processors. (Update your algorithm to run faster with $n + m$ processors if it helps it run faster.)

3.3 Geometric Algorithms

Let's consider the Nearest-Neighbors problem¹²: given a list of 2-D points, we need to find the two points that are closest. (If there's a tie, then any of the nearest neighbor pairs will do.)

Q: Let's start out by writing a distance algorithm for 2-D points.

A: Algorithm: `distance( $p_1, p_2$ )`
Input: p_1, p_2 , two arrays with two elements each
Output: d
1: $d \leftarrow \sqrt{(p_1[0] - p_2[0])^2 + (p_1[1] - p_2[1])^2}$

¹²This is also known as the Closest Pair problem, but I like the alliteration.

Q: Write a Brute-Force algorithm for this problem.

Algorithm: **NearestNeighbors**(A)

Input: A , a list of points

Output: $p = ((u_x, u_y), (v_x, v_y))$

A:

```
1:  $p \leftarrow (A[0], A[1])$ 
2:  $minDist \leftarrow \text{distance}(A[0], A[1])$ 
3: for  $i \in [0, \dots, A.length - 2]$  do
4:   for  $j \in [i + 1, \dots, A.length - 1]$  do
5:      $dist \leftarrow \text{distance}(A[i], A[j])$ 
6:     if  $dist < minDist$  then
7:        $minDist \leftarrow dist$ 
8:        $p \leftarrow (A[i], A[j])$ 
9:     end if
10:  end for
11: end for
```

Q: What's the running time of this algorithm?

A:

Q: Can we parallelize any of the loops as is?

A: Not as is. We need to be a bit more careful.

Q: Write a thread-safe parallel version.

A:

Q: Can we parallelize that inner for loop?

A: Not safely!

Q: What's $TP(n, n)$ for this algorithm?

A:

If there's time, talk about Convex Hull. (TODO)

Exercises for 3.3

Exercise 0

- a) Find $TP(n, n^2)$ for the parallel version of the nearest neighbors algorithm that we did above/in class.
- b) How does this compare with $TP(n, n)$?
- c) What is the smallest number of processors that gives us the best time?

(Answer 3.3.0 in Appendix)

Exercise 1

- a) Find $TP(n, n^3)$ for the parallel version of the nearest neighbors algorithm that we did above/in class.
- b) How does this compare with $TP(n, n)$?

Exercise 2

- a) Find $TP(n, \sqrt{n})$ for the parallel version of the nearest neighbors algorithm that we did above/in class.
- b) How does this compare with $TP(n, n)$?

3.4 Exhaustive Search

Q: The power set of a set S , $\mathcal{P}(S)$, is the set of all subsets of S . If $|S| = n$, what is $|\mathcal{P}(n)|$?

A:

Q: How can we search all possible subsets of S ?

Two ways:

A:

- Consider each item as one of n boolean digits. Then try each number from 0 to $2^n - 1$ and grab the appropriate subset (not trivial).
- Recursively try the two subsets based on whether the first element is in there or not.

Here's a pythonic version of that second one:

```
def all_subsets(set):
    # uses list operations instead of sets
```



```

    if len(set) == 0:
        return []
    else:
        subsets = []
        #recursively get all subsets of everything but
the first element
        for subset in all_subsets(set[1:])
            subsets.append(subset)
            subsets.append([set[0]] + subset)
        return subsets

```

3.5 Graph Searching

Let's consider searching a graph for a value. A common algorithm to solve this is via a Depth-First Search:

Algorithm: DFS

Input: vertex v , and value $value$, a starting vertex in graph G and the sought value.

Output: b , whether $value$ is on a vertex in G

```

1:  $b \leftarrow false$ 
2: if  $v.marked \neq true$  then
3:      $v.marked \leftarrow true$ 
4:     if  $v.value = value$  then
5:          $b \leftarrow true$ 
6:     else
7:         for  $u \in v.adjacent$  do
8:             if  $DFS(u, value) = true$  then
9:                  $b \leftarrow true$ 
10:                break
11:            end if
12:        end for

```

13: **end if**

14: **end if**

Q: What is $T((n, m))$ of the DFS algorithm when n is the number of vertices in G and m is the number of edges?

A: In order to analyze this easily, we need to get a bit creative with our analysis. In the worst case, each vertex gets marked exactly one time, and all adjacencies are checked. In that case, the algorithm will traverse each edge twice, so $T((n, m)) = \Theta(2m) = \Theta(m)$.

Q: Can we parallelize any part of this algorithm?

A:

Q: If we do that, might we visit any vertices more than once?

A: Yes! It could be that two threads make calls on the same vertex at nearly the same time, then they will both test that the vertex is marked, getting false, then enter that branch.

Q: Okay, what's $S((n, m))$ if we do that?

A: When we visit each vertex, all edges from that vertex will be traversed at the same time. In the worst case, some thread will have to visit all n vertices, so $S((n, m)) = \Theta(n)$.

Q: What is $TP((n, m), n)$?

A:

The other common method of searching a graph is a Breadth-First Search. This form uses a Queue to decide which vertex to visit next:

Algorithm: BFS

Input: vertex v , value $value$

Output: b , whether $value$ is on a vertex in G

1: $b \leftarrow false$

```
2:  $Q \leftarrow \text{new Queue}$ 
3:  $Q.add(v)$ 
4: while  $Q$  is not empty do
5:    $w \leftarrow Q.remove()$ 
6:   if  $w.marked \neq \text{true}$  then
7:      $w.marked \leftarrow \text{true}$ 
8:     if  $w.value = \text{value}$  then
9:        $b \leftarrow \text{true}$ 
10:      break
11:    else
12:      for  $u \in w.adjacent$  do
13:         $Q.add(u)$ 
14:      end for
15:    end if
16:  end if
17: end while
```

Q: What is $T((n, m))$ for this algorithm?

A: In the worst case, we will add each vertex to the queue one time for each other edge it's on. (We could test whether it's marked before we add it to the queue, but those tests still require a step.) In total, that's going to be $\Theta(m)$ steps.

Q: Can we parallelize this at all?

A:

Q: Why can't we parallelize the big loop?

A:

Q: I'm bummed. This doesn't seem very parallel. Why is this okay?

A: The parallel DFS uses a lot of elements of the BFS anyways by trying all the neighbors.

Q: Okay, in the new parallel version we have with the inner forall loop, it seems like we really want to have the test in there so we don't put marked vertices into the Queue.

A: I agree! Now that seems like a very worthwhile change!

Q: What is $S((n, m))$ of our parallel and modified algorithm?

A: The inner loop runs in parallel, so we're really concerned with how many times we will run the outer loop. Since we won't add already-marked vertices, it seems like this is only going to take $\Theta(n)$ time. Unfortunately, if G has a lot of edges (e.g. in a complete graph), then it would still require $\Theta(m)$ time, because the later vertices would still be added $\Theta(n)$ times.

Q: There must be a solution to this! Is there?

A: There is! Instead of marking the vertices when we visit them, we mark them when we add them to the Queue and then prevent them from entering the Queue if they're marked.

Here is the new algorithm:

Algorithm: BFS (improved)

Input: vertex v , value $value$

Output: b , whether $value$ is on a vertex in G

```
1:  $b \leftarrow false$ 
2:  $Q \leftarrow$  new Queue
3:  $v.marked \leftarrow true$ 
4:  $Q.add(v)$ 
5: while  $Q$  is not empty do
6:    $w \leftarrow Q.remove()$ 
```

```
7:   if  $w.value = value$  then
8:        $b \leftarrow true$ 
9:       break
10:  else
11:      for all  $u \in w.adjacent$  do
12:          if  $u.marked = false$  then
13:               $u.marked \leftarrow true$ 
14:               $Q.add(u)$ 
15:          end if
16:      end for
17:  end if
18: end while
```

Q: Great! What's the $S((n, m))$ of that algorithm?

A: Now $S((n, m)) \in \Theta(n)$.

Exercises for 3.5

Exercise 0

- Consider a graph implemented using adjacency lists. Write an optimal algorithm to return whether the graph contains two vertices that are only connected to each other.
- Find $T((n, m))$, the running time of your algorithm if the graph has n vertices and m edges.
- Find $TP((n, m), n)$, the running time of your algorithm with n processors. (Update your algorithm to run faster with n processors if it helps it run faster.)

(Answer 3.5.0 in Appendix)

Exercise 1

- a) Consider a graph implemented using adjacency lists. Write an optimal algorithm to determine whether there is any vertex that is not adjacent to any other vertices.
- b) Find $T((n, m))$, the running time of your algorithm if the graph has n vertices and m edges.
- c) Find $TP((n, m), n)$, the running time of your algorithm with n processors. (Update your algorithm to run faster with n processors if it helps it run faster.)

Exercise 2

- a) Consider a graph implemented using adjacency lists. Write an algorithm to return a sorted array containing the degrees of the vertices. (The degree of a vertex is the number of edges leaving it.) You can use BubbleSort to do your sorting for you.
- b) Find $T((n, m))$, the running time of your algorithm if the graph has n vertices and m edges. Recall that BubbleSort has $T(n) \in \Theta(n^2)$.
- c) Find $TP((n, m), n)$, the running time of your algorithm with n processors. You may need to go back to update your algorithm to run faster with n processors if it helps it run faster. Recall that BubbleSort runs with $TP(n, p) \in \Theta\left(\frac{n^2}{p} + n\right)$.

Exercise 3

- a) Consider a graph implemented using adjacency lists. Write an optimal algorithm to determine whether there is any trio of three vertices that are all connected to each other. (They can be connected to other vertices as well.)
- b) Find $T((n, m))$, the running time of your algorithm if the graph has n vertices and m edges.
- c) Find $TP((n, m), n)$, the running time of your algorithm with n processors. (Update your algorithm to run faster with n processors if it helps it run faster.)

Exercise 4

Write out the parallel version of BFS that we first came up with (not the final version). Remember that in this version, we changed the inner loop to be for all and checked that the vertex was unmarked before adding it to the Queue.

(Answer 3.5.4 in Appendix)

Exercise 5

Write out the final version of BFS that we first came up with.

Exercise 6

Write a version of DFS that uses a Stack instead of recursion.

Exercise 7

Write out a depth-first search algorithm to search for the shortest path from vertex x to vertex y . (Assume all edges have the a weight of 1.)

Exercise 8

Find $TP((n, m), m)$ for the DFS algorithm we came up with in class.

(Answer 3.5.8 in Appendix)

Exercise 9

Find $TP((n, m), \sqrt{n})$ for the DFS algorithm we came up with in class. Hint: Use the fact that Θ is the intersection of O and Ω .

Exercise 10

Find $TP((n, m), \sqrt{m})$ for the DFS algorithm we came up with in class.

Exercise 11

Find $TP((n, m), n)$ for the final BFS algorithm we came up with in class. (I recommend doing 3.5.5 first, but it's not strictly necessary.)

(Answer 3.5.11 in Appendix)

Exercise 12

Why will the parallel running times for our best DFS and BFS be the same for the same number of processors?

4 Decrease and Conquer

Big idea: Use the relationship between a problem's solution and a solution to a smaller instance of the problem. Usually falls in to one of three categories:

- Decrease by a constant

- Decrease by a constant factor
- Decrease by a variable amount

There are often two ways to structure decrease-and-conquer algorithms: top-down and bottom-up. Top-down algorithms are usually recursive, translating a recurrence relation about the problem. Bottom-up are usually iterative.

Let's do some examples to compute 5^n .

Q: How could we solve the problem of finding 5^n in a decrease-and-conquer way where we are decreasing by a constant (of 1)?

A: Use $5^n = 5 \times 5^{n-1}$. More concretely:

$$5^n = \begin{cases} 1 & , \text{ if } n = 0 \\ 5^{n-1} \times 5 & , \text{ if } n > 0 \end{cases}$$

This gives us a recursive solution, e.g.:

$$5^4 = 5 \times (5 \times (5 \times (5 \times 1)))$$

Q: How many multiplications does this take?

A: 4

Q: In general, how many multiplications does it take to compute 5^n this way?

A:

A “top-down” approach generally gives us a recursive solution like this. Alternatively, we could just start here:

$$5^n = 1 \times \overbrace{5 \times 5 \times 5 \times 5 \times \cdots \times 5}^{n \text{ 5s}}$$

Then we have an iterative solution by creating a product variable and multiplying 5 to it over and over. This is more of a “bottom-up” approach. (Yes, there is a lot of overlap between decrease-by-a-constant-and-conquer and brute force algorithms.)

Q: How many multiplications are made here?

A:

There is, however, another way to find powers of 5.

Q: How can I find 5^8 with only four multiplications? Hint: I count four because I go all the way down to zero.

A:

Q: What's the recursive formula now to compute 5^n where n is a power of 2? (With a base case of $n = 0$).

A:
$$5^n = \begin{cases} 1 & , n = 0 \\ (5^{n/2})^2 & , n > 0 \end{cases}$$

Q: Let's improve that now to cover all positive values of n .

A:
$$5^n = \begin{cases} 1 & , n = 0 \\ (5^{n/2})^2 & , n > 0 \text{ and } n \text{ is even} \\ (5^{n-1/2})^2 \times 5 & , n > 0 \text{ and } n \text{ is odd} \end{cases}$$

Let's write out a table to see how this grows.

n	# mults		n	# mults	
0	0	1	8	?	$(5^4)^2$
1	2	$(5^0)^2 \times 5$	9	?	$(5^4)^2 \times 5$
2	3	$(5^1)^2$	10	?	$(5^5)^2$
3	4	$(5^1)^2 \times 5$	11	?	$(5^5)^2 \times 5$
4	4	$(5^2)^2$	12	?	$(5^6)^2$
5	5	$(5^2)^2 \times 5$	13	?	$(5^6)^2 \times 5$
6	5	$(5^3)^2$	14	?	$(5^7)^2$
7	6	$(5^3)^2 \times 5$	15	?	$(5^7)^2 \times 5$
			16	?	$(5^8)^2$

Q: Fill in the second half of the table

A:

Q: What is this kind of decrease-and-conquer algorithm called?

A: decrease by a *constant factor*

Q: Let's express the number of multiplications to calculate 5^n in this way, $T(n)$ as a recurrence relation. What does that look like?

A:
$$T(n) = \begin{cases} 0 & , \text{ if } n = 0 \\ T(n/2) + 1 & , \text{ if } n > 0 \text{ is even} \\ T(n-1/2) + 2 & , \text{ if } n > 0 \text{ is odd} \end{cases}$$

Let's use backsubstitution and see if we can find a closed form for this. We'll simplify things a bit by combining the worst parts of the recursive cases for $T(n)$ above to bound it: $T(n) \leq T(n/2) + 2$. Assume k is "large enough":

$$\begin{aligned}
T(k) &\leq T\left(\frac{k}{2}\right) + 2 \\
&\leq \left[T\left(\frac{k}{4}\right) + 2\right] + 2 \\
&= T\left(\frac{k}{4}\right) + 4 \\
&\leq \left[T\left(\frac{k}{8}\right) + 2\right] + 4 \\
&= T\left(\frac{k}{8}\right) + 6 \\
&\leq \left[T\left(\frac{k}{16}\right) + 2\right] + 6 \\
&= T\left(\frac{k}{16}\right) + 8 \\
&\vdots \\
&\leq T\left(\frac{k}{2^i}\right) + 2i
\end{aligned}$$

We can't reach $T(0)$ by dividing, but we can reach $T(1)$ when $2^i = k$, or $i = \log_2(k)$.

$T(k) \leq T(k/k) + 2\log_2(k) = T(1) + 2\log_2(k) = 2 + 2\log_2(k) \in O(\log_2(k))$. We can use a similar inequality to show that $T(n) \in \Omega(\log_2(n))$, so that $T(n) \in \Theta(\log_2(n))$.¹³

Quickly talk about how logarithms are one way to “undo” exponentiation:

¹³I should turn this second half into an exercise!

$$\begin{aligned}b^x = a &\Leftrightarrow \sqrt[x]{a} = b \\ &\Leftrightarrow \log_b(a) = x\end{aligned}$$

It's really just notation for when we need to solve for x instead of for a .

Q: Why are logs so great?

A: $\Theta(\log(n))$ is WAY faster than $\Theta(n)$. It's even way faster than $\Theta(\sqrt{n})$.

Q: What is another way to reason that there are $\Theta(\log(n))$ multiplications to find 5^n in the way we showed above?

A: Every time we make a recursive call, we have cut the problem size in at least half, and are incurring a cost of at most two multiplications. Thus, we need to know exactly how many times n can be divided by 2 to get to 1.

Q: Why are you using the ceiling?

A: Because n may not be exactly a power of 2. We're only off by 1 in the worst case, so it's fine.

Q: Why is it safe to ignore the base of the logarithm?

A: Because of the change of base formula! As long as $b, c > 1$, $\log_b(a) = \frac{\log_c(a)}{\log_c(b)}$. If c and b are both constants, then $\log_c(b)$ is also a constant.

Q: This algorithm for 5^n is top-down. Is there an iterative bottom-up algorithm for this?

A: Absolutely! First, calculate the list of 5^{2^k} values up to $2^k = \lfloor n \rfloor$. Then rewrite n in binary. For each index in that binary i where there's a 1, you need to multiply those elements of that first list together.

Q: How many multiplications is that for 5^{12} ?

A:

- $\Theta(\log(n))$ to make the list, and
- $O(\log(n))$ to pick out the ones where there are 1s in the binary and multiply them together.

In total: $\Theta(\log(n))$

Talk about Theta notation and logs

I don't have a good example for 5^n for decrease by a vari-

able amount, so... we're going to use Greatest Common Divisor instead.

Q: How were you taught to find GCDs in grade school?

To find $\text{GCD}(x, y)$:

A:

- Find prime factorization of x and y .
- Multiply all common factors.

Q: Let's do it for 210 and 126!

A:

- $210 = 2 \times 3 \times 5 \times 7$
- $126 = 2 \times 3^2 \times 7$

So the GCD has: $2 \times 3 \times 7 = 42$.

Q: Is this a good algorithm? (Hopefully some cybersecurity students can answer this!)

A:

Q: What can we do instead to find the GCD of two numbers?

A: We can use Euclid's Algorithm.

Q: What formula is Euclid's Algorithm based on?

A: $\gcd(m, n) = \gcd(n, m \bmod n)$

(The full recurrence relation of the formula is saved for a homework problem.)

Q: Let's try it on our prior example! Find GCD of 126 and 210.

A:

$$\begin{aligned}\gcd(126, 210) &= \gcd(210, 126) \\ &= \gcd(126, 84) \\ &= \gcd(84, 42) \\ &= \gcd(42, 0) \\ &= 42\end{aligned}$$

Q: What is the GCD of 55 and 154?

A:

Notice that we don't decrease in any normal way; the way the inputs change varies.

- Decrease by a constant algorithms look a lot like Brute Force; there is a subtle difference.
- Decrease by a constant factor algorithms look a lot like divide-and-conquer, which we'll see later.

Q: What is the recurrence relation for $T(n)$ of the top-down decrease-by-a-constant algorithm for finding 5^n we saw earlier? (The one based on $5^n = 5^{n-1} \times 5$)

A:
$$T(n) = \begin{cases} 1 & , n = 0 \\ T(n-1) + 2 & , n > 0 \end{cases}$$

It turns out a lot of decrease-and-conquer algorithms have a recursive case with this form, but a bit more general: $aT(n-b) + f(n)$. If that $f(n) \in \Theta(n^d)$, then the whole thing looks like the following. For some constants $a, b, c, d > 0, n_0 \geq 0$:

$$T(n) = \begin{cases} c & , n \leq n_0 \\ aT(n-b) + \Theta(n^d) & , n > n_0 \end{cases}$$

In these cases, there's a quick way to find the asymptotic notation for $T(n)$!

*Baby Master Theorem*¹⁴: Anytime you have a recurrence with the above form, the asymptotic running time is based on the values of a , b and d :

$$T(n) \in \begin{cases} \Theta(n^d) & , a < 1 \\ \Theta(n^{d+1}) & , a = 1 \\ \Theta(n^d a^{n/b}) & , a > 1 \end{cases}$$

Exercises for 4.0

¹⁴It's not actually called the "Baby Master Theorem" outside of this text, but we'll see the actual Master Theorem in the Divide-and-Conquer chapter and this is like a little prelude to that. If you use the term "Baby Master" outside of this course, people are likely to look at you oddly. In fact, the actual master of a baby is usually the Babysitter, so maybe it should be called the Babysitter Theorem instead...

Exercise 0

$5^n = 5 \times 5^{n-1}$ doesn't completely specify the algorithm because it causes infinite recursion. Use cases to fully describe the function. It should work for all non-negative integers n .

Exercise 1

In our description of Euclid's algorithm above, it doesn't catch the base case. Use two cases to fully describe $\text{gcd}(m, n)$. You can only assume that both n and m are non-negative integers.

4.1 Decrease-and-Conquer Sorting

Q:	What if we need to insert a single element into an array that's already sorted? How can we do that?
-----------	---

A:	Create an array that's one-element larger, then copy all the smaller elements in, then the new element, then the larger elements.
-----------	---

Q:	What if they're already in the same array? Elements 0 through $n - 1$ are sorted, but the n^{th} could go anywhere. What do we do?
-----------	---

A:	Slide that last element down until it's in the right place.
-----------	---

You could make an Algorithm out of this!¹⁵

¹⁵Blatant reference to History of the Entire World, I Guess (<https://www.youtube.com/watch?v=Hh3D7R1Uo64>).

Algorithm: InsertionSort

Input: array A

Output: the same array, A , but sorted

```
1: for  $i \in [1, 2, \dots, A.length - 1]$  do  $x \leftarrow A[i]$ 
2:   for  $j \in [i - 1, i - 2, \dots, 0]$  do
3:     if  $x < A[j]$  then
4:       Swap  $A[j] \leftrightarrow A[j + 1]$ 
5:     else
6:       break
7:     end if
8:   end for
9: end for
```

Q: What is the running time of Insertion Sort?

A: $\Theta(n^2)$

Q: Why?

A:

Q: What is the worst case for InsertionSort?

A:

[com/watch?v=xuCn8ux2gbs](https://www.youtube.com/watch?v=xuCn8ux2gbs)).

Q: What is the best case for InsertionSort?

A: An already-sorted list.

Q: How long does it take on that case and why?

A:

Q: Can we turn the inner loop into a forall?

A: Not really. (You can talk a lot about this!)

Q: What about the outer for loop?

A: Same thing. Not really.

Exercises for 4.1

Exercise 0

Execute `InsertionSort` on the list $[3, 2, 1]$. Show how the list is modified after each step.

(Answer 4.1.0 in Appendix)

Exercise 1

Execute `InsertionSort` on the list $[E, X, A, M, P, L, E]$. (Use normal lexicographical comparisons for strings.) Show how the list is modified every time it changes. (This is an extension of exercise 4.1.7 of Levitin, [3].)

4.2 Topological Sorting

4.3 Generating Combinatorial Objects

4.4 Constant-Factor Decr & Conq Algos

The best example of a decrease-by-a-constant-factor algorithm is one you're probably already familiar with: Binary Search. Let's write this recursively with a helper function:

Algorithm: `BinarySearch`

Input: Array of elements, A , and potential element x

Output: b , indicating whether $x \in A$

1: $b \leftarrow \text{BinarySearchH}(A, x, 0, A.length - 1)$

Algorithm: `BinarySearchH`

Input: Array A , item x , integers i, j

Output: b , indicating whether $x \in A$ in the range $[i, j]$

1: **if** $i \geq j$ **then**

2: $b \leftarrow (A[i] = x)$

3: **else**

4: $m \leftarrow \lfloor (i + j) / 2 \rfloor$

5: **if** $x \leq A[m]$ **then**

6: $b \leftarrow \text{BinarySearchH}(A, x, i, m)$

7: **else**

```

8:          $b \leftarrow \text{BinarySearchH}(A, x, mid + 1, j)$ 
9:     end if
10: end if

```

Q: Can we write a recurrence to count the number of comparisons made by this algorithm?

A: Yes!

$$T(n) = \begin{cases} 1 & , n \leq 1 \\ T(\lfloor \frac{n}{2} \rfloor) + 2 & , n > 1 \end{cases}$$

Q: Can we solve this recurrence?

A:

Q: So what's the running time of $T(n)$?

A: $T(n) \in \Theta(\log(n))$

Ancient Egyptian Multiplication¹⁶

Q: How can we multiply 43×28 ?

A: Expect the normal multiplication scheme.

Q: What's another way to do this that doesn't really use any multiplication?

A: Ancient Egyptian Multiplication!

TODO: finish this!

Exercises for 4.4

Exercise 0

- a) Rewrite `BinarySearch` to be iterative instead of recursive.
- b) Let's write out $T(n)$ but only count comparisons. Fill in the question marks in this formulation for $T(n)$:

$$T(n) = \sum_{i=0}^{\lceil \log(n) \rceil} ??$$

- c) Solve the summation to show that this version of $T(n)$ has the same time complexity as the recursive version.

¹⁶In the Levitin text, this is listed as Russian Peasant Multiplication [3].

Exercise 1

What happens to our running time for `BinarySearch` if we (nonsensically) have to look at both sides, so the recurrence is as follows?

$$T(n) = \begin{cases} 1 & , n \leq 1 \\ 2T(\lfloor \frac{n}{2} \rfloor) + 2 & , n > 1 \end{cases}$$

What is the running time of $T(n)$ now? (If you've already learned the Master Theorem, don't use it.)

(Answer 4.4.1 in Appendix)

Exercise 2

What happens to our running time for `BinarySearch` if the recurrence is as follows?

$$T(n) = \begin{cases} 1 & , n \leq 1 \\ 3T(\lfloor \frac{n}{2} \rfloor) + 2 & , n > 1 \end{cases}$$

What is the running time of $T(n)$ now? (If you've already learned the Master Theorem, don't use it.)

(Answer 4.4.2 in Appendix)

Exercise 3

What happens to our running time for `BinarySearch` if the recurrence has a 4 in the coefficient?

$$T(n) = \begin{cases} 1 & , n \leq 1 \\ 4T(\lfloor \frac{n}{2} \rfloor) + 2 & , n > 1 \end{cases}$$

What is the running time of $T(n)$ now? (If you've already learned the Master Theorem, don't use it.)

4.5 Variable-Size Decr & Conq Algos

TODO: `QuickSelect`

TODO: `InterpolationSearch`

TODO: Searching and Inserting into a BST

5 Divide and Conquer

Basic Plan:

1. Split problem into two subpieces.
2. Solve half-problem A
3. Solve half-problem B
4. Use two solutions to solve whole problem.

Q:	What happens if it's natural to split into more than 2 sub-cases?
-----------	---

A:	That works too!
-----------	-----------------

Q:	There's an obvious application of multiple processors here. What is it?
-----------	---

A:	Use a processor on each of the problems.
-----------	--

For that reason, we need some new algorithmic syntax: **SimultDo** is a compound statement such that each line inside of it is executed concurrently using a separate processor. We'll use this in a bit.

Q: If we treat the above “basic plan” as a recursive algorithm, what form will the recurrence relation take?

A:
$$T(n) = \begin{cases} c & , n \leq 1 \\ 2T\left(\frac{n}{2}\right) + f(n) & , n > 1 \end{cases}$$

Let’s create a divide-and-conquer algorithm to find 5^n . (Note: this will *not* be better than the decrease-by-a-constant-factor and conquer algorithm we devised at the beginning of section 4.)

$$5^n = \begin{cases} 1 & , n = 0 \\ 5 & , n = 1 \\ 5^{\lfloor n/2 \rfloor} \cdot 5^{\lceil n/2 \rceil} & , n > 1 \end{cases}$$

Q: What does $f(n)$ represent the time to calculate here? What is $f(n)$?

- Calculate $\lfloor n/2 \rfloor$
- Calculate $\lceil n/2 \rceil$
- Final multiplication

$$f(n) = 3 \ (\in \Theta(1))$$

Q: Okay, so what's our total recurrence?

A:
$$T(n) = \begin{cases} 1 & , n \leq 1 \\ 2T(\frac{n}{2}) + 3 & , n > 1 \end{cases}$$

We could solve this here the normal way (we won't, mostly because it's too similar to exercise 4.4.1) but that's a bit tedious. I wish there were some way to determine the asymptotic category of a recurrence based on some of its values.

There is! Introducing the *Master Theorem*¹⁷:

$$\text{If } T(n) = \begin{cases} c & , n \leq n_0 \\ aT\left(\frac{n}{b}\right) + \Theta(n^d) & , n > n_0 \end{cases}$$

where a, b, c, d, n_0 are all constants greater than 0 ($b > 1$) then:

$$T(n) \in \begin{cases} \Theta(n^d) & , a < b^d \\ \Theta(n^d \log(n)) & , a = b^d \\ \Theta(n^{\log_b(a)}) & , a > b^d \end{cases}$$

Important note: it works even when the recursive input to T is either the floor or the ceiling of $\frac{n}{b}$ as well.

¹⁷There are more complicated versions of this theorem that cover more cases online. This will definitely be good enough for our purposes.

Q: Use the Master Theorem to find $T(n)$ for our 5^n algorithm above.

A:

Q: Which of the five constants do we not need to use in our calculation? Why did I list them before then?

A:

Let's look at a divide-and-conquer algorithm for determining the maximum element of an array:

Algorithm: **ArrayMax**

Input: Array A , integers low and $high$

Output: x

```
1: if  $low = high$  then
2:    $x \leftarrow A[low]$ 
3: else
4:    $mid \leftarrow (low + high)/2$ 
5:   //suspiciously empty line
6:    $y \leftarrow \text{ArrayMax}(A, low, mid)$ 
7:    $z \leftarrow \text{ArrayMax}(A, mid + 1, high)$ 
8:   //another suspicious empty line
9:    $x \leftarrow \max(y, z)$ 
```

10: **end if**

Q: What is a recurrence relation for $T(n)$ of **ArrayMax**?

A:

Q: Use the Master Theorem to find $T(n)$ for **ArrayMax**.

A:

Q: It's the same as the linear search in a for loop. Why did I bother showing this to you?

A: Because it's a good use of case 3.

Q: Can we rewrite this to use multiple processors?

Yes! Let's wrap those recursive calls in a simultaneous-do:

Algorithm: **ParallelArrayMax**

Input: Array A , integers low and $high$

Output: x

A:

```
1: if  $low = high$  then  
2:    $x \leftarrow A[low]$   
3: else  
4:    $mid \leftarrow (low + high)/2$   
5:   SimultDo  
6:      $y \leftarrow \text{ArrayMax}(A, low, mid)$   
7:      $z \leftarrow \text{ArrayMax}(A, mid + 1, high)$   
8:   EndDo  
9:    $x \leftarrow \max(y, z)$   
10: end if
```

Q: What is the recurrence relation for $S(n)$ for this algorithm?

A:

Q: Find a closed form asymptotic formula for $S(n)$ using the Master Theorem.

A:

Q: What is $TP(n, n)$ for `ArrayMax`?

A:

$$\begin{aligned} TP(n, n) &\in \Theta \left(\frac{W(n)}{n} + S(n) \right) \\ &= \Theta \left(\frac{\Theta(n)}{n} + \Theta(\log(n)) \right) \\ &= \Theta(\Theta(1) + \Theta(\log(n))) \\ &= \Theta(\Theta(\log(n))) \\ &= \Theta(\log(n)) \end{aligned}$$

Exercises for 5.0

Exercise 0

Use the Master Theorem to check our math from the beginning of Section 4, that for:

$$T(n) = \begin{cases} 1 & , n \leq 1 \\ T(\lfloor \frac{n}{2} \rfloor) + 2 & , n > 1 \end{cases}$$

$$T(n) \in \Theta(\log(n))$$

(Answer 5.0.0 in Appendix)

Exercise 1

Use the Master Theorem to solve for $T(n)$, where

$$T(n) = \begin{cases} 1 & , n \leq 1 \\ T(\lfloor \frac{n}{2} \rfloor) + 44n & , n > 1 \end{cases}$$

Exercise 2

Use the Master Theorem to solve for $T(n)$, where

$$T(n) = \begin{cases} 1 & , n \leq 1 \\ T(\lfloor \frac{n}{2} \rfloor) + 8n^2 - 3n & , n > 1 \end{cases}$$

Exercise 3

Use the Master Theorem to solve for $T(n)$, where

$$T(n) = \begin{cases} 1 & , n \leq 1 \\ 3T(\lfloor \frac{n}{2} \rfloor) + 13n^2 - 13 & , n > 1 \end{cases}$$

Exercise 4

Use the Master Theorem to solve for $T(n)$, where

$$T(n) = \begin{cases} 1 & , n \leq 1 \\ 10T(\lfloor \frac{n}{3} \rfloor) + n^2 + n + 1 & , n > 1 \end{cases}$$

(Answer 5.0.4 in Appendix)

Exercise 5

Use the Master Theorem to solve for $T(n)$, where

$$T(n) = \begin{cases} 1 & , n \leq 1 \\ 5T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 68n^2 & , n > 1 \end{cases}$$

Exercise 6

Use the Master Theorem to solve for $T(n)$, where

$$T(n) = \begin{cases} 1 & , n \leq 1 \\ 11T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + n^3 + 3n^2 + 3n + 1 & , n > 1 \end{cases}$$

Exercise 7

Use the Master Theorem to solve for $T(n)$, where

$$T(n) = \begin{cases} 1 & , n \leq 1 \\ 7T\left(\left\lfloor \frac{n}{3} \right\rfloor\right) + 33n^2 + 22n + 11 & , n > 1 \end{cases}$$

Exercise 8

Use the Master Theorem to solve for $T(n)$, where

$$T(n) = \begin{cases} 1 & , n \leq 1 \\ 8T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 3.3n^3 & , n > 1 \end{cases}$$

5.1 MergeSort

MergeSort is our first divide-and-conquer sorting algorithm!

Algorithm: **MergeSort**

Input: Array of integers A

Output: A , sorted

1: **if** $A.length > 1$ **then**

```

2:    $mid \leftarrow A.length/2$ 
3:    $B \leftarrow$  new array of length  $mid$ 
4:    $C \leftarrow$  new array of length  $A.length - mid$ 
5:   for  $i \in [0, 1, \dots, B.length - 1]$  do
6:        $B[i] \leftarrow A[i]$ 
7:   end for
8:   for  $i \in [0, 1, \dots, C.length - 1]$  do
9:        $C[i] \leftarrow A[i + mid]$ 
10:  end for
11:  //suspiciously empty line
12:  MergeSort( $B$ )
13:  MergeSort( $C$ )
14:  //suspiciously empty line
15:   $j \leftarrow 0$ 
16:   $k \leftarrow 0$ 
17:  for  $i \in [0, 1, \dots, A.length - 1]$  do
18:      if  $j = B.length$  or  $(k < C.length$  and  $C[k] < B[j])$ 
19:          then
20:               $A[i] \leftarrow C[k]$ 
21:               $k \leftarrow k + 1$ 
22:          else
23:               $A[i] \leftarrow B[j]$ 
24:               $j \leftarrow j + 1$ 
25:          end if
26:      end for

```

That last part is **Merge**, and is often written as its own separate algorithm:

Algorithm: **Merge**

Input: Two sorted arrays of integers B, C

Output: A , a sorted array containing all elements of both B and C

```
1:  $A \leftarrow$  new array of length  $B.length + C.length$ 
2:  $j \leftarrow 0$ 
3:  $k \leftarrow 0$ 
4: for  $i \in [0, 1, \dots, A.length - 1]$  do
5:   if  $j = B.length$  or  $(k < C.length$  and  $C[k] < B[j])$ 
   then
6:      $A[i] \leftarrow C[k]$ 
7:      $k \leftarrow k + 1$ 
8:   else
9:      $A[i] \leftarrow B[j]$ 
10:     $j \leftarrow j + 1$ 
11:   end if
12: end for
```

Q: What is our recurrence relation for the running time of MergeSort?

A:

Q: What are our parameters (a, b, d) for the Master Theorem?

A:

Q: Okay, then what's the closed form for T ?

A: $T(n) \in \Theta(n \log(n))$

Q: How can we parallelize this algorithm?

A: We can use our new notation to do the two recursive calls in parallel!

Here's the critical part we're going to change:

Algorithm: **ParallelMergeSort**

Input: Array of integers A

Output: A , sorted

```

1: if  $A.length > 1$  then
2:    $\vdots$ 
3:   SimultDo
4:     MergeSort( $B$ )
5:     MergeSort( $C$ )
6:   EndDo
7:    $\vdots$ 
8: end if
```

Here's the whole thing again:

Algorithm: **ParallelMergeSort**

Input: Array of integers A

Output: A , sorted

```

1: if  $A.length > 1$  then
2:    $mid \leftarrow A.length/2$ 
```

```
3:    $B \leftarrow$  new array of length  $mid$ 
4:    $C \leftarrow$  new array of length  $A.length - mid$ 
5:   for  $i \in [0, 1, \dots, B.length - 1]$  do
6:      $B[i] \leftarrow A[i]$ 
7:   end for
8:   for  $i \in [0, 1, \dots, C.length - 1]$  do
9:      $C[i] \leftarrow A[i + mid]$ 
10:  end for
11:  SimultDo
12:    MergeSort( $B$ )
13:    MergeSort( $C$ )
14:  EndDo
15:   $j \leftarrow 0$ 
16:   $k \leftarrow 0$ 
17:  for  $i \in [0, 1, \dots, A.length - 1]$  do
18:    if  $j = B.length$  or  $(k < C.length$  and  $B[j] < C[k])$ 
19:      then
20:         $A[i] \leftarrow C[k]$ 
21:         $k \leftarrow k + 1$ 
22:      else
23:         $A[i] \leftarrow B[j]$ 
24:         $j \leftarrow j + 1$ 
25:      end if
26:    end for
27:  end if
```

Q: What is our recurrence relation for the $S(n)$ of this new algorithm?

A:
$$S(n) = \begin{cases} 1 & , n \leq 1 \\ S(n/2) + \Theta(n) & , n > 1 \end{cases}$$

Q: So what is the closed form for $S(n)$?

A:

Q: What is $TP(n, n)$?

A:

Talk about Tromino Puzzle

Exercises for 5.1

Exercise 0

Execute **Merge** on the two lists $[1, 5], [2, 2, 3]$. Show each change as an element is taken from one of the two lists to put into the merged list. (You can remove the elements from the two lists getting merged if you want, though that's not actually what's happening if they're arrays.)

(Answer 5.1.0 in Appendix)

Exercise 1

Execute **Merge** on the two lists $[A, E, X], [E, L, M, P]$. Show each change as an element is taken from one of the two lists to put into the merged list. (You can remove the elements from the two lists getting merged if you want, though that's not actually what's happening if they're arrays.)

Exercise 2

- a) Execute **Merge** on the two lists $[I, N, C, O, R], [C, E, R, T]$. Show each change as an element is taken from one of the two lists to put into the merged list. (You can remove the elements from the two lists getting merged if you want, though that's not actually what's happening if they're arrays.)
- b) What happened? Why didn't the final list turn out the way it "should"?
- c) What has to be true about the two input lists in order to result in a final sorted list?

Exercise 3

Execute **MergeSort** on the list $[4, 2, 1, 3]$ (using only one processor). Show each of the split and merge steps. Don't skip splits and merges, even on already-sorted lists!

(Answer 5.1.3 in Appendix)

Exercise 4

Execute **MergeSort** on the list $[E, X, A, M, P, L, E]$ (using only one processor). Show each of the split and merge steps. Don't skip splits and merges, even on already-sorted lists! (This exercise is from Levitin [3] exercise 5.1.1.)

Exercise 5

Execute **MergeSort** on the list $[D, I, V, I, C, O, N, Q]$ (using only one processor). Show each of the split and merge steps. Don't skip splits and merges, even on already-sorted lists!

Exercise 6

What would happen to the running time of **MergeSort** if it required three recursive calls of size $\frac{n}{2}$ instead of 2? What would the running time be? Compare the new time complexity to the original one.

5.2 QuickSort

QuickSort is often faster than MergeSort, but it relies on randomness, which is a huge topic that we can't completely dive in to. In our algorithm below, we use $\leftarrow_R List$ to denote that a (uniformly) random element of *List* is chosen to assign to the variable on the left. There are two versions of the algorithm: a simple one that uses additional arrays, and a more complex (but slightly faster) in place version.

We present the basic version first:

Algorithm: **QuickSortBasic**

Input: Array of numbers, *A*

Output: *A*, sorted

1: **QuickSortBasicHelper**(*A*, *A.length* - 1)

Algorithm: **QuickSortBasicHelper**

Input: Array of numbers, *A*, integer *high*

Output: *A*, sorted from indices 0 to *high*, inclusive

1: **if** *high* > 0 **then**

2: $Lower \leftarrow$ new array of length *high*

```

3:   Higher  $\leftarrow$  new array of length high
4:   j  $\leftarrow$  0
5:   k  $\leftarrow$  0
6:   p  $\leftarrow_R [0, 1, \dots, \textit{high}]$ 
7:   pivot  $\leftarrow A[p]$ 
8:   for i  $\in [0, 1, \dots, \textit{high}]$  do
9:       if i  $\neq p$  and  $A[i] \leq \textit{pivot}$  then
10:          Lower[j]  $\leftarrow A[i]$ 
11:          j  $\leftarrow j + 1$ 
12:       else if i  $\neq p$  then
13:          Higher[k]  $\leftarrow A[i]$ 
14:          k  $\leftarrow k + 1$ 
15:       end if
16:   end for
17:   QuickSortBasicHelper(Lower, j)
18:   QuickSortBasicHelper(Higher, k)
19:   for i  $\in [0, 1, \dots, j]$  do
20:       A[i]  $\leftarrow \textit{Lower}[i]$ 
21:   end for
22:   A[j + 1]  $\leftarrow \textit{pivot}$ 
23:   for i  $\in [0, 1, \dots, k]$  do
24:       A[j + 1 + i]  $\leftarrow \textit{Higher}[i]$ 
25:   end for
26: end if

```

Note: Elements of *A* after index *high* might be empty, as it doesn't necessarily fill the whole thing up.

Talk about best-case time complexity.

Talk about worst-case time complexity.

What about average-case complexity? Talk about failure of $2T(3n/4) + \dots$ analysis. The actual average-case analysis is

beyond the scope of this course, using probability to measure the expected running time. That analysis produces¹⁸:

$$ET(n) \in \Theta(n \ln(n)) = \Theta(n \log(n))$$

If you are interested in learning that analysis, I recommend looking into a graduate level Intro to Randomized Algorithms or Randomness in Computing course¹⁹.

Q: Is this the entire story?

A: Not at all. There's a bigger concern here. We don't just want to know the expected running time (or the median running time) but also the likelihood that the algorithm will fall into the $\Theta(n^2)$ case. As it turns out, that probability is vanishingly small as n grows.

20

¹⁸I'm using ET to mean "Expected Running Time" to differentiate from our already-used $T(n)$.

¹⁹You already know a bit of probability stuff from Discrete Math. If you want to learn more, I also recommend a full (undergrad or graduate) probability course.

²⁰I'm avoiding a lot of the details here. Again, I recommend a graduate course covering Randomized Algorithms. The actual analysis of this is not hard, but it does require some additional knowledge.

Q:

Wait a second! I thought we were always supposed to use Worst Case time complexity! How come we can use the Average Case here? I'm upset about this!!!!

A:

This is a very reasonable thing to be disturbed by. When considering randomized algorithms, however, we consider the average case across the randomness we encounter. We still consider the worst possible case for the inputs, but not the random (stream of) bits we're using to generate our random²¹ choices.

Q:

Why is it okay to assume average randomness, but not average inputs?

A:

This is a bit of a philosophical point. We still want to consider worst-case inputs because you might have some situation that creates these worst-case examples. If that's happening, you'd be pretty upset if you kept getting terrible running times. The algorithm shouldn't need to assume some sort of probability distribution over the inputs, but it is much safer to assume a probability distribution over the random bits.

Q: Are there any times where we consider distributions over the inputs?

A: Absolutely! One such path is the study of Smoothed Analysis (https://en.wikipedia.org/wiki/Smoothed_analysis), where you assume that there are small perturbations on the inputs. This makes sense, since normal scientific measurements are often an approximation of the actual value.

This basic version of QuickSort is not what is usually implemented, however. Instead, there's an in-place algorithm that avoids a significant chunk of the running time, though not enough to put it below $\Theta(n \log(n))$. Here is the advanced version:

Algorithm: **QuickSort**

Input: Array of numbers, A

Output: A , sorted

1: **QuickSortHelper**($A, 0, A.length - 1$)

Algorithm: **QuickSortHelper**

Input: Array of numbers, A , integers $low, high$

Output: A , sorted from index low to $high$, inclusive

```

1: if  $low < high$  then
2:    $p \leftarrow_R [low, low + 1, \dots, high]$ 
3:   Swap  $A[p] \leftrightarrow A[low]$ 
4:    $i \leftarrow low$ 
5:    $j \leftarrow high$ 
6:    $p \leftarrow low$ 
7:   while  $i < j$  do
8:     if  $A[i] > A[j]$  then
```

```
9:         Swap  $A[i] \leftrightarrow A[j]$ 
10:        if  $p = i$  then
11:             $i \leftarrow i + 1$ 
12:             $p \leftarrow j$ 
13:        else
14:             $j \leftarrow j - 1$ 
15:             $p \leftarrow i$ 
16:        end if
17:    else
18:        if  $p = i$  then
19:             $j \leftarrow j - 1$ 
20:        else
21:             $i \leftarrow i + 1$ 
22:        end if
23:    end if
24: end while
25: QuickSortHelper( $A, low, i - 1$ )
26: QuickSortHelper( $A, i + 1, high$ )
27: end if
```

Exercises for 5.2

Exercise 0

Execute QuickSortBasic on the list $[2, 4, 1, 3]$ (using only one processor). Instead of randomly generating a pivot, p , just assume you always choose 0. As with the MergeSort exercises in the prior section, show how you break up the array at each step, then copy them back together. (I also wrote the pivot between the two split arrays. See the solution to this problem for a good example.)

(Answer 5.2.0 in Appendix)

Exercise 1

Execute **QuickSortBasic** on the list $[E, X, A, M, P, L, E]$ (using only one processor). Instead of randomly generating a pivot, p , just assume you always choose 0. As with the **MergeSort** exercises in the prior section, show how you break up the array at each step, then copy them back together. (I also wrote the pivot between the two split arrays. This exercise is from Levitin [3] exercise 5.2.1.)

Exercise 2

Execute **QuickSortBasic** on the list $[D, I, V, I, C, O, N, Q]$ (using only one processor). Instead of randomly generating a pivot, p , just assume you always choose 0. As with the **MergeSort** exercises in the prior section, show how you break up the array at each step, then copy them back together. (I also wrote the pivot between the two split arrays.)

Exercise 3

Execute **QuickSort** (the advanced version) on the list $[2, 4, 1, 3]$ (using only one processor). Instead of randomly generating a pivot, p , just assume you always choose *low*. Show each of the swaps, and (symbolically) break up the lists when you make the recursive calls. (See the solution to this problem for a good example.)

(Answer 5.2.3 in Appendix)

Exercise 4

Execute **QuickSort** (the advanced version) on the list $[E, X, A, M, P, L, E]$ (using only one processor). Instead of randomly generating a pivot, p , just assume you always choose *low*. Show each of the swaps, and (symbolically) break up the lists

when you make the recursive calls. (This exercise is from Levitin [3] exercise 5.2.1.)

Exercise 5

Execute **QuickSort** (the advanced version) on the list $[D, I, V, I, C, O, N, Q]$ (using only one processor). Instead of randomly generating a pivot, p , just assume you always choose *low*. Show each of the swaps, and (symbolically) break up the lists when you make the recursive calls.

Exercise 6

We could consider the “average” case of **QuickSort** being the one where we split the array a quarter of the way down. That would make our recurrence relation be: (AT for Average Time)

$$AT(n) \in \begin{cases} c & , n \leq n_0 \\ AT(3n/4) + AT(n/4) + \Theta(n) & , n > n_0 \end{cases}$$

We could use this to pretty confidently say that $ET(n) \approx AT(n)$. Unfortunately, the formula for AT doesn’t match anything we can apply our version of the Master Theorem to. We could, however, overestimate and say that $ET(n)$ is in big- O of that overestimation. For that overestimation, we could pretend there are two recursive calls of size $3n/4$ instead of two calls of different sizes. As it turns out, that (sadly) doesn’t result in the $\Theta(n \log(n))$ that more advanced math tells us is correct for $ET(n)$.

In the opposite direction, what similar trick could we use to show a *lower* bound on the expected running time of $ET(n)$? Do the math out to find the lower bound and express ET in terms of that.

Sadly, this lower bound doesn’t tell us anything new. Why is

that?

(Answer 5.2.6 in Appendix)

Exercise 7

Use the Best-Case Time,

$$BCT(n) = \begin{cases} c & , n \leq n_0 \\ 2BCT(n/2) + \Theta(n) & , n > n_0 \end{cases}$$

as a lower bound on $ET(n)$, the expected running time of QuickSort, and use the Above-Average-Case Time, $AACT(n) =$

$$\begin{cases} c & , n \leq n_0 \\ 2AACT(3n/4) + \Theta(n) & , n > n_0 \end{cases}$$

as an upper bound (derived in a similar way to the prior problem). Simplify these to describe $ET(n)$ using asymptotic notation as the intersection of two sets.

Exercise 8

Modify the `QuickSortHelper` algorithm to run in parallel.

(Answer 5.2.8 in Appendix)

Exercise 9

Modify the `QuickSortBasicHelper` algorithm to run in parallel. Parallelize as much as possible. (There's more to do here than in the advanced one!) Hint: the expected step complexity ($ES(n)$) formula in the recursive case should come out to be $ES(n/2) + \Theta(n)$.

Exercise 10

Use BCT and $AACT$ from exercise 5.2.7 to find asymptotic notation for $ES(n)$, the Expected Step complexity of QuickSort. (I called the two step complexity functions BCS and $AACS$.)

Exercise 11

Use your answer to exercise 5.2.10 to find $ETP(n, n^{1.5})$ for the parallel `QuickSort` or `QuickSortBasic`.

5.3 Binary Trees**5.4 Large Numbers and Matrices****5.5 Geometric Div & Conq Algos**

Talk about Nearest Neighbors

6 Transform and Conquer

As our text points out, there are three main forms of Transform-and-Conquer algorithms:

- **Instance Simplification** transforms an instance of a problem into a more simple (or more convenient) instance.
- **Representation Change** transforms an instance of a problem into a different representation of the same instance.
- **Problem Reduction** transforms an instance of a problem into an instance of a different problem.

Each of these has two parts, with a possible third:

1. Perform the transformation.
2. Solve the transformed problem.
3. (Maybe) Translate the solution on the transformed problem back to get the original solution.

Consider the problem, *ARRAYDUPLICATES*, of determining whether a list has any duplicate elements. Let's start by assuming the list is represented using an array.

Q: What is an instance-simplification approach to this problem?

A:

Q: How long does that take to execute?

A: $\Theta(n \log(n))$ to sort, then $\Theta(n)$ to check for a total of $\Theta(n \log(n))$.

Q: What is a representation-change approach to solve this problem? (This one took me the longest to come up with.)

A:

Q: How long does that take?

A: If we use a Hash Map (this assumes the elements are hashable) then it takes $\Theta(n)$ time to add all the elements. Then it takes $\Theta(n)$ time to traverse the keys and investigate whether there are any doubles.

Q: What about a problem-reduction solution? Hint: use sets.

A:

Q: How long does this solution take?

A: Again, if the elements are hashable, then it takes $\Theta(n)$ time to add all the elements to a HashSet. Then it takes $\Theta(n)$ time to count them all up and see whether the total equals n .

Q:

For many of these, it seems like we could just solve the problem during the transformation, potentially speeding things up. Why don't we do this?

Multiple reasons:

- First, the code to solve to either perform the transformation or solve the transformed version might already exist. In that case, it's easier to just reuse that code and write the other half. (Not to mention that other half is likely more reusable!)

A:

- Doing those extra checks might add an extra constant to the runtime that is just as expensive as performing both steps. In that case, why bother?
- If we have to solve multiple problems, then performing the transformation once might let us reuse that transformed data to more easily solve the problem.

6.1 Presorting

Presorting is often used as an instance-simplification transformation. As in our intro example to this section, we can use it to help determine whether a list has duplicate elements in less than $\Theta(n^2)$ time. We also used this multiple times in our divide-and-conquer algorithm for Nearest Neighbors.

Presorting is used extensively in geometric and list-based problems.

Note, let's assume that sorting takes $\Theta(n \log(n))$ time, for a few reasons:

- At this point in the text I used to use ([3]) the same time is also used.
- We haven't covered so-called linear time sorting algorithms such as **RadixSort** yet.
- There are other considerations with **RadixSort** that aren't always covered in courses that lead up to this one. We will cover all that in due time.

Many of the following exercises are adapted from the Levitin exercises in section 6.1 [3].

Exercises for 6.1

Exercise 0

- a) Write a presorting algorithm to compute the mode of an array of numbers. (The mode is the element that occurs the most number of times.)
- b) Analyze the running time of your algorithm.
- c) Compare your running time to the running time of the brute-force version of this algorithm. (It may help to describe the brute-force version in words; you do not need to write the algorithm out.) Is the presorting version faster?

(Answer 6.1.0 in Appendix)

Exercise 1

- a) Write a presorting algorithm to determine whether an array of numbers contains a specified number.
- b) Analyze the running time of your algorithm.
- c) Compare your running time to the running time of the brute-force version of this algorithm. (It may help to describe the brute-force version in words; you do not need to write the algorithm out.) Is the presorting version faster?

Exercise 2

- a) Write a presorting algorithm to count the number of times an array of numbers contains a specified number.
- b) Analyze the running time of your algorithm. Include both n , the length of the array and k , the number of times the number appears in the array.
- c) Compare your running time to the running time of the brute-force version of this algorithm, using both n and k . (It may help to describe the brute-force version in words; you do not need to write the algorithm out.) When is the presorting version faster?

Exercise 3

- a) Write a presorting algorithm that returns a list of all elements common to two given lists of integers.

- b) Analyze the running time of your algorithm. Include both n , the length of the first array and m , the length of the second.
- c) Compare your running time to the running time of the brute-force version of this algorithm, using both n and m . (It may help to describe the brute-force version in words; you do not need to write the algorithm out.) When is the presorting version faster?

This exercise is inspired by 6.1.2 in Levitin [3].

6.2 Gaussian Elimination

Linear Algebra!!!!

6.3 Balanced Search Trees

AVL trees! :)

Q:	What is the inherent problem with basic binary search trees?
A:	If elements get added in the “wrong” order, they are essentially just linked lists.

Even if you add elements so it’s a full tree, it can still be very unbalanced!

Q: So what can we do?

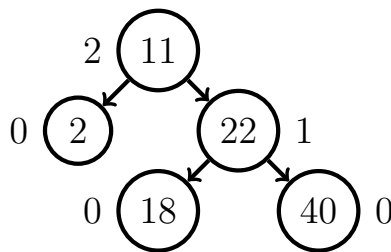
A: Self-balancing trees! Let's talk about AVL trees, which are self-balancing binary search trees.

Adelson-Velsky Landis²² trees are self-balancing binary trees. In order to perform the self-balancing, each node must keep track of its own height, meaning the number of edges to get to the furthest leaf under it. (Empty trees have a height of -1.)

Q: What does an AVL tree do “on the way back up” after it's added a node in the appropriate place?

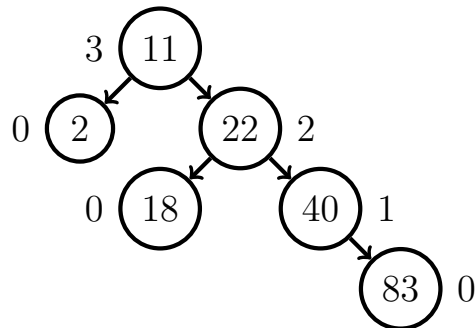
A: At each node it checks the heights of the two subtrees. If they're off by 2 or more, it means it needs to rebalance.

Example: In the following tree, with heights labelled on the sides, no rebalancing is needed:



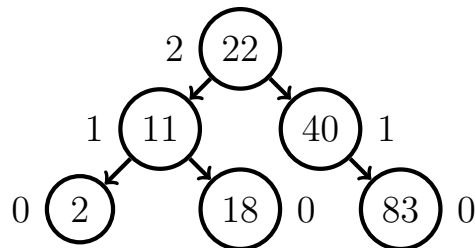
However, after we add 83 in the appropriate place, the root is out of balance:

²²Two creators, not three: https://en.wikipedia.org/wiki/AVL_tree.

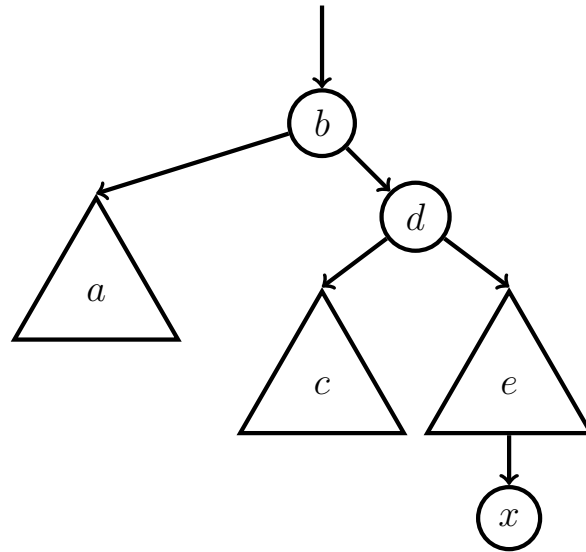


The rebalancing that fixes this is a *single-L rotation* around the node with the 11. This is because the right subtree of that node is higher, and the furthest away node is in the right subtree of that subtree.

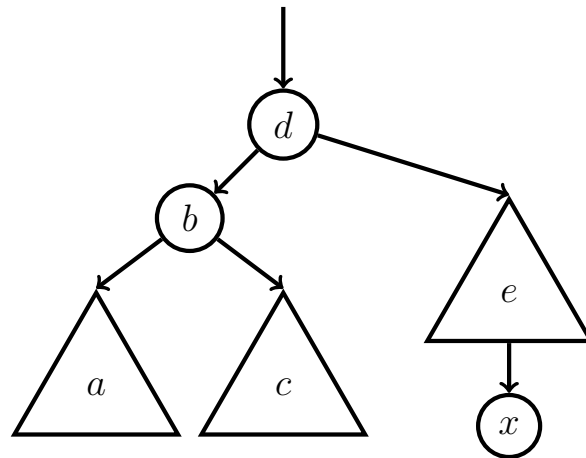
A single-L rotation rebalances the tree to this:



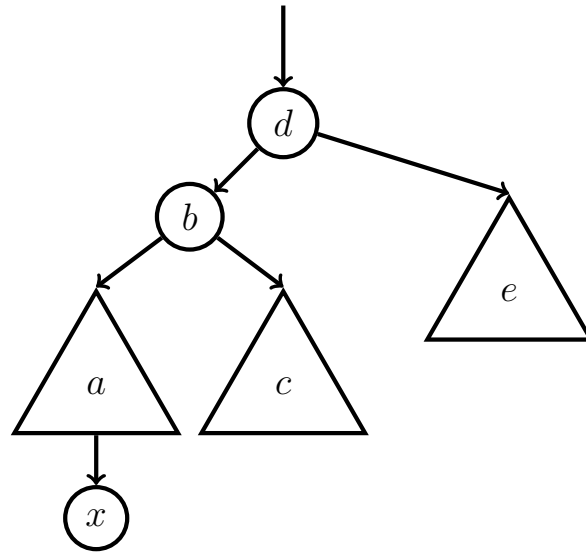
That may seem like a lot, but there are only three nodes that actually changed what their children are: the node with 11, the node with 22, and, if there were a parent of the 11, that as well. In general, we can describe a single-L rotation with the following figure:



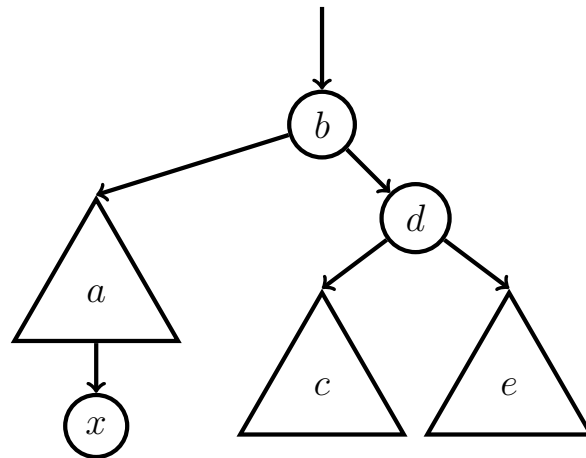
This becomes:



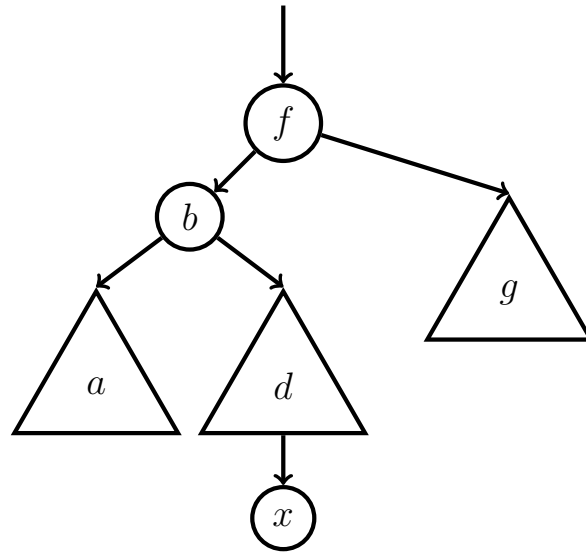
... and now all parts are balanced. This is more specifically referred to as a *single L-rotation around b*, because *b* is the original root we rebalanced around. A single R-rotation works very similarly. Here's a *single R-rotation around d*:



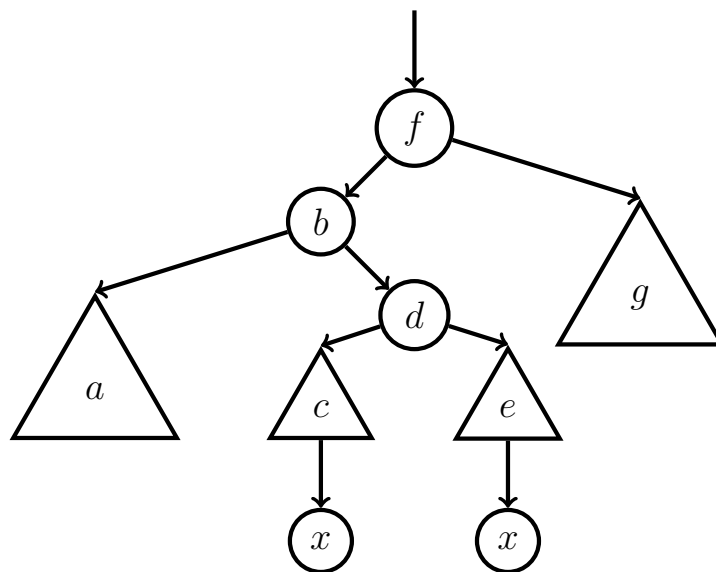
rotates to:



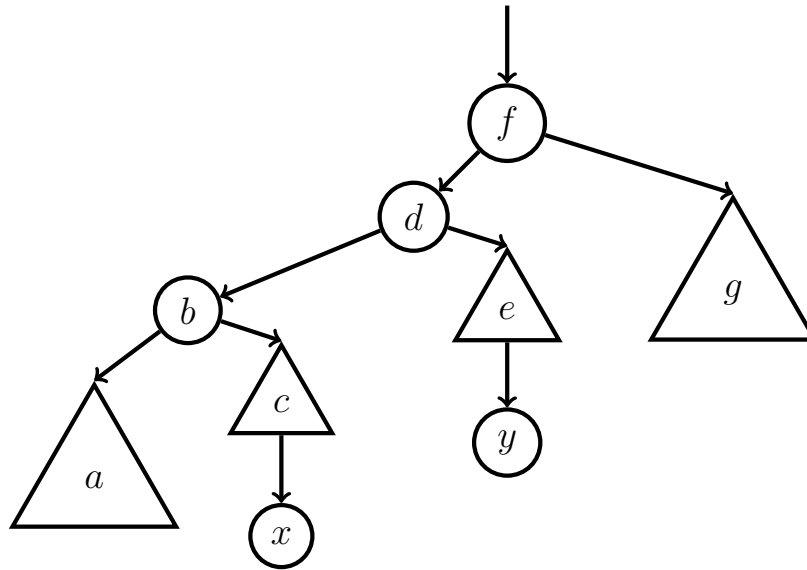
Double rotations are more complicated. There are two types: double-RL and double-LR, and they happen when the taller subsubtree is on the inside instead of the outside. For example:



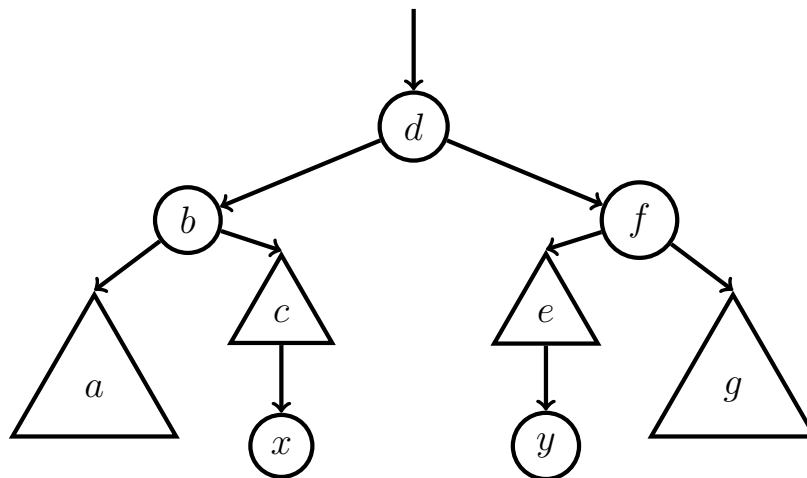
Now, unfortunately, we have to split d up and handle the two cases. Note that there are two cases of where x can be. I've included both, even though only one of them has actually been added.



To fix this, we need a double-LR rotation, which consists of two separate single rotations. First, a single-L rotation around b :



... followed by a single-R rotation around f :



Even though only one of x and y exist, we are rebalanced! The double-RL rotation is the mirror image of that.

Exercises for 6.3**Exercise 0**

Consider representing AVL trees with 4-tuples: (value, height, leftSubtree, rightSubtree). We use $\emptyset$ to represent an empty tree.

- a) Draw the tree represented by the tuple

$$(10, h_1, (5, h_2, \emptyset, \emptyset), (23, h_3, \emptyset, \emptyset))$$

- b) What is the value of each h_i ?
- c) Is the tree balanced? If any trees or subtrees are unbalanced, which ones?

(Answer 6.3.0 in Appendix)

Exercise 1

Consider representing AVL trees with 4-tuples: (value, height, leftSubtree, rightSubtree). We use $\emptyset$ to represent an empty tree.

- a) Draw the tree represented by the tuple

$$(2, h_1, \emptyset, (7, h_2, \emptyset, (8, h_3, \emptyset, \emptyset)))$$

- b) What is the value of each h_i ?
- c) Is the tree balanced? If any trees or subtrees are unbalanced, which ones?

Exercise 2

Consider representing AVL trees with 4-tuples: (value, height, leftSubtree, rightSubtree). We use $\emptyset$ to represent an empty tree.

a) Draw the tree represented by the tuple

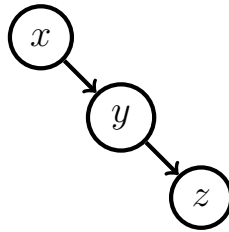
$$(-80, h_1, (-200, h_2, (-220, h_3, \emptyset, \emptyset), \emptyset), (-10, h_4, \emptyset, (-1, h_5, \emptyset, \emptyset)))$$

b) What is the value of each h_i ?

c) Is the tree balanced? If any trees or subtrees are unbalanced, which ones?

Exercise 3

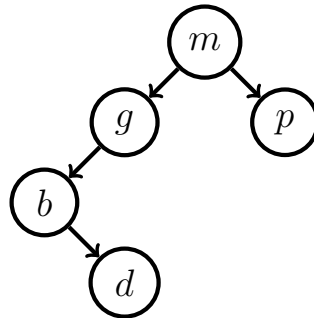
The following AVL tree just had an element added, and needs to be rebalanced. (a) Determine which element was just added, then (b) show and label the rotation used to perform the rebalancing.



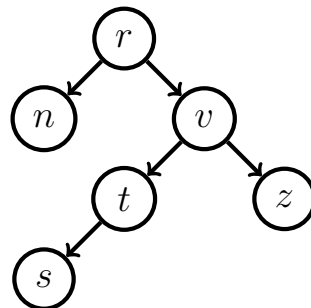
(Answer 6.3.3 in Appendix)

Exercise 4

The following AVL tree just had an element added, and needs to be rebalanced. (a) Determine which element was just added, then (b) show and label the rotation used to perform the rebalancing.

**Exercise 5**

The following AVL tree just had an element added, and needs to be rebalanced. (a) Determine which element was just added, then (b) show and label the rotation used to perform the rebalancing.

**Exercise 6**

Consider representing AVL trees with 4-tuples: (value, height, leftSubtree, rightSubtree). We use $\emptyset$ to represent an empty tree. Perform a Single R-rotation on

$$(x_1, h_1, (x_2, h_1 - 1, A, B), C)$$

and write the resulting tree in 4-tuple-form in terms of A , B , C , x_1 , x_2 , and h_1 .

(Answer 6.3.6 in Appendix)

Exercise 7

Consider representing AVL trees with 4-tuples: (value, height, leftSubtree, rightSubtree). We use $\emptyset$ to represent an empty tree. Perform a Single L-rotation on

$$(x_1, h_1, A, (x_2, h_1 - 1, B, C))$$

and write the resulting tree in 4-tuple-form in terms of A , B , C , x_1 , x_2 , and h_1 .

Exercise 8

Consider representing AVL trees with 4-tuples: (value, height, leftSubtree, rightSubtree). We use $\emptyset$ to represent an empty tree. Perform a Double LR-rotation on

$$(x_1, h_1, (x_2, h_1 - 1, A, (x_3, h_1 - 2, B, C)), D)$$

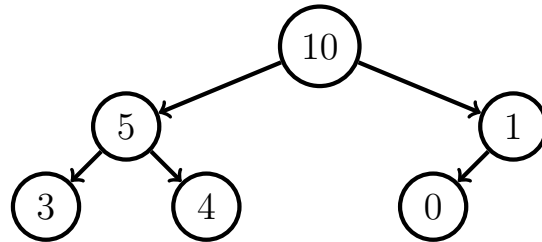
and write the resulting tree in 4-tuple-form in terms of A , B , C , D , x_1 , x_2 , x_3 , and h_1 . (Be sure to show the complete intermediate step as part of your work.)

6.4 Heap Sort

Heap Sort!

Exercises for 6.4**Exercise 0**

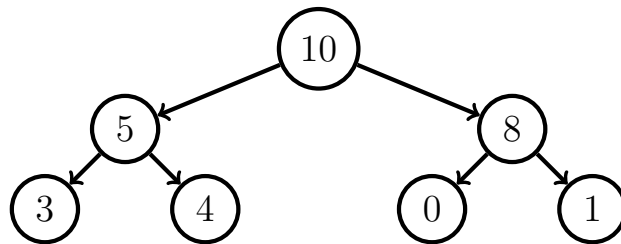
Add the number 8 to the following max-heap. Draw the entire heap with every change that is made during heapification.



(Answer 6.4.0 in Appendix)

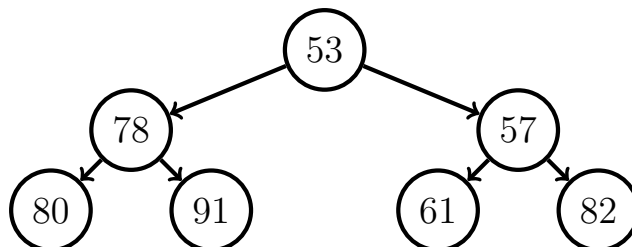
Exercise 1

Add the number 7 to the following max-heap. Draw the entire heap with every change that is made during heapification.



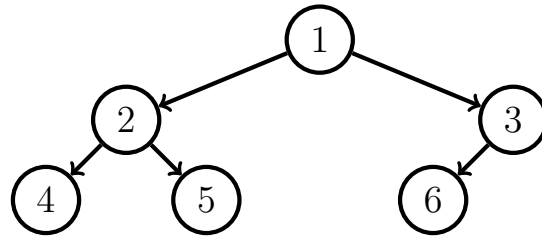
Exercise 2

Add the number 33 to the following min-heap. Draw the entire heap with every change that is made during heapification.



Exercise 3

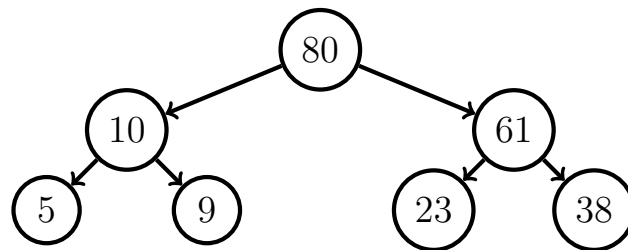
Remove the element from the root of the following min-heap. Draw the entire heap with every change that is made during reheapification.



(Answer 6.4.3 in Appendix)

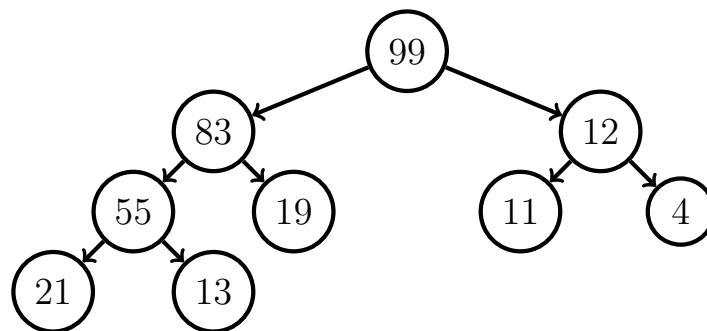
Exercise 4

Remove the element from the root of the following max-heap. Draw the entire heap with every change that is made during reheapification.



Exercise 5

Remove the element from the root of the following max-heap. Draw the entire heap with every change that is made during reheapification.



Exercise 6

Find a min-heap with at least five unique values that has a different state if you remove the top value, then add that back to the heap. (Your example must be different from the answer of anyone else in the class.) Show the initial state, the state after removing the element, and the state after adding the element back, which must be different from the first.

Exercise 7

Find a min-heap with at least five unique values that has the *same* state after you remove the top value, then add that back to the heap. (Your example must be different from the answer of anyone else in the class.) Show the initial state, the state after removing the element, and the state after adding the element back, which must be *the same* as the first.

Exercise 8

Why does it take $\Theta(n)$ time to add the numbers $1, 2, \dots, n$ (in that order) to a min-heap?

Exercise 9

Is it possible to find a min-heap, H , and two numbers x and y such that when you add x , then perform a removal, it removes x and H returns to its original state, but when you add y , then perform a removal, y is removed, but H does not return to its original state. Explain why this is or isn't possible.

Exercise 10

Run `HeapSort` (the array version) on the list $[4, 3, 1, 2]$. Show all changes to the list and the array. (I'm expecting you to use a min-heap, but you can alternatively do it with a max-heap.)

(Answer 6.4.10 in Appendix)

Exercise 11

Run **HeapSort** (the array version) on the list $[E, X, A, M, P, L, E]$. Show all changes to the list and the array. (I'm expecting you to use a min-heap, but you can alternatively do it with a max-heap.)

Exercise 12

Run **HeapSort** (the array version) on the list $[T, R, A, N, S, F, O, R, M]$. Show all changes to the list and the array. (I'm expecting you to use a min-heap, but you can alternatively do it with a max-heap.)

6.5 Horner's Rule

6.6 Problem Reduction

Just like we were lied to about GCDs in school, we were also lied to about Least Common Multiples! Here's the algorithm we were taught in grade school: (We were actually taught more of a Guess-and-Check, but this linear search suffices.)

Algorithm: **GradeSchoolLCM**

Input: Pair of positive integers, x, y .

Output: m , the smallest positive multiple of both x and y .

```
1:  $i \leftarrow 1$ 
2:  $j \leftarrow 1$ 
3: while  $i \cdot x \neq j \cdot y$  do
4:   if  $i \cdot x < j \cdot y$  then
5:      $i \leftarrow i + 1$ 
6:   else
```

```

7:       $j \leftarrow j + 1$ 
8:  end if
9: end while
10:  $m \leftarrow i \cdot x$ 

```

Q: Why can't we binary search the above algorithm instead of doing a linear search?

A:

The better algorithm is to just calculate it using this formula that involves the GCD:

$$\text{LCM}(x, y) = \frac{x \cdot y}{\text{GCD}(x, y)}$$

Linear Programming (LP) is a common optimization problem used all over scientific computing. A linear programming problem has the following form:

maximize $c_0x_0 + c_1x_1 + c_2x_2 + \cdots c_{n-1}x_{n-1}$

subject to:

$$m_{0,0}x_0 + m_{0,1}x_1 + \cdots m_{0,n-1}x_{n-1} \leq d_0$$

$$m_{1,0}x_0 + m_{1,1}x_1 + \cdots m_{1,n-1}x_{n-1} \leq d_1$$

$\vdots$

$$m_{k-1,0}x_0 + m_{k-1,1}x_1 + \cdots m_{k-1,n-1}x_{n-1} \leq d_{k-1}$$

The top line is called the *objective function* and the remaining lines are the various restrictions.

Q:

What's the solution to this very basic example?

maximize x_1

subject to:

$x_0 + x_1 \leq 1$ (blue)

$-\frac{1}{2}x_0 + x_1 \leq \frac{7}{4}$ (red)

A:**Q:**

What must be true about the solution to a linear program?

A:

It must always be at one of the vertices. (If there are multiple optimal solutions, one will be at one of the vertices.)

The *Simplex Algorithm* is a famous algorithm for solving these problems. Use a die to show off how it works. In grad school I met some people who would just plug linear program-

ming problems into the Simplex Algorithm all the time to solve problems. (These were other grad students outside of CS.)

Q:

Is this an instance of Linear Programming?

maximize x_0 subject to:

$$x_0 \geq 0$$

$$x_1 \geq 0$$

$$x_0 \leq x_1 + 3$$

$$2x_1 - 8 \leq x_0$$

$$x_1 \leq -2x_0 + 9$$

A:

No! It doesn't have the correct form. That means we can't plug it into the Simplex Algorithm.

Q: Can we transform it into a LP instance?

A:

Exercises for 6.6

Exercise 0

Use the GCD to determine $\text{LCM}(25, 60)$. Show all your work to calculate both the GCD and LCM.

(Answer 6.6.0 in Appendix)

Exercise 1

Use the GCD to determine $\text{LCM}(48, 132)$. Show all your work to calculate both the GCD and LCM.

Exercise 2

Use the GCD to determine $\text{LCM}(126, 336)$. Show all your work to calculate both the GCD and LCM.

Exercise 3

What is the running time of the grade school algorithm for finding $\text{LCM}(x, y)$? Write your answer in terms of x , y , and $g = \text{GCD}(x, y)$.

Exercise 4

Solve the following linear program in two dimensions. Provide the location of the best point as well as the value of the objective function at that point. Do not use any automated solvers to find your answer. (Hint: plot the constraints)

Maximize x_0 subject to:

$$x_1 \leq 2$$

$$x_0 - x_1 \leq 0$$

and $x_0, x_1 \geq 0$

(Answer 6.6.4 in Appendix)

Exercise 5

Solve the following linear program in two dimensions. Provide the location of the best point as well as the value of the objective function at that point. Do not use any automated solvers to find your answer. (Hint: plot the constraints)

Maximize $x_0 + x_1$ subject to:

$$2x_0 + x_1 \leq 8$$

$$-x_0 + x_1 \leq 2$$

$$-x_1 \leq -1$$

and $x_0, x_1 \geq 0$

Exercise 6

Solve the following linear program in two dimensions. Provide the location of the best point as well as the value of the objective function at that point. Do not use any automated solvers to find your answer. (Hint: plot the constraints)

Maximize $x_0 + x_1$ subject to:

$$x_0 + 3x_1 \leq 7$$

$$x_0 - x_1 \leq 3$$

$$-x_0 + x_1 \leq 2$$

and $x_0, x_1 \geq 0$

Exercise 7

Transform the following problem into a linear programming problem, then use technology to solve it. (I know that many online solvers are very flexible with their inputs, but you need to put it in the form described in the chapter.) Cite the site you used and how you used it!

Minimize $x_0 - x_1 + x_2$ subject to:

$$x_0 + 5 \geq x_1 + 2x_2$$

$$x_0 + x_1 \leq 10x_2$$

$$3x_1 \leq 10 - x_2$$

$$x_0 \geq -x_2$$

and $x_0, x_1, x_2 \geq 0$

(Answer 6.6.7 in Appendix)

Exercise 8

Transform the following problem into a linear programming problem, then use technology to solve it. (I know that many online solvers are very flexible with their inputs, but you need to put it in the form described in the chapter.) Cite the site you used and how you used it!

Maximize $5x_0 + x_1 - x_2$ subject to:

$$100(x_0 - 50) \leq -100(x_1 + x_2)$$

$$7x_1 + 10x_2 + 20 \geq x_0$$

$$4x_0 + 3x_1 + 2x_2 + 1 \leq 100$$

$$4x_0 + 3x_1 + 2x_2 + 1 \geq 2$$

and $x_0, x_1, x_2 \geq 0$

Exercise 9

Transform the following problem into a linear programming problem, then use technology to solve it. (I know that many online solvers are very flexible with their inputs, but you need to put it in the form described in the chapter.) Cite the site you used and how you used it!

Minimize $-2x_0 + 2x_1 - x_2$ subject to:

$$x_0 + x_1 + x_2 \geq 4$$

$$10 \leq 3x_0 + x_1 + 3x_2 \leq 100$$

$$x_0 \geq -20 + x_1$$

and $x_0, x_1, x_2 \geq 0$

7 Time and Space Tradeoffs

Sometimes time and/or space is at a premium while the other is not. It can make sense to sacrifice one for the other.

7.1 Sorting and Counting

7.2 String Matching

7.3 Hashing

Hashing is a strategy of storing elements of a large space into an array of size k using a *hash function* from the larger space to $[0, 1, 2, \dots, k - 1]$.

The hash function, h , has some extra requirements:

- The size of the large space should be significantly greater than k . (If it was $\leq k$, then you could just store them in the array without a hash function.)
- k must be larger than the number of elements that will be stored in the hash table, n . (It should really be significantly bigger.) $k > 2n$ is often considered the bare minimum.
- $\forall x$, it should take $O(1)$ time to compute $h(x)$ and it must be deterministic.
- h must “appear random”.

This last point needs some explanation. It means that given an output, y , it must be hard to figure out which inputs, x , satisfy $h(x) = y$. In other words, if x_1 and x_2 are similar, they should map to two very different values $h(x_1)$ and $h(x_2)$. Another way to put it is that it must be very difficult to find two

$x_1 \neq x_2$ such that $h(x_1) = h(x_2)$. (Note that there must be some because h 's range is much smaller than its domain.) A more technical term for “appearing random” is *collision resistant*.

We'll first talk about the ways to use a hash function and array to implement a Hash Set. Afterwards, we'll show the simple extension to do a hash table instead. There are two main ways to use h to build the Hash Set. Both of them use an array, A , with indices $[0, 1, \dots, k - 1]$.

Q: What operations do we have to implement for our Hash Set?

A: Add, contains, and remove, all of which need to run in $\Theta(1)$ time.

Closed Hashing

Closed Hashing is a way of implementing a hash set that uses only an array and no extra data structures. You start with an empty array, A , of size k and add elements to the indices based on the hash values of the elements. If we're adding x and $h(x) = i$, then we check cell $A[i]$ to see whether it's empty. If it is, we store x into it: $A[i] \leftarrow x$.

The tricky part comes when $A[i]$ is not empty. (Collision!) In that case, we still want to store the element, so we look at the next space to see whether there's an empty spot there. If so, we store the element in that spot. If not, we keep going until we find an empty spot to store it. We want collisions to be rare so that it's unlikely that we have to do this extra traversal. And, that when we do, it should likely complete in a constant number of checks.

Q: How do we implement contains?

A:

Q: What's a potential problem with remove?

A: Remove might leave a hole that breaks contains.

Q: What do we have to do instead of emptying out the cell?

A: Put a *sentinel* value in it (often referred to as a tombstone) that says an element was removed from here.

Q: Now how does searching for x in contains work?

Get the initial index from the hash function.
Check out the cell at that index.

A:

- If it's empty, return false.
- If it's a tombstone, increment index and go again.
- If it's full with x , return true.
- If it's full with a value other than x , increment the index and go again.

Q: What do you do if you get close to filling up a closed hash set?

A:

Q: What are some benefits of open hashing?

A:

Q: What are the downsides?

A:

Open Hashing

Open Hashing avoids all use of traversing the array by instead having it be an array of linked lists. Whenever you add an element, you put it in the linked list that starts on the index of the hash of the element. You never need to traverse into other cells of the array.

Q: How long does it take to look up whether an open hash set contains x ?

A: $\Theta(1 + s)$, where s is the number of elements in the set that hash to $h(x)$.

Q: Why do we assume this is still $\Theta(1)$?

A:

Q: Can an open hash set get filled up in the same way the closed hash set can?

A:

Q: Is there any need for tombstones?

A:

Q: These are all good things. What's the downside of open hashing?

A:

Hash Tables

So far we've described using hashing to store a set of elements. A *hash table* (a.k.a., a hash map) is a simple extension of that. A hash table needs to provide an output for every element of the input. These inputs still come from that same initial big space. The outputs can be from any space.

Just as with the hash set, we use the hash function on the input elements to determine a location in the array. The only difference is that instead of storing the input in the array, we store both the input and output element as a pair.

For example, in a hash set, we might store the number 633

in the array using our hash function. Now, we want to associate an output value with that 633, say 282. Then we store the pair, (633, 282) in the space in the array. In both the closed and open systems, when checking to see whether a pair with input 633 is already in the table, we iterate and look for that 633 in the first element of the pairs. If we see it, then we know that it's already stored in the table, and that the output value is 282.

Do Hash Functions Exist?

All parts of a hash function are reasonable... except (maybe) for the part about acting random. The thing that makes them collision resistant may not exactly be possible. As we'll learn about in section 11.3, there is an open problem concerning two sets of computational problems: P and NP . If $P \subsetneq NP$, as many people believe, then hash functions may exist. If, however, $P = NP$, then they do not exist. In that case, we do not have constant-time hashing that resists collisions. That changes a lot of things, including the running times of our optimal sorting algorithms!

Hashing Milestones

Although it's important for hash functions to be collision resistant, it's even more important in cryptography. Here's a list of some of the well-known Hash Functions. (Some are not considered CHF's (Cryptographic Hash Functions), because they weren't used in crypto algorithms.)

Note: *digest size* is the term for the output of the function.

- MD-2 ("Message Digest"):
 - Not considered a CHF.
 - Published 1989
 - Digest size: 2^7 bits

- Collisions for part of it found in 1995, but not the whole thing.
- First pre-image attack successful in 2004.
- Disabled from major use (e.g. OpenSSL) in 2009.
- MD-4:
 - Not considered a CHF.
 - Published 1990
 - Digest size: 2^7 bits
 - First full collision attack: 1995.
- MD-5:
 - Published 1991
 - Digest size: 2^7 bits
 - Commonly used as checksum algorithm!
 - First full collision attack: 2004.
- SHA-0 (“Secure Hash Algorithm”):
 - Not considered a CHF.
 - Published 1993
 - First public collision: 2004
- SHA-1:
 - Published 1995
 - Digest size: 160 bits
 - Secure until 2005 (against “Well-funded opponents”)
 - First public collision: 2017

- Whirlpool:
 - Published 2000
 - Digest size: 2^9 bits
 - First public collision: 2009
- SHA-2:
 - Published 2001
 - Digest size: 224, 256, 384, or 512 bits
- MD-6:
 - Published 2008
 - Digest size: up to 2^9 bits
 - Commonly used as checksum algorithm!
 - First full collision attack: 2004.
- SHA-3:
 - Published 2015
 - Digest size: arbitrary
 - Uses Keccak family of algorithms. Decided from a competition.

Exercises for 7.3

Exercise 0

Consider the hash function $h : [0, 1, 2, \dots, 1023] \rightarrow [0, 1, 2, \dots, 12]$ where $h(x) = x \bmod 13$. Let's create a set, S , implemented with a closed hash table, T , that uses h .

- a) Draw T after adding elements 1004, 561, 620, 81, 120, and 180, to S , in that order.
 - b) How many cells are checked to determine that $400 \notin S$?
 - c) How many cells are checked to determine that $327 \notin S$?
 - d) How many cells are checked to determine that $120 \in S$?
- (Answer 7.3.0 in Appendix)

Exercise 1

Consider the hash function $h : [0, 1, 2, \dots, 4095] \rightarrow [0, 1, 2, \dots, 12]$ where $h(x) = (217x) \bmod 13$. Let's create a set, S , implemented with a closed hash table, T , that uses h .

- a) Draw T after adding elements 3000, 3001, 3002, 8, 125, and 1234, to S , in that order.
- b) How many cells are checked to determine that $400 \notin S$?
- c) How many cells are checked to determine that $327 \notin S$?
- d) How many cells are checked to determine that $125 \in S$?

Exercise 2

Consider the hash function $h : [0, 1, 2, \dots, 16183] \rightarrow [0, 1, 2, \dots, 12]$ where $h(x) = (\lfloor x/128 \rfloor + (x \bmod 128)) \bmod 13$. Let's create a set, S , implemented with a closed hash table, T , that uses h .

- a) Draw T after adding elements 10000, 10001, 10002, 17, 16161, and 12345, to S , in that order.
- b) How many cells are checked to determine that $400 \notin S$?
- c) How many cells are checked to determine that $15907 \notin S$?

- d) How many cells are checked to determine that $17 \in S$?

Exercise 3

Consider the hash function $h : [0, 1, 2, \dots, 1023] \rightarrow [0, 1, 2, \dots, 12]$ where $h(x) = x \bmod 13$. Let's create a set, S , implemented with an open hash table, T , that uses h .

- a) Draw T after adding elements 1004, 561, 620, 81, 120, and 180, to S , in that order.
- b) How many tests on lists are made to determine that $400 \notin S$?
- c) How many tests on lists are made to determine that $327 \notin S$?
- d) How many tests on lists are made to determine that $120 \in S$?

(Answer 7.3.3 in Appendix)

Exercise 4

Consider the hash function $h : [0, 1, 2, \dots, 4095] \rightarrow [0, 1, 2, \dots, 12]$ where $h(x) = (217x) \bmod 13$. Let's create a set, S , implemented with an open hash table, T , that uses h .

- a) Draw T after adding elements 3000, 3001, 3002, 8, 125, and 1234, to S , in that order.
- b) How many tests on lists are made to determine that $400 \notin S$?
- c) How many tests on lists are made to determine that $327 \notin S$?

- d) How many tests on lists are made to determine that $125 \in S$?

Exercise 5

Consider the hash function $h : [0, 1, 2, \dots, 16183] \rightarrow [0, 1, 2, \dots, 12]$ where $h(x) = (\lfloor x/128 \rfloor + (x \bmod 128)) \bmod 13$. Let's create a set, S , implemented with an open hash table, T , that uses h .

- a) Draw T after adding elements 10000, 10001, 10002, 17, 16161, and 12345, to S , in that order.
- b) How many tests on lists are made to determine that $400 \notin S$?
- c) How many tests on lists are made to determine that $15907 \notin S$?
- d) How many tests on lists are made to determine that $17 \in S$?

Exercise 6

Use Algorithmic Notation to write `ClosedHashContains`, which takes A , h , and x , and returns whether the Hash contains x . Don't forget to account for tombstones! (I assumed that if a cell was empty, it held the value `None`. I used *tombstone* as my sentinel tombstone value.)

You can assume that there are fewer than k elements in the hash set.

7.4 B-Trees

I don't cover this in this course because I cover this in OS & Concurrency when I teach that course.

8 Dynamic Programming

Basic idea. Top down vs Bottom up.

Talk about Fibonacci.

Change-Making

Q: What if we have a country with coin denominations of 1, 7, 10, and 20. How should I make change to total 15?

A: There are multiple ways! Either one 10-coin and five 1-coins, or: two 7-coins and one 1-coin.

Q: Which of those two is “better”?

A: Usually the one with fewer coins.

Let’s consider the Change-Making problem in general!

Brute-Force version: (TODO: Move to brute force section?)

This algorithm tries one of each type of coin, then recurses in each case.

Algorithm: **MakeChange**

Input: D , array of denominations of length m ; n , amount to sum to.

Output: C , array of amounts of each denomination such that $C \cdot D = n$

```
1: if  $n = 0$  then  
2:    $C \leftarrow$  array of zeroes of length  $D.length$   
3: else  
4:    $C \leftarrow false$ 
```

```
5:   countMin  $\leftarrow \infty$ 
6:   for  $i \in [0, 1, \dots, D.length - 1]$  do
7:       if  $n \geq D[i]$  then
8:            $R \leftarrow \text{MakeChange}(D, n - D[i])$ 
9:           if  $R \neq \text{false}$  then
10:               $R[i] \leftarrow R[i] + 1$ 
11:               $count \leftarrow \text{sum}(R)$ 
12:              if  $count < countMin$  then
13:                   $C \leftarrow R$ 
14:                   $countMin \leftarrow count$ 
15:              end if
16:          end if
17:      end if
18:  end for
19: end if
```

This algorithm tries all possible combination of coins without going over the total.

Q:

What is the running time of this algorithm if m is the number of denominations and n is the amount? (Assume that n is bigger than the maximum element of D .) What's an upper bound for this?

A:

Q: What's a lower bound? Assume that the sum of all elements of D is less than n .

A:

Q: Based on these, what can we say about the running time of this algorithm?

A: $T(n, m) \in O(n^m) \cap \Omega(2^n)$. It's some (bad) flavor of exponential!

Q: Can we get this to be polynomial? How can we modify this into a dynamic programming problem?

A: Instead of recursing down the array, we'll iterate down the array and recurse at each index.

Algorithm: `DynamicProgrammingMakeChange`

Input: D , array of denominations, n , amount to sum to, h , the hash table of integers to arrays.

Output: C , array of amounts of each denomination such that $C \cdot D = n$

```
1: if  $h.contains(n)$  then  
2:    $C \leftarrow h.get(n)$   
3: else  
4:    $Z \leftarrow$  array of zeroes of length  $D.length$ 
```

```
5:   if  $n = 0$  then
6:        $C \leftarrow Z$ 
7:        $h.put(n, C)$ 
8:   else
9:        $C \leftarrow false$ 
10:       $countMin \leftarrow \infty$ 
11:      for  $i \in [0, 1, \dots, D.length - 1]$  do
12:          if  $n \geq D[i]$  then
13:               $R \leftarrow \text{DynamicProgrammingMakeChange}(D, n -$ 
14:                   $D[i], h)$ 
15:                  if  $R \neq false$  then
16:                       $R[i] \leftarrow R[i] + 1$ 
17:                       $count \leftarrow sum(R)$ 
18:                      if  $count < countMin$  then
19:                           $C \leftarrow R$ 
20:                           $countMin \leftarrow count$ 
21:                      end if
22:                  end if
23:              end for
24:               $h.put(n, C)$ 
25:          end if
26:      end if
```

Exercises for 8.0

Exercise 0

Execute `DynamicProgrammingMakeChange([4, 3, 1], 6, new HashTable)`. Show each of the additions to the hash table in the correct order. You can include any other computational steps you want as long as you show at least the hash table additions.

(Answer 8.0.0 in Appendix)

Exercise 1

Execute `DynamicProgrammingMakeChange([1, 3, 4], 6, new HashTable)`. Show each of the additions to the hash table in the correct order. You can include any other computational steps you want as long as you show at least the hash table additions. (This is the same as exercise 8.0.0, but with the elements of D in the reverse order.)

Exercise 2

Execute `DynamicProgrammingMakeChange([5, 4, 2], 8, new HashTable)`. Show each of the additions to the hash table in the correct order. You can include any other computational steps you want as long as you show at least the hash table additions. Hint: there will be some instances of *false* in the hash table, since the smallest denomination isn't 1.

Exercise 3

Execute `DynamicProgrammingMakeChange([10, 7, 5, 3], 19, new HashTable)`. Show each of the additions to the hash table in the correct order. You can include any other computational steps you want as long as you show at least the hash table additions.

Exercise 4

Write a bottom-up dynamic programming solution to the Making Change problem, `DPBottomUpMakeChange( $D, n$ )`. Execute your program on the same inputs as exercise 8.0.3, showing the order that elements are added to the hash table.

9 Greedy Algorithms

Common terminology: *The Greedy Algorithm.*

Common conversation:

- Software Developer A: “So, we have this problem to solve...”
- Software Developer B: “What about the greedy algorithm?”
- A: “Oh! Will that work?”
- B: “Hmmm, I dunno.”
- A: “Let’s try it and see.”

Q: What is “The” (a) Greedy Algorithm for a given problem?

A: The algorithm that tries to solve it as quickly as possible and hopes that that will work.

In general, Greedy Algorithms are most applicable to optimization problems. At each step of a greedy algorithm, it makes a greedy (maybe myopic, naive, or reckless) choice instead of being careful about making the absolutely *correct* choice. Nevertheless, sometimes this works out.

Our text points out three better points about the choice that is made. It must be:

- feasible
- locally optimal
- irrevocable (We’re not going to backtrack.)

Some Greedy Algorithms don't work. (E.g. Pretty Printing.)

Q: What is the Greedy Algorithm for the Change-Making problem?

A: Always pick the biggest denomination that is less than or equal to the total.

Q: Does it work?

A:

Q: What could happen instead?

A:

Q: Can you give me an example of each?

A:

It's good to be skeptical of greedy algorithms. However, in the rest of this chapter we'll look at some greedy algorithms that work.

9.1 Prim's Algorithm

First, explain what a spanning tree is.

Algorithm: **Prim**(G)

Input: $G = (V, E, w)$, a connected weighted graph

Output: $T \subset E$, a minimum spanning tree of G

```

1:  $T \leftarrow \emptyset$ 
2:  $v \leftarrow$  any vertex in  $V$ 
3:  $V_T \leftarrow \{v\}$ 
4:  $P \leftarrow$  PriorityQueue of edges and weights, prioritizing low
   weights
5: for  $(v, u) \in v.edges$  do (Order listed will matter.)
6:    $P.add(((v, u), w(v, u)))$ 
7: end for
8: for  $i \in [1, 2, \dots, |V|]$  do
9:    $e \leftarrow null$ 
10:  while  $e = null$  do
11:     $((x, y), k) \leftarrow P.remove()$ 
12:    if  $y \notin V_T$  then
13:       $e \leftarrow (x, y)$ 
14:    end if
15:  end while
16:  for  $(y, z) \in y.edges$  do
17:     $P.add(((y, z), w(y, z)))$ 
18:  end for
19:   $T \leftarrow T \cup \{(x, y)\}$ 
20:   $V_T \leftarrow V_T \cup \{y\}$ 
21: end for

```

Proof of correctness!

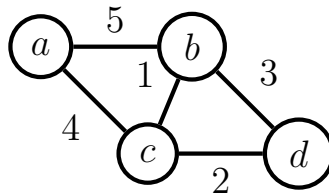
Q: What's the running time?

A:

Exercises for 9.1

Exercise 0

Use Prim's Algorithm to find the minimum spanning tree of the graph below, starting from vertex a :

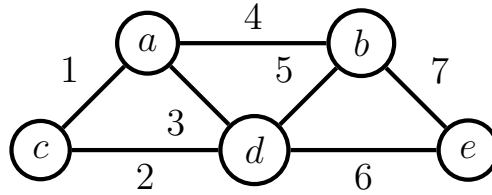


- Show the state of V_T , T , and P each time T and V_T change and the new edges are added to P . You do not have to include duplicate edges in P . (I wrote out P as a sorted list.)
- Draw the final spanning tree with the edge weights. (Do not draw the unchosen edges.)

(Answer 9.1.0 in Appendix)

Exercise 1

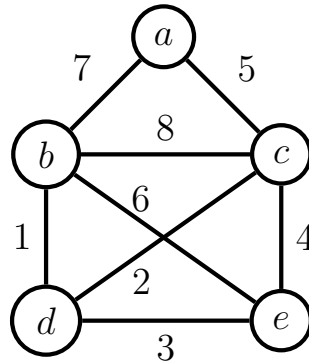
Use Prim's Algorithm to find the minimum spanning tree of the graph below, starting from vertex a :



- a) Show the state of V_T , T , and P each time T and V_T change and the new edges are added to P . You do not have to include duplicate edges in P . (I wrote out P as a sorted list.)
- b) Draw the final spanning tree with the edge weights. (Do not draw the unchosen edges.)

Exercise 2

Use Prim's Algorithm to find the minimum spanning tree of the graph below, starting from vertex a :



- a) Show the state of V_T , T , and P each time T and V_T change and the new edges are added to P . You do not have to include duplicate edges in P . (I wrote out P as a sorted list.)

- b) Draw the final spanning tree with the edge weights. (Do not draw the unchosen edges.)

9.2 Kruskal's Algorithm

Kruskal's goes through all edges, from lightest to heaviest, adding them if they don't create a cycle.

Algorithm: `Kruskal( $G$ )`

Input: $G = (V, E, w)$, a connected weighted graph

Output: $T \subset E$, a minimum spanning tree of G

```

1:  $T \leftarrow \emptyset$ 
2:  $P \leftarrow$  PriorityQueue of edges and weights, prioritizing low
   weights
3: for  $e \in E$  do
4:    $P.add((e, w(e)))$ 
5: end for
6:  $S \leftarrow \{\{v_i\} \mid v_i \in V\}$ 
7: while  $|T| < |V| - 1$  do
8:    $((x, y), k) \leftarrow P.remove()$ 
9:   if  $x$  and  $y$  are in different elements,  $s_x$  and  $s_y$  of  $S$  then
10:     $T \leftarrow T \cup \{(x, y)\}$ 
11:     $S \leftarrow \{s_x \cup s_y\} \cup (S \setminus \{s_x, s_y\})$ 
12:   end if
13: end while

```

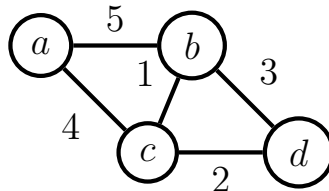
In order to do this well, we need to talk about Union-Find.

Running time! :)

Exercises for 9.2

Exercise 0

Use Kruskal's Algorithm to find the minimum spanning tree of the graph below.

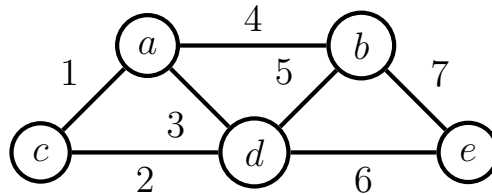


- Show the state of T , S , and P each time T changes. (I wrote out P as a sorted list.)
- Draw the final spanning tree with the edge weights. (Do not draw the unchosen edges.)

(Answer 9.2.0 in Appendix)

Exercise 1

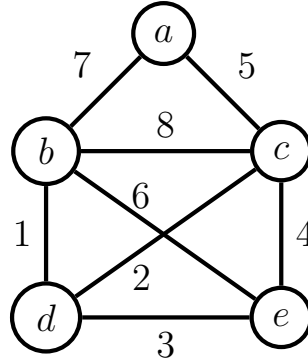
Use Kruskal's Algorithm to find the minimum spanning tree of the graph below.



- Show the state of T , S , and P each time T changes. You do not have to include duplicate edges in P . (I wrote out P as a sorted list.)
- Draw the final spanning tree with the edge weights. (Do not draw the unchosen edges.)

Exercise 2

Use Kruskal's Algorithm to find the minimum spanning tree of the graph below.



- Show the state of T , S , and P each time T changes. (I wrote out P as a sorted list.)
- Draw the final spanning tree with the edge weights. (Do not draw the unchosen edges.)

9.3 Borůvka's Algorithm

Borůvka's algorithm is the oldest of the bunch, first figured out in 1926 by Otakar Borůvka.

A *forest* is a collection of disjoint trees in a graph. A *spanning forest* is a forest that contains all nodes in a graph.

Algorithm: $\text{Borůvka}(G)$

Input: $G = (V, E, w)$, a connected weighted graph

Output: $T \subset E$, a minimum spanning tree of G

- 1: $Trees \leftarrow \{\{v\} \mid v \in V\}$
- 2: $id \leftarrow \text{new Map } V \rightarrow \mathbb{Z}^+$
- 3: $T \leftarrow \emptyset$

```

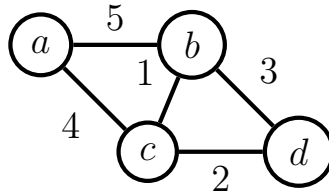
4: for  $i \in [1, 2, \dots, |V|]$  do
5:    $v \leftarrow$  next element of  $V$ 
6:    $id.put(v, i)$ 
7: end for
8: while  $|Trees| > 1$  do
9:    $tw \leftarrow$  new Array of length  $|V|$  where each value is  $(\infty,$ 
    null) Will be cheapest edges leaving tree by vertex.
10:  for  $(x, y) \in E$  do
11:    if  $tw[id(x)][0] > w((x, y))$  then
12:       $tw[id(x)] \leftarrow (w((x, y)), (x, y))$ 
13:    end if
14:    if  $tw[id(y)][0] > w((x, y))$  then
15:       $tw[id(y)] \leftarrow (w((x, y)), (x, y))$ 
16:    end if
17:  end for
18:  for  $tree \in Trees$  do
19:     $(x, y) \leftarrow tw[tree[0]][1]$ 
20:     $T \leftarrow T \cup \{(x, y)\}$ 
21:     $minId \leftarrow \min(id[x], id[y])$ 
22:     $maxId \leftarrow \max(id[x], id[y])$ 
23:    if  $minId < id[tree[0]]$  then
24:      for  $v \in tree$  do
25:         $id[v] \leftarrow minId$ 
26:      end for
27:    end if
28:  end for
29:  Union trees in  $Trees$  based on the new edges added
30: end while

```

Exercises for 9.3

Exercise 0

Use Borůvka's Algorithm to find the minimum spanning tree of the graph below.

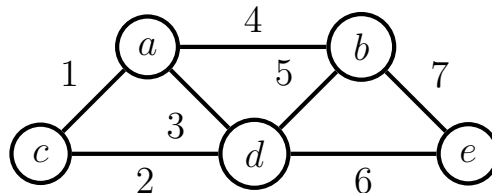


- Show T , and $Trees$ initially and after every round of the main while loop. Also, for each iteration of the main while loop, show the edge (x, y) each tree selects during the second inner loop (**for** $tree \in Trees$ **do**).
- Draw the final spanning tree with the edge weights. (Do not draw the unchosen edges.)

(Answer 9.3.0 in Appendix)

Exercise 1

Use Borůvka's Algorithm to find the minimum spanning tree of the graph below.

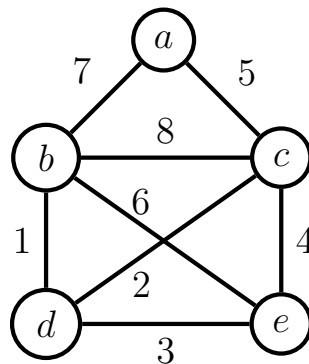


- Show T , and $Trees$ initially and after every round of the main while loop. Also, for each iteration of the main while loop, show the edge (x, y) each tree selects during the second inner loop (**for** $tree \in Trees$ **do**).

- b) Draw the final spanning tree with the edge weights. (Do not draw the unchosen edges.)

Exercise 2

Use Borůvka's Algorithm to find the minimum spanning tree of the graph below.



- a) Show T , and $Trees$ initially and after every round of the main while loop. Also, for each iteration of the main while loop, show the edge (x, y) each tree selects during the second inner loop (**for** $tree \in Trees$ **do**).
- b) Draw the final spanning tree with the edge weights. (Do not draw the unchosen edges.)

Exercise 3

Create an example of a graph G that requires three rounds of the main while loop of Borůvka's Algorithm to find the minimum spanning tree.

9.4 Dijkstra's Shortest Path Algorithm

Greedy algorithms are used for more than just minimum spanning trees! We can also use them to find shortest paths between vertices.

The high-level idea is that:

- We're going to grow a tree from the starting vertex.
- At each round we're going to add the next closest vertex to that start to the tree.
- We can determine which vertex is next by keeping track of the vertices one edge outside of the tree, and the closest distance we've seen to them.
- Once we add t to the tree, we're done. We can work backwards through the tree to find the shortest path from s to t .

Do an example here!

Note: It's common for s to be used for the “source” vertex, and t to be used for the “goal” or “sink” vertex in searching algorithms. That's why they are used below.

Algorithm: `DijkstraShortestPath( $G, s, t$ )`

Input: $G = (V, E, w)$, a weighted graph, $s \in V$, $t \in V$

Output: *Path*, list of vertices that form the shortest path from s to t in G .

```
1: for  $v \in V$  do
2:    $v.dist \leftarrow \infty$ 
3:    $v.parent \leftarrow null$ 
4: end for
5:  $s.dist \leftarrow 0$ 
```

```

6:  $P \leftarrow$  new Priority Queue of pairs (distance, edge), prioritiz-
   ing lowest first element
7:  $last \leftarrow s$ 
8: while  $last \neq t$  do
9:   for  $e = (last, y) \in last.edges$  do
10:     $distToY \leftarrow last.dist + w(e)$ 
11:    if  $distToY < y.dist$  then
12:       $y.dist \leftarrow distToY$ 
13:       $P.add(distToY, (last, y))$ 
14:    end if
15:  end for
16:   $(d, (x, y)) \leftarrow P.remove()$ 
17:  while  $d > y.dist$  do
18:     $(d, (x, y)) \leftarrow P.remove()$ 
19:  end while
20:   $y.parent \leftarrow x$ 
21:   $last \leftarrow y$ 
22: end while
23:  $Path \leftarrow [last]$ 
24: while  $last \neq s$  do
25:    $last \leftarrow last.parent$ 
26:    $Path \leftarrow [last] + Path$ 
27: end while

```


Q:

How long does this algorithm take to run?

- The top loop runs in $\Theta(n)$ time.
- The bottom loop could traverse all vertices in the worst case, which would be another $\Theta(n)$ time.
- The middle (main) loop runs up to n times, but each loop is variable, so it's easier to look at the amortized total operations in that loop instead of multiplying each run by n .

A:

- In the worst case, the main loop may add and remove all edges from priority queue P . No other step gets executed significantly more than those, so that's a good thing to count. Adding and removing all m edges yields $\Theta(m \log(m))$ total steps.
- In total, that's $\Theta(n + m \log(m) + n) = \Theta(m \log(m))$.

Sometimes instead of finding the shortest path between two vertices, you want to find the entire spanning tree of a graph that represents the set of shortest paths from a start vertex to all other vertices. That algorithm has one fewer input: no sink vertex.

(I don't usually present this in class; students can look it up

here.)

Algorithm: `DijkstraShortestPathTree( $G, s$ )`

Input: $G = (V, E, w)$, a weighted graph, $s \in V$, $t \in V$

Output: $T \subset E$, forming the shortest path tree from s on G .

```

1:  $T \leftarrow \emptyset$ 
2: for  $v \in V$  do
3:    $v.dist \leftarrow \infty$ 
4: end for
5:  $s.dist \leftarrow 0$ 
6:  $P \leftarrow$  new Priority Queue of pairs (distance, edge), prioritiz-
   ing lowest first element
7:  $last \leftarrow s$ 
8: while  $|T| < n - 1$  do
9:   for  $e = (last, y) \in last.edges$  do
10:     $distToY \leftarrow last.dist + w(e)$ 
11:    if  $distToY < y.dist$  then
12:       $y.dist \leftarrow distToY$ 
13:       $P.add(distToY, (last, y))$ 
14:    end if
15:  end for
16:   $(d, (x, y)) \leftarrow P.remove()$ 
17:  while  $d > y.dist$  do
18:     $(d, (x, y)) \leftarrow P.remove()$ 
19:  end while
20:   $last \leftarrow y$ 
21:   $T \leftarrow T \cup \{(x, y)\}$ 
22: end while

```

Q: Does this work if there are negative edges? What if $G = (V, E, w)$ and $w : E \rightarrow \mathbb{R}$ instead of $\mathbb{R}^+$?

A: It does not work. The whole reason this works is because the distance from the root that we're looking at never goes down. With negative edges, it's possible that after we add a vertex to the tree, we later find a shorter path to it using one or more negative weight edges.

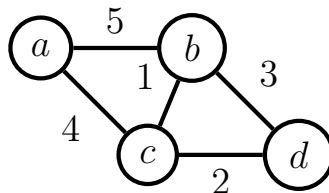
Q: So what do we do in that case?

A: There's another algorithm, Bellman-Ford, that solves this problem. It runs in $\Theta(nm)$ time, which is significantly longer than $\Theta(m \log(m))$. Bellman-Ford is beyond the scope of these notes. (At this point, anyways.)

Exercises for 9.4

Exercise 0

Use Dijkstra's Shortest Path Algorithm to find the shortest path from a to d in the graph below.

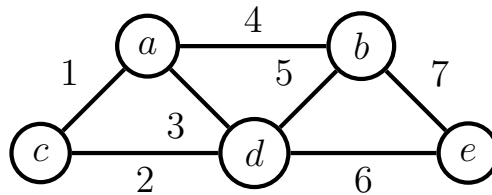


- a) Show the state of P and list the distances of the nodes at the end of the first for loop inside the first while loop, and after that big while loop. (I wrote out P as a sorted list.) Break ties alphabetically by the name of the vertices.
- b) Show SP at the end.

(Answer 9.4.0 in Appendix)

Exercise 1

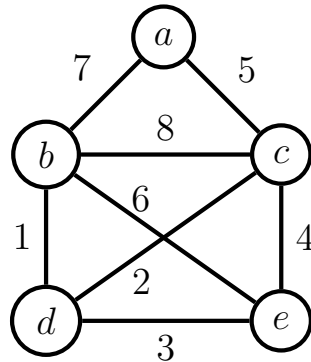
Use Dijkstra's Shortest Path Algorithm to find the shortest path from c to e in the graph below.



- a) Show the state of P and list the distances of the nodes at the end of the first for loop inside the first while loop, and after that big while loop. (I wrote out P as a sorted list.) Break ties alphabetically by the name of the vertices.
- b) Show SP at the end.

Exercise 2

Use Dijkstra's Shortest Path Algorithm to find the shortest path from b to e in the graph below.



- a) Show the state of P and list the distances of the nodes at the end of the first for loop inside the first while loop, and after that big while loop. (I wrote out P as a sorted list.) Break ties alphabetically by the name of the vertices.
- b) Show SP at the end.

10 Iterative Improvement

11 Algorithmic Limitations

Algorithms can't solve all problems. The most important lesson in Theory of Computation is the halting problem, which no algorithm can solve all cases of, no matter how long they have.

11.1 Lower Bounds

Lower Bounds: “What is the absolute fastest we could solve this problem?”

AKA: “What's the fastest we could get an algorithm to run to solve this problem in the worst case?”

At this point we're considering the timing for a *problem* instead of simply one algorithm.

Trivial Lower Bounds! Talk about these!

11.2 Decision Trees

We can use Decision Trees to reason about lower bounds. Let's consider a sorting algorithm that compares elements to determine how to rearrange the elements in the array. After each comparison, it branches into one of two cases²³. We can model this computation as a binary tree, where each leaf represents the end of the rearrangement that properly sorts the array. In other words, that leaf is a single permutation of the elements of the array into sorted order.

Q:

What's the minimum number of leaves that we need? Hint: What is the number of possible ways an array of n elements could be ordered? That's the same number!

A:

$n!$

Q:

What does the height of the decision tree (of the branching program) represent?

A:

The worst-case running time of the algorithm.

²³This is called a branching program.

Q: What kind of binary tree has the lowest height?

A: A complete tree, or any balanced tree will do.

Q: Okay, what's the height of a balanced binary tree with $n!$ leaves?

A: $\Theta(\log(n!))$

Q: Can we express that without the factorial? Hint: Stirling's Approximation!

A: Stirling's Approximation says that $\ln(n!) = n \ln(n) - n + O(\ln(n))$. That simplifies to $\Theta(n \log(n))$.

24

Notice: that's exactly what we get for our comparison based algorithms: MergeSort, QuickSort, and HeapSort!

11.3 P vs NP

Let's talk a little bit about finite sets of integers. Let's define addition on these sets to be the set of all possible pairwise sums:

$$\forall S, T \subseteq \mathbb{Z} : S + T = \{s + t \mid s \in S, t \in T\}$$

²⁴https://en.wikipedia.org/wiki/Stirling's_approximation

Q: What is $\{1, 3\} + \{2, 5\}$?

A: $\{3, 5, 6, 8\}$

Q: What is $\{1, 3, 5\} + \{2, 5, 6\}$?

A: $\{3, 5, 6, 7, 8, 9, 10, 11\}$

Q: What is $\{1, 3\} + \{-5, 2\} + \{-1, 1\}$?

A: $\{-5, -3, -1, 2, 4, 6\}$

Q: What if we have n such sets, $S_1, \dots, S_n$, each with three elements, and want to decide whether some $x \in \sum_{i=1}^n S_i$?
How many elements might that sum have?

A:

Q: Why is it unfeasible to search that space to look for x ?

A: It has exponential size!

Algorithms are generally considered to be *tractable* if they can be executed in polynomial time²⁵. Other common synonyms include feasible, efficient, easy, fast, practical.

Exponential time classes are therefore intractable, unfeasible, inefficient, hard, slow, and impractical²⁶.

Q: Our input for the problem is the list of n sets. What could we possibly do feasibly to find out whether x is in the sum of those sets?

A: This is an open-ended question! I hope there's lots of discussion.

Q: What if we picked $s_1 \in S_1, s_2 \in S_2, \dots, s_n \in S_n$ and then added them up to get an x ? We could play a mean guessing game and ask other people whether x is in that sum set. What would we do if they said yes?

A: Ask them to prove it by finding $s_1, s_2, \dots, s_n$ from each of the sets that add up to x . That would suck for them!

²⁵First from the Cobham-Edmonds Thesis from two papers in 1965 [1][2] (https://en.wikipedia.org/wiki/Cobham%27s_thesis) listing it as P , later extended to BPP, “Bounded-Error Probabilistic Polynomial Time” ([https://en.wikipedia.org/wiki/BPP_\(complexity\)](https://en.wikipedia.org/wiki/BPP_(complexity))).

²⁶There is certainly some space inbetween, e.g. pseudopolynomial (https://en.wikipedia.org/wiki/Pseudo-polynomial_time) and quasi-polynomial (https://en.wikipedia.org/wiki/Quasi-polynomial_time).

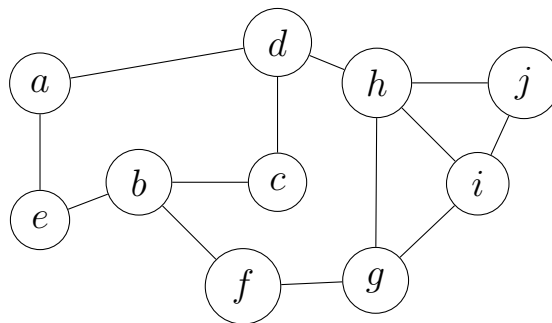
Q: What if they said no?

A: Then we could prove them wrong by revealing our $s_1, s_2, \dots, s_n$.

Similar scenarios are asked in classes and lectures all the time! The speaker has the proof of something and wants the audience to go searching through a large space to see if they can also find the proof. When someone finds it, they can describe the working proof in polynomial time, even if getting there required an exponential-time algorithm. (Often exhaustive search.)

There are a lot of problems like this; they have a polynomial-sized proof that a solution exists when one does. (Even if finding it is hard.) Let's look at another example!

Consider the computational problem of finding an *independent set* on a graph: given a simple unweighted graph $G = (V, E)$, and an integer $k \leq |V|$, determine whether there is a subset of the vertices, $S \subset V$, such that $|S| \geq k$ and that no two elements of S share an edge. For example, consider this graph:



Q: Is $S = \{a, b, d, j, g\}$ an independent set of G above?

A: No! $(a, d) \in E$.

Q: Is there an independent set of size four in G ?

A:

Q: Is there an independent set of size five in G ?

A:

Notice that if there's a solution and you find it, you can convince someone else quickly. If there isn't, it's not as easy to convince someone else.

In these notes we have mostly talked about problems in P , the set of (decision) problems that can be solved in a Polynomial amount of time²⁷. In other words, $T(n)$ for that problem is in $\Theta(n^c)$ for some constant c . That is very different from EXPTIME, which is the set of all problems that can be solved in exponential time, e.g. $O(2^n)$. Because of this clear delineation, we know that $P \subsetneq \text{EXPTIME}$.

Although algorithms have been studied heavily since the 1950s,

²⁷I will mostly avoid the distinction between decision problems and general purpose computational problems, as that is mostly beyond the scope of this course. It's interesting stuff to get in to, though!

there are some problems that still don't have fast algorithms to solve them. In the 1970s, a special collection of these problems was recognized: NP, which stands for Non-deterministic Polynomial-time. There are two equivalent ways to characterize problems in NP:

- If there's a solution to the problem, that solution can be found in polynomial-time in the (imaginary) situation where we can try all possible solutions in parallel.
- If there's a solution to the problem, and we already know that solution, then we can show that solution to someone else and they can verify that solution in polynomial-time.

INDEPENDENT SET is exactly one of these problems, so it's inside of NP.

$P \subset NP$, because if a solution exists, we can already find it without any help in polynomial time. We currently don't know about the $\neq$ part, though. As of the writing of these notes, we don't know whether $P \subsetneq NP$ or $P = NP$. If there was a proof that $\text{INDEPENDENT SET} \notin P$, then we would know that $P \neq NP$. So far, we don't have that.

Another of these problems is SATISFIABILITY: given a formula of boolean variables, is there a way to assign all variables so that the formula is true? E.g.:

$$(x_1 \vee x_2 \vee x_3) \wedge (\overline{x_1} \vee x_2 \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_3}) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee x_2 \vee \overline{x_3})$$

Q: Is there a satisfying assignment of variables?

A:

Any formula can be rewritten to an equivalent formula in the above form (3-CNF, Conjunctive Normal Form) in polynomial time, so we usually talk about SATISFIABILITY in terms of this subproblem, known as 3SAT.

Q: Is $3\text{SAT} \in \text{NP}$?

A:

In the early 1970s, the Cook-Levin Theorem was proven, showing that every problem in NP can be transformed into an equivalent 3SAT problem in polynomial time. (Since the phrase “in polynomial time” is so common, the term *efficient* was coined to have that same meaning, so we’ll use that sometimes.)

Q: What does that mean if someone finds an efficient algorithm to solve 3SAT?

A: $P = \text{NP}$

Q: Contrapositively, what does it mean if $P \neq \text{NP}$?

A: $3\text{SAT} \in \text{NP} \setminus P$

This makes 3SAT (one of the) hardest problems in NP. If we can solve it efficiently, we can solve anything in NP efficiently.

Shortly after the Cook-Levin theorem, Richard Karp found 21 problems that had the same property! (INDEPENDENT SET is one of these.)

Q: How could Karp quickly (relatively) show that everything in NP can be transformed to INDEPENDENT SET?

A:

These kinds of transformations are called reductions! A *reduction* is a function:

- $f : A \rightarrow B$, A and B are both computational problems.
- f can be performed efficiently
- $f(x)$ has a solution $\Leftrightarrow x$ has a solution

(We studied these in a different context in an earlier section.)

Let's check out the reduction $3SAT \rightarrow INDEPENDENT SET$.

Q: What do we need?

A: A function that takes a 3SAT instance and creates an INDEPENDENT SET instance, where there is a satisfying assignment exactly when there's a working independent set.

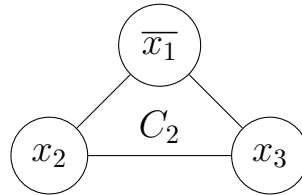
Q: What will that INDEPENDENT SET instance consist of?

A: A graph and an integer.

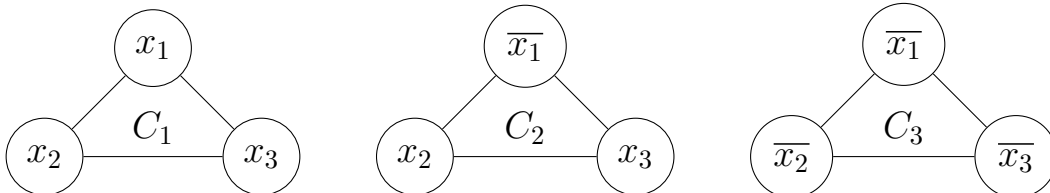
Q: What will that 3SAT instance consist of?

A: A boolean formula in 3-CNF.

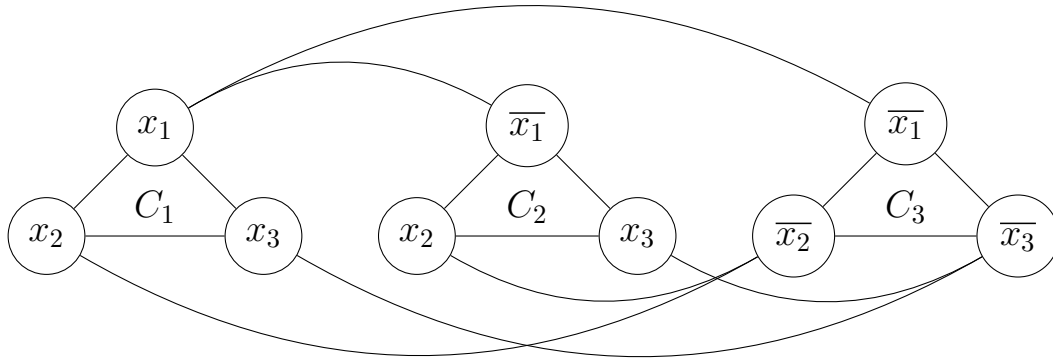
Here's how the reduction works. The integer k will be the number of clauses. For each clause, create a 3-cycle with the vertices labelled by the literals. E.g., for the second clause in our example above, $C_2 = (\overline{x_1} \vee x_2 \vee x_3)$, we'll get this triple:



With the first three clauses, we get this graph:



We're not done yet! Notice that since we're trying to find an independent set of size equal to the number of clauses, that must include choosing exactly one node from each clause gadget. We want those choices to correlate exactly to a satisfying assignment, so we can enforce that by connecting vertices that can't be chosen together. In other words, we're going to add edges between all opposite labels. (E.g. x_1 and $\overline{x_1}$.)



The above graph has an independent set of size 3 exactly when this formula is satisfiable:

$$(x_1 \vee x_2 \vee x_3) \wedge (\overline{x_1} \vee x_2 \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_3})$$

Because of this reduction, we now know that INDEPENDENT SET is also one of these hardest problems in NP. We call these problems NP-complete, which means two things:

- The problem is in NP, and
- The problem is NP-*hard*, which means it's at least as hard as the hardest things in NP.

There are a lot of different complexity classes aside from P, NP, and EXPTIME. I really like the video “P vs NP and the Computational Complexity Zoo” by hackerdashery on YouTube²⁸. Completeness works the same for all of them. A problem Z being X-complete means:

- $Z \in X$, and

²⁸<https://www.youtube.com/watch?v=YX40hbAHx3s>

- Z is X-hard; it's at least as hard as everything in X.

The most common way to show hardness is via a reduction, but they can also be used to show inclusion (or easiness). Remember these two things:

- **Hardness follows a reduction.** If you have a reduction $A \rightarrow B$ and A is X-hard, then B must also be X-hard.
- **Easiness salmons a reduction.** If you have a reduction $A \rightarrow B$ and $B \in X$, then A must also be in X .²⁹

Most proofs that some problem Z is NP-complete:

- Use a reduction from some known NP-hard problem $A \rightarrow Z$, and
- Separately show that $Z \in \text{NP}$ without using a reduction.

Since it has been over 50 years since the discovery of NP-hardness and no one has found an efficient algorithm for any of the hundreds of known NP-complete problems, it is generally assumed that $P \neq \text{NP}$, and thus NP-hard problems are considered *intractible*.

Q: What about our original problem, the ADDING FROM SUBSETS problem? Is that NP-hard?

A: Yes! We can also reduce from 3SAT.

Here's the reduction:³⁰

²⁹My students hate the word *salmons*, but I love it.

³⁰Thank you to Matt Ferland, https://www.dickinson.edu/site/custom_scripts/dc_faculty_profile_index.php?fac=ferlandm, for finding this and explaining it to me.

Given a 3SAT instance with n variables, $x_1, \dots, x_n$, and m clauses, $C_0, \dots, C_{m-1}$ (I'm indexing by zero on purpose here) we need to create our sets S_i and our target sum, k . Let $k = \underbrace{333 \cdots 33}_{m \text{ digits}}$. We will have $n + m$ sets:

- $\forall \text{ variable } x_i : p_i = \sum_{x_i \in C_j} 10^j$

and $q_i = \sum_{\bar{x}_i \in C_j} 10^j$

Then $S_i = \{p_i, q_i\}$

- $\forall C_i : S_{n+i+1} = \{0, 10^i, 2 \times 10^i\}$

Now, so long as we can get anywhere between a 1 and a 3 in each digit from the first n sets, we can use the remaining m to get to all the 3s we need. However, getting those digits from the first n sets is exactly finding a satisfying assignment from the 3SAT instance.

Exercises for 11.3

Exercise 0

Consider three computational problems, A , B , and C such that B is NP-hard and C is in PSPACE. Assume we find two reductions, $f_1: A \rightarrow B$ and $f_2: B \rightarrow C$.

- What's the strongest thing we can say about the computational complexity of A and why?
- What's the strongest thing we can say about the computational complexity of B and why?

- c) What's the strongest thing we can say about the computational complexity of C and why?
- d) Would any of this change our current understanding of complexity classes?

(Answer 11.3.0 in Appendix)

Exercise 1

Consider five computational problems, A , B , C , D and E such that B is NP-hard and D is in NP. Assume we find four reductions, $f_1: A \rightarrow B$, $f_2: B \rightarrow C$, $f_3: C \rightarrow D$, and $f_4: D \rightarrow E$.

- a) What's the strongest thing we can say about the computational complexity of A and why?
- b) What's the strongest thing we can say about the computational complexity of B and why?
- c) What's the strongest thing we can say about the computational complexity of C and why?
- d) What's the strongest thing we can say about the computational complexity of D and why?
- e) What's the strongest thing we can say about the computational complexity of E and why?
- f) Would any of this change our current understanding of complexity classes?

Exercise 2

Consider three computational problems, A , B , and C such that A is NP-hard and B is in EXPTIME. Assume we find two reductions, $f_1: A \rightarrow B$ and $f_2: B \rightarrow C$. For each of the following scenarios, determine whether it (i) follows our current understanding of computational complexity classes, (ii) shows that some of our complexity classes are equal, or (iii) is inconsistent with the known relationships between classes. Explain your rationale for each choice.

- An algorithm to solve C is found that runs in $\Theta(n^7)$ time.
- C is shown to be PSPACE-hard.

11.4 Using Hardness (Cryptography)

Activity time! Pick a symbol (secretly) from this table:

♣	2	+	6	∞	10	○	14	✓	18	□	22	■	26	⊗	30
◇	3	−	7	∀	11	⊙	15	‡	19	◻	23	◆	27	‡	31
♥	4	×	8	∃	12	∝	16	♣	20	⊠	24	▲	28	⊙	32
♠	5	÷	9	≠	13	⊗	17	♠	21	⊞	25	★	29		

- Take the number associated with your symbol and raise it to the seventh power.
- Then find the remainder of that modulo 33.
- That should be a new number that corresponds to a symbol.
- Tell your instructor what the new symbol is.

You can use python interactive mode to do the calculation if you want, e.g.:

```
$ python3
>>> x = # your secret number
```

```
>>> y = (x ** 7) % 33
>>> print(y)
```

Q: Can your instructor figure out what your original symbol was? (They are also allowed to use technology!)

A:

Q: Did your instructor solve that by inverting $y = (x^7) \bmod 33$? Did they find $\sqrt[7]{y} = x$ in modulo 33?

A: No. Unfortunately, that requires a brute-force computation that's equivalent to trying all numbers from 2 to 32 and hoping we get lucky.

This is an example of a *one-way function*.

Q: What do you think a one-way function is?

A function where:

- A:**
- It's computationally easy (fast) to compute $y = f(x)$, but
 - It's computationally hard to compute $x = f^{-1}(y)$

In particular, this is a *trap-door function*.

Q: What do you think a trap-door function is?

It's a one-way function where there's a secret, s , such that a function g exists where:

- A:**
- $g(s, y) = f^{-1}(y)$ and
 - $g(s, y)$ is computationally easy to compute!

Q: Is there a trap-door function in the above scheme?

A: There must be, otherwise it wouldn't have worked.

Q: Wait, why can't someone just build a table to “undo” the symbol table above?

A: A *message* to send from one person to the next isn't just a single symbol, but a number based on the concatenation of a bunch of letters in order.

Q: How big is that (binary) number/string?

A: $\Theta(n)$, where n is the number of characters.

Q: And how many different possible messages could be in that?

A: An exponential amount! Worse than $\Theta(2^n)$. Because of this, our lookup table would have to be exponentially large; too big to store.

Q: Okay, how does the trap-door work in what we did above?

A:

Q: What were all the numbers from the example we did above?

- 33 is the product of two primes 3×11 , so $p = 3$ and $q = 11$.

- $\varphi(33) = (p - 1) \times (q - 1) = 2 \times 10 = 20$

A: • 7 is coprime with 20, so we can use that as our e .

- $7 \times 3 \bmod 20 = 1$, so 3 works for d .

- Thus, if $y = x^e \bmod n$, then $x = y^d \bmod n$.

31

Q: What is needed for a crypto-system?

Three things:

- A:**
- A fast way to generate keys (key-generating function),
 - An encryption function, and
 - A decryption function.

³¹Thanks to <https://www.cs.utexas.edu/~mitra/honors/soln.html> for the good example to work through that I used here!

Q: What does the key-generation do?

It quickly creates random keys to use in the other two functions:

A: $\text{gen} : S \rightarrow \mathbb{N}^c$, where S is the set of security parameters (possible strengths of the encryption), and c is the number of keys it should generate.

Q: What do the encryption and decryption functions look like more formally? What are the domain and range? What should be true of enc ?

if M is the message space and C is the ciphertext space:

A:

- $\text{enc} : M \rightarrow C$
- $\text{dec} : C \rightarrow M$
- enc needs to be one-way

Q: What if you compose the two? What is $\text{dec}(\text{enc}(m))$?

A: $\text{dec}(\text{enc}(m)) = m$

Q: What are **enc** and **dec** for our encryption function above?

A:

- $\text{enc}(m) = m^7 \bmod 33$
- $\text{dec}(c) = c^3 \bmod 33$

Q: Is **enc** really a one-way function?

A: No, because it's easy to invert if we know to use 3 in the decryption function.

Q: Can we use the **gen** key-generation function to make it more difficult?

A: Yes. We're going to include the secret key as one of the parameters to **dec** so that it's needed to decrypt.

Q: We're describing the RSA Encryption Scheme. What are the three functions in general?

A:

- $\text{gen}(1^k) = (PK, SK) = ((n, e), (n, d))$, where $n = p \times q$ is the product of two primes and $d \times e \bmod ((p-1) \times (q-1)) = 1$.

- $\text{enc}_{PK=(n,e)}(m) = m^e \bmod n$

- $\text{dec}_{SK=(n,d)}(c) = c^d \bmod n$

32

Q: What part is the public key and what part is the secret key in this cryptosystem?

A: In this cryptosystem, the public key (PK) is (n, e) and the secret key (SK) is (n, d) .

Q: Why can't we (efficiently) find the secret key from the public key?

A: All we have is n and e . There's no known polynomial-time algorithm to find d from those numbers. (On a classical computer!)

³²RSA was created in 1977 by Ron Rivest, Adi Shamir, and Len Adleman. More info: https://en.wikipedia.org/wiki/RSA_cryptosystem.

Q:	What if an attacker could factor n into $p \times q$ efficiently?
A:	Then they could multiply $(p - 1) \times (q - 1)$ and determine d from e !

Let's do it again, but with bigger numbers!

In secret, come up with a two-letter message you want to send to me. (I will reveal them to the class.) Encode the two characters in 10-bit ASCII (the entries up through 127 on an ASCII table³³ as s_1 and s_2 . To make your actual numeric message: $m = s_1 + (128 * s_2)$.

My public key is: $(n = 174683, 35)$. Give me your encrypted message! I will decrypt it and read it to the class.

For example, if you want to send me: “yo”, that would be:

$$\begin{aligned}
 s_1 &= \text{ascii}('y') \\
 &= 121 \\
 s_2 &= \text{ascii}('o') \\
 &= 111 \\
 (s_1 + (128 \times s_2))^{35} \bmod 174683 &= (121 + (128 \times 111))^{35} \bmod 174683 \\
 &= 14329^{35} \bmod 174683 \\
 &= 105613
 \end{aligned}$$

(Note to Instructor: You might want to do this out in Python for them in front of the class.)

³³Like here: <https://www.ascii-code.com/>.

Q: Can anyone figure out my secret key?

To decrypt:

1. $m = c^{280} \bmod 174683$

2. $s_1 = m \bmod 128$

3. $s_2 = \lfloor m/128 \rfloor$

My secret key is (174683, 89387). Students might be able to find this by brute-force factoring 174683 into 307 and 569.

A: To decrypt the example above:

$$m = 105613^{89387} \bmod 174683$$

$$= 14329$$

$$s_1 = 14329 \bmod 128$$

$$= 121 \rightarrow \text{'y'}$$

$$s_2 = \lfloor 14329/128 \rfloor$$

$$= 111 \rightarrow \text{'o'}$$

12 Constant-Time Max Finding

Note: this is an extra section that does not correspond to our Levitin textbook.

Q: How long does it take to find the max of an array of n elements with a single processor?

A: We've done this many times: $\Theta(n)$ time.

Q: What if we have n processors?

A: We can use a divide-and-conquer approach!
 $\Theta(\log(n))$ time.

Q: What's the step complexity, $S(n)$ of that algorithm?

A: $\Theta(\log(n))$

Q: Is there any way we can go faster with more processors?

Yes! It's going to take some things:

- A:**
- An algorithm similar to MatrixSort, from the beginning of the semester!
 - A lot of processors, and
 - Perhaps some imaginative physics.

Q: What will have to be true of any non-constant-round for loops we use?

A: They will all have to be parallel loops.

Here's the algorithm we're going to use:³⁴

Algorithm: **MatrixMax**(A)

Input: array $A[0 : n - 1]$

Output: x , the max element of A

```
1: Let  $M$  be an  $n \times n$  matrix.
2: for all  $i \in [0 : n - 1]$  do
3:   for all  $j \in [0 : n - 1]$  do
4:     if  $A[i] > A[j]$  or  $(A[i] = A[j]$  and  $i \geq j)$  then
5:        $M[i][j] \leftarrow 1$ 
6:     else
7:        $M[i][j] \leftarrow 0$ 
8:     end if
9:   end for
10: end for
11: Let  $S$  be an array of length  $n$ 
12: for all  $i \in [0 : n - 1]$  do
13:    $S[i] \leftarrow 1$ 
14:   for all  $j \in [0 : n - 1]$  do
15:     if  $M[i][j] = 0$  then
16:        $S[i] \leftarrow 0$ 
17:     end if
18:   end for
19: end for
```

³⁴Thanks to Marc Smith (<https://www.cs.vassar.edu/people/mlsmith/top>) for teaching me this algorithm in college!


```
20:  $x \leftarrow 0$ 
21: for all  $i \in [0 : n - 1]$  do
22:   if  $S[i] = 1$  then
23:      $x \leftarrow A[i]$ 
24:   end if
25: end for
```

Q: Does this algorithm work?

A:

- Yes, it builds a matrix like in MatrixSort, with 1s and 0s. (There's a small difference where we give ties a 1 instead of a 0.)
- Instead of being the sum of each row, S becomes all zeroes except in the row from M that was all 1s.
- Then in parallel we check S to see whether there's a 1 in that cell. If so, we take the corresponding element of A and write it into x .

Q: What's the $S(n)$ of this algorithm?

A: $\Theta(1)$, every loop is in parallel.

Q: What is $W(n)$?

A: $\Theta(n^2)$

Q: So how many processors do we need to make this run in constant time?

A: $\Theta(n^2)$

Q: Is that a reasonable number of processors?

A: Not really! Many people think linear isn't even a reasonable number of processors.

Q: Okay, why does this need some weird physics?

A: We're not only reading from a variable in parallel, we're *writing* in parallel. We need a physical system that can handle that. I don't know whether that actually exists in conventional computing!

Sequential computing uses an EREW model: Exclusive Read Exclusive Write. CREW, Concurrent Read Exclusive Write, seems reasonable. It's not clear (to me) that CRCW is attainable. If such a physical system exists, then this gives us an

algorithm to accomplish it!

A Answers to Exercises

Answer of exercise 2.2.0

Question:

Using our asymptotic notation definitions:

- a) Show that $20n^3 + 21n^2 + 22n + 23 \in O(n^3)$
- b) Show that $20n^3 + 21n^2 + 22n + 23 \in \Omega(n^3)$
- c) Use the previous two parts to show that $20n^3 + 21n^2 + 22n + 23 \in \Theta(n^3)$

Answer:

For all parts, let $f(n) = 20n^3 + 21n^2 + 22n + 23$ and $g(n) = n^3$.

- a) Let $n_0 = 1$ and $c = 86$. Then:

$$\begin{aligned}\forall n \geq n_0 : f(n) &= 20n^3 + 21n^2 + 22n + 23 \\ &\leq 20n^3 + 21n^3 + 22n^3 + 23n^3, \text{ since } n \geq 1 \\ &= 86n^3 \\ &= c \cdot g(n)\end{aligned}$$

Thus, $\forall n \geq n_0 : f(n) \leq c \cdot g(n)$, meaning $f(n) \in O(n^3)$.

- b) Let $n_0 = 1$ and $c = 20$. Then:

$$\begin{aligned}\forall n \geq n_0 : f(n) &= 20n^3 + 21n^2 + 22n + 23 \\ &\geq 20n^3, \text{ since } n \geq 0 \\ &= c \cdot g(n)\end{aligned}$$

Thus, $\forall n \geq n_0 : f(n) \geq c \cdot g(n)$, meaning $f(n) \in \Omega(n^3)$.

- c) Since $f(n) \in O(n^3)$ and $f(n) \in \Omega(n^3)$, $f(n) \in O(n^3) \cap \Omega(n^3)$ and thus $f(n) \in \Theta(n^3)$.

Answer of exercise 2.2.1

Question:

Using our asymptotic notation definitions:

- a) Show that $3n^3 - 10n^2 - 5n + 10 \in O(n^3)$
- b) Show that $3n^3 - 10n^2 - 5n + 10 \in \Omega(n^3)$
- c) Use the previous two parts to show that $3n^3 - 10n^2 - 5n + 10 \in \Theta(n^3)$

Answer:

For all parts, let $f(n) = 3n^3 - 10n^2 - 5n + 10$ and $g(n) = n^3$.

- a) Let $n_0 = 1$ and $c = 13$. Then:

$$\begin{aligned}\forall n \geq n_0 : f(n) &= 3n^3 - 10n^2 - 5n + 10 \\ &\leq 3n^3 + 10, \text{ since } n \geq 0 \\ &\leq 3n^3 + 10n^3, \text{ since } n \geq 1 \\ &= 13n^3 \\ &= c \cdot g(n)\end{aligned}$$

Thus, $\forall n \geq n_0 : f(n) \leq c \cdot g(n)$, meaning $f(n) \in O(n^3)$.

- b) Let $n_0 = 15$ and $c = 1$. Then:

$$\begin{aligned}\forall n \geq n_0 : f(n) &= 3n^3 - 10n^2 - 5n + 10 \\ &> 3n^3 - 10n^2 - 5n \\ &= 2n^3 + n^3 - 10n^2 - 5n \\ &\geq 2n^3 + n^3 - 10n^2 - 5n^2, \text{ since } n \geq 1 \\ &= 2n^3 + n^3 - 15n^2 \\ &= 2n^3 + n^2(n - 15) \\ &\geq 2n^3, \text{ since } n \geq 15 \\ &= c \cdot g(n)\end{aligned}$$

Thus, $\forall n \geq n_0 : f(n) \geq c \cdot g(n)$, meaning $f(n) \in \Omega(n^3)$.

- c) Since $f(n) \in O(n^3)$ and $f(n) \in \Omega(n^3)$, $f(n) \in O(n^3) \cap \Omega(n^3)$ and thus $f(n) \in \Theta(n^3)$.

Answer of exercise 2.2.2

Answer Not Provided

Answer of exercise 2.2.3

Answer Not Provided

Answer of exercise 2.2.4

Answer Not Provided

Answer of exercise 2.2.5

Answer Not Provided

Answer of exercise 2.2.6

Question:

Simplify each of the following expressions.

- a) $\Omega(n^4) \cap \Theta(n^2)$
- b) $\Omega(n^4) \cap \Theta(n^5)$
- c) $O(n) \cup O(n^3)$

Answer:

- a) The two sets are disjoint, so $\Omega(n^4) \cap \Theta(n^2) = \emptyset$
- b) $\Theta(n^5) \subset \Omega(n^4)$, so $\Omega(n^4) \cap \Theta(n^5) = \Theta(n^5)$
- c) $O(n) \subset O(n^3)$, so $O(n) \cup O(n^3) = O(n^3)$

Answer of exercise 2.2.7

Answer Not Provided

Answer of exercise 2.2.8

Answer Not Provided

Answer of exercise 2.3.0

Question:

How long does it take to create a new binary search tree of integers and add the number 5 to it n times? (Assume your tree is able to hold multiple copies of the same value.) Write out the code if that helps you analyze the running time.

Answer:

```
BinarySearchTree<Integer> tree = new BinarySearchTree<>();  
for (int i = 0; i < n; i++) {  
    tree.add(5);  
}
```

add takes $\Theta(\log(n))$ time, so in total this requires $n \times \Theta(\log(n)) = \Theta(n \log(n))$ time.

Answer of exercise 2.3.1

Answer Not Provided

Answer of exercise 2.3.2

Answer Not Provided

Answer of exercise 2.3.3

Question:

How long does it take to reverse the elements in a queue of n elements? Write out the code if that helps you analyze the running time.

Answer:

```
Stack tempStack = new Stack();
while (!queue.isEmpty()) {
    tempStack.push(queue.remove());
}
while (!tempStack.isEmpty()) {
    queue.add(tempStack.pop());
}
```

push, pop, remove, and add each take $\Theta(1)$ time, and each loop runs n iterations, so in total it takes $2n \times \Theta(1) = \Theta(n)$ time.

Answer of exercise 2.3.4

Answer Not Provided

Answer of exercise 2.3.5

Question:

Using big- Θ notation, describe how long the following code takes to execute. `stack` is a `Stack<String>` with n elements.

```
Queue<Integer> queue = new Queue<>(0);
while (!stack.isEmpty()) {
    queue.add(stack.pop());
}
```

Answer:

`Stack.pop()` takes $\Theta(1)$ time, and `Queue.add()` takes $\Theta(1)$ time. The loop iterates n times, so in total this takes

$$\begin{aligned} n \times (\Theta(1) + \Theta(1)) &= n \times \Theta(1) \\ &= \Theta(n) \text{ time} \end{aligned}$$

Answer of exercise 2.3.6

Answer Not Provided

Answer of exercise 2.3.7

Answer Not Provided

Answer of exercise 2.3.8

Answer Not Provided

Answer of exercise 2.3.9

Answer Not Provided

Answer of exercise 2.4.0

Question:

Consider the algorithm, H, below:

Algorithm: $H(n)$

Input: integer n

Output: integer s

```
1:  $s \leftarrow 0$ 
2: for  $i \in [1 : n]$  do
3:    $s \leftarrow s + i$ 
4: end for
```

- a) Find the running time of H using big- Θ notation in terms of n .
- b) Write out the formula to calculate the return value in terms of the parameter. (Hint: summation!)
- c) That formula is very common and has a nice simplification. If you don't know it, look it up online. Show the simplification and list where you found the simplification of the formula.

- d) Write a new algorithm, $J(n)$, to calculate and return the same value using the simplified formula. Calculate the running time of that formula.

Answer:

- a) If we count all operations, it looks like:
- Line 1 contributes 1 operation.
 - Line 2 contributes 2 per cycle (conditional check and assignment)
 - Line 3 contributes 2 per cycle (addition, and assignment)
 - The loop from lines 2 to 4 runs n times.

In total, that's $T(n) = 4n + 1$. That's a polynomial, so we can easily see that $T(n) \in \Theta(n)$.

b) $s = \sum_{i=1}^n i$

- c) We derived it earlier in these notes, but it's also described at the Wikipedia entry for Triangular Numbers: https://en.wikipedia.org/wiki/Triangular_number. $s = n(n+1)/2$
- d) We can simplify the algorithm significantly:

Algorithm: $J(n)$

Input: integer n

Output: integer s

1: $s \leftarrow (n \times (n + 1))/2$

There are three arithmetic operations happening in that line, so the running time is $\Theta(3) = \Theta(1)$.

Answer of exercise 2.4.1

Answer Not Provided

Answer of exercise 2.4.2

Answer Not Provided

Answer of exercise 2.4.3

Question:

Consider an algorithm with these two parallel running times:

- $TP(n, 1) = 10n^4 + 8n^3 \log(n) + 10n^2$
- $TP(n, \infty) = 121n^2$

Use Brent's Theorem to find $TP(n, n^3)$.

Answer:

$$\begin{aligned} TP(n, n^3) &= \Theta \left(\frac{W(n)}{n^3} + S(n) \right) \\ &= \Theta \left(\frac{TP(n, 1)}{n^3} + TP(n, \infty) \right) \\ &= \Theta \left(\frac{10n^4 + 8n^3 \log(n) + 10n^2}{n^3} + 121n^2 \right) \\ &= \Theta \left(10n + 8 \log(n) + \frac{10}{n} + 121n^2 \right) \\ &= \Theta(121n^2) \\ &= \Theta(n^2) \end{aligned}$$

Answer of exercise 2.4.4

Answer Not Provided

Answer of exercise 2.4.5

Answer Not Provided

Answer of exercise 2.4.6

Question:

Consider the algorithm, H , below:

Algorithm: $H(n)$

Input: arrays A, B , of same length

Output: arrays A, B

```
1: for all  $i \in [0 : \text{length}(A) - 2]$  do
2:    $A[i + 1] \leftarrow B[i]$ 
3:    $B[i + 1] \leftarrow A[i]$ 
4: end for
5: for all  $i \in [0 : \text{length}(A) - 1]$  do
6:    $A[i] \leftarrow A[i] - 1$ 
7: end for
```

- a) Run H on the arrays $A = [1, 2, 3], B = [11, 12, 13]$. Are there multiple possibilities for the value the output can have? If so, list at least two of them. Is there a race condition in H on general inputs?
- b) If there's a race condition, rewrite H by changing the loops that cause race conditions to iterative loops instead of parallel loops.
- c) Find $W(n)$ and $S(n)$ of H (after you rewrote, if you had to).
- d) Find $TP(n, n)$.

Answer:

- a) The first forall loop runs twice because $n = 3$.
- If the $i = 0$ iteration runs completely before the $i = 1$ iteration, then the resulting lists are: $A = [0, 10, 0]$, $B = [11, 1, 11]$.
 - If the $i = 1$ iteration runs completely before the $i = 0$ iteration, then the resulting lists are $A = [0, 10, 11]$, $B = [11, 1, 2]$.
 - If line 2 in iteration $i = 0$ happens first, then all of iteration $i = 1$, then line 3 of iteration $i = 0$, then the resulting lists are: $A = [0, 10, 11]$, $B = [11, 1, 11]$. This is also the result if line 2 of iteration $i = 1$ happens first, then all of iteration $i = 0$, followed by line 3 of iteration $i = 1$.

There are multiple possible outputs, so there is a race condition here. If there's a race condition here, there are race conditions in general.

- b) Only the first parallel loop causes race conditions. Here's the new version of H :

Algorithm: $H(n)$

Input: arrays A , B , of same length

Output: arrays A , B

```
1: for  $i \in [0 : \text{length}(A) - 2]$  do
2:    $A[i + 1] \leftarrow B[i]$ 
3:    $B[i + 1] \leftarrow A[i]$ 
4: end for
5: for all  $i \in [0 : \text{length}(A) - 1]$  do
6:    $A[i] \leftarrow A[i] - 1$ 
7: end for
```

c)

$$\begin{aligned}W(n) &= TP(n, 1) \\&= n \times \Theta(2) + n \times \Theta(1) \\&= \Theta(n) \\S(n) &= TP(n, \infty) \\&= n \times \Theta(2) + \Theta(1) \\&= \Theta(n)\end{aligned}$$

d)

$$\begin{aligned}TP(n, n) &= \Theta \left(\frac{W(n)}{n} + S(n) \right) \\&= \Theta \left(\frac{\Theta(n)}{n} + \Theta(n) \right) \\&= \Theta (\Theta(1) + \Theta(n)) \\&= \Theta(n)\end{aligned}$$

Answer of exercise 2.4.7

Answer Not Provided

Answer of exercise 2.4.8

Answer Not Provided

Answer of exercise 2.5.0

Answer Not Provided

Answer of exercise 3.1.0

Question:

Execute **SelectionSort** on the list $[3, 1, 2]$. Show how the list is modified with each change (i.e. every time a swap is made).

Answer:

- $[3, 1, 2]$
- $[1, 3, 2]$
- $[1, 2, 3]$ ✓

Answer of exercise 3.1.1

Answer Not Provided

Answer of exercise 3.1.2

Question:

Execute (sequential) **BubbleSort** on the list $[3, 2, 1]$. Show how the list is modified with each change (i.e. every time a swap is made).

Answer:

- $[3, 2, 1]$
- $[2, 3, 1]$
- $[2, 1, 3]$
- $[1, 2, 3]$ ✓

Answer of exercise 3.1.3

Answer Not Provided

Answer of exercise 3.1.4

Question:

Find $TP(n, n^2)$ for the parallelized version of BubbleSort that we wrote.

Answer:

$$\begin{aligned} TP(n, n^2) &\in \Theta \left(\frac{W(n)}{n^2} + S(n) \right) \\ &= \Theta \left(\frac{\Theta(n^2)}{n^2} + \Theta(n) \right) \\ &= \Theta(\Theta(1) + \Theta(n)) \\ &= \Theta(\Theta(n)) \\ &= \Theta(n) \end{aligned}$$

Answer of exercise 3.1.5

Answer Not Provided

Answer of exercise 3.2.0

Question:

Consider an array of strings.

- a) Write an (optimal) algorithm to determine whether any of those strings equals "monkey".
- b) Find $T(n)$, the running time of your algorithm when the array has length n .
- c) Find $TP(n, n)$, the running time of your algorithm with n processors. (Update your algorithm to run in parallel if you need to.)

Answer:

a) Algorithm: **hasMonkey(A)**

Input: A , an array of strings

Output: b , whether A contains "monkey"

```
1:  $b \leftarrow \text{false}$ 
2: for all  $i \in [0, \dots, A.length - 1]$  do
3:   if  $A[i] = \text{"monkey"}$  then
4:      $b \leftarrow \text{true}$ 
5:   end if
6: end for
```

b) The loop runs n times and has a constant number of steps in it, for a total of $T(n) = \Theta(n)$ time.

c) Since the loop takes constant time, $S(n) \in \Theta(1)$

$$\begin{aligned} TP(n, n) &\in \Theta\left(\frac{W(n)}{n} + S(n)\right) \\ &= \Theta\left(\frac{\Theta(n)}{n} + \Theta(1)\right) \\ &= \Theta(\Theta(1) + \Theta(1)) \\ &= \Theta(\Theta(1)) \\ &= \Theta(1) \end{aligned}$$

Answer of exercise 3.2.1

Answer Not Provided

Answer of exercise 3.2.2

Question:

- a) Write a sequential search algorithm to determine whether a `Stack<Integer>` of size n has any neighboring numbers that are equal. (Your algorithm has to return the stack to its original state when it's done.)
- b) Find $T(n)$, the running time of your algorithm when the stack has a size of n .
- c) Find $TP(n, n)$, the running time of your algorithm with n processors. (Update your algorithm to run in parallel if you need to.)

Answer:

- a) Algorithm: `equalStackNeighbors(s)`

Input: s , a Stack of integers

Output: b , whether s has any equal neighboring elements

```
1:  $b \leftarrow \text{false}$ 
2:  $t \leftarrow \text{new Stack}$ 
3:  $x \leftarrow \text{"banana"}$ 
4: while  $s$  not empty do
5:    $y \leftarrow s.\text{pop}()$ 
6:   if  $x = y$  then
7:      $b \leftarrow \text{true}$ 
8:   end if
9:    $t.\text{push}(x)$ 
10:   $x \leftarrow y$ 
11: end while
12: while  $t$  not empty do
13:    $s.\text{push}(t.\text{pop}())$ 
14: end while
```

We cannot run either of the loops in parallel because the `Stack(s)` could get out of order.

- b) The loops each run n times. Since the `Stack.pop` and `Stack.push` methods each run in constant time, and there are a constant number of other steps in the loops, the each run of the loops takes $\Theta(1)$ time. Thus, $T(n) = n \times \Theta(1) = \Theta(n)$ time.
- c) We cannot parallelize either of the loops, because if we were able to modify them to be for all loops, the stacks could get out of order. Since there are no parallel elements, $S(n) \in \Theta(n)$, so

$$\begin{aligned} TP(n, n) &\in \Theta \left(\frac{W(n)}{n} + S(n) \right) \\ &= \Theta \left(\frac{\Theta(n)}{n} + \Theta(n) \right) \\ &= \Theta(\Theta(1) + \Theta(n)) \\ &= \Theta(\Theta(n)) \\ &= \Theta(n) \end{aligned}$$

Answer of exercise 3.2.3

Answer Not Provided

Answer of exercise 3.2.4

Answer Not Provided

Answer of exercise 3.3.0

Question:

- a) Find $TP(n, n^2)$ for the parallel version of the nearest neighbors algorithm that we did above/in class.
- b) How does this compare with $TP(n, n)$?
- c) What is the smallest number of processors that gives us the best time?

Answer:

a)

$$\begin{aligned}
 TP(n, n^2) &\in \Theta \left(\frac{W(n)}{n^2} + S(n) \right) \\
 &= \Theta \left(\frac{\Theta(n^2)}{n^2} + \Theta(n) \right) \\
 &= \Theta (\Theta(1) + \Theta(n)) \\
 &= \Theta (\Theta(n)) \\
 &= \Theta(n)
 \end{aligned}$$

- b) In class we saw that $TP(n, n) = \Theta(n)$, so this is the same running time.
- c) The parallel for loop can run with $n - 1$ processors, if that many are available, so that's the maximum number of processors that are useful.

Answer of exercise 3.3.1

Answer Not Provided

Answer of exercise 3.3.2

Answer Not Provided

Answer of exercise 3.5.0

Question:

- a) Consider a graph implemented using adjacency lists. Write an optimal algorithm to return whether the graph contains two vertices that are only connected to each other.
- b) Find $T((n, m))$, the running time of your algorithm if the graph has n vertices and m edges.
- c) Find $TP((n, m), n)$, the running time of your algorithm with n processors. (Update your algorithm to run faster with n processors if it helps it run faster.)

Answer:

- a) Algorithm: `hasDoubleton(g)`
Input: g , a graph using adjacency lists
Output: b , whether g has two vertices that are only adjacent to each other

```
1:  $b \leftarrow \text{false}$ 
2: for all  $v \in g.\text{vertices}$  do
3:   if  $v.\text{adjacent.length} = 1$  then
4:      $u \leftarrow v.\text{adjacent}[0]$ 
5:     if  $u.\text{adjacent.length} = 1$  then
6:        $b \leftarrow \text{true}$ 
7:     end if
8:   end if
9: end for
```
- b) The loop runs n times. Inside the loop it does not iterate over the edges so it just takes constant steps. Thus, $T((n, m)) = n \times \Theta(1) = \Theta(n)$

- c) $S((n, m)) = \Theta(1)$ since the outer loop is a forall loop and there are constant steps inside of it.

$$\begin{aligned} TP((n, m), n) &\in \Theta \left(\frac{W((n, m))}{n} + S((n, m)) \right) \\ &= \Theta \left(\frac{\Theta(n)}{n} + \Theta(1) \right) \\ &= \Theta(\Theta(1) + \Theta(1)) \\ &= \Theta(\Theta(1)) \\ &= \Theta(1) \end{aligned}$$

Answer of exercise 3.5.1

Answer Not Provided

Answer of exercise 3.5.2

Answer Not Provided

Answer of exercise 3.5.3

Answer Not Provided

Answer of exercise 3.5.4

Question:

Write out the parallel version of BFS that we first came up with (not the final version). Remember that in this version, we changed the inner loop to be forall and checked that the vertex was unmarked before adding it to the Queue.

Answer:

Algorithm: BFS2

Input: vertex v , value $value$

Output: b , whether $value$ is on a vertex in G

```
1:  $b \leftarrow false$ 
2:  $Q \leftarrow \text{new Queue}$ 
3:  $Q.add(v)$ 
4: while  $Q$  is not empty do
5:    $w \leftarrow Q.remove()$ 
6:   if  $w.marked \neq true$  then
7:      $w.marked \leftarrow true$ 
8:     if  $w.value = value$  then
9:        $b \leftarrow true$ 
10:    break
11:   else
12:     for all  $u \in w.adjacent$  do
13:       if  $u.marked \neq true$  then
14:          $Q.add(u)$ 
15:       end if
16:     end for
17:   end if
18: end if
19: end while
```

Answer of exercise 3.5.5

Answer Not Provided

Answer of exercise 3.5.6

Answer Not Provided

Answer of exercise 3.5.7

Answer Not Provided

Answer of exercise 3.5.8

Question:

Find $TP((n, m), m)$ for the DFS algorithm we came up with in class.

Answer:

$$\begin{aligned} TP((n, m), m) &\in \Theta \left(\frac{W((n, m))}{m} + S((n, m)) \right) \\ &= \Theta \left(\frac{\Theta(m)}{m} + \Theta(n) \right) \\ &= \Theta (\Theta(1) + \Theta(n)) \\ &= \Theta (\Theta(n)) \\ &= \Theta(n) \end{aligned}$$

Answer of exercise 3.5.9

Answer Not Provided

Answer of exercise 3.5.10

Answer Not Provided

Answer of exercise 3.5.11

Question:

Find $TP((n, m), n)$ for the final BFS algorithm we came up with in class. (I recommend doing 3.5.5 first, but it's not strictly necessary.)

Answer:

$$\begin{aligned} TP((n, m), n) &\in \Theta \left(\frac{W((n, m))}{n} + S((n, m)) \right) \\ &= \Theta \left(\frac{\Theta(m)}{n} + \Theta(n) \right) \\ &= \Theta \left(\frac{O(n^2)}{n} + \Theta(n) \right), \text{ since } m \in O(n^2) \\ &= \Theta(\Theta(n) + \Theta(n)) \\ &= \Theta(\Theta(n)) \\ &= \Theta(n) \end{aligned}$$

Answer of exercise 3.5.12

Answer Not Provided

Answer of exercise 4.0.0

Answer Not Provided

Answer of exercise 4.0.1

Answer Not Provided

Answer of exercise 4.1.0

Question:

Execute `InsertionSort` on the list $[3, 2, 1]$. Show how the list is modified after each step.

Answer:

- $[3, 2, 1]$
- $[2, 3, 1]$
- $[2, 1, 3]$

- $[1, 2, 3]$

Answer of exercise 4.1.1

Answer Not Provided

Answer of exercise 4.4.0

Answer Not Provided

Answer of exercise 4.4.1

Question:

What happens to our running time for `BinarySearch` if we (nonsensically) have to look at both sides, so the recurrence is as follows?

$$T(n) = \begin{cases} 1 & , n \leq 1 \\ 2T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 2 & , n > 1 \end{cases}$$

What is the running time of $T(n)$ now? (If you've already learned the Master Theorem, don't use it.)

Answer:

$$\begin{aligned}
 T(k) &= 2T\left(\left\lfloor \frac{k}{2} \right\rfloor\right) + 2 \\
 &= 2\left(2T\left(\left\lfloor \frac{k}{4} \right\rfloor\right) + 2\right) + 2 \\
 &= 4T\left(\left\lfloor \frac{k}{4} \right\rfloor\right) + 6 \\
 &= 4\left(2T\left(\left\lfloor \frac{k}{8} \right\rfloor\right) + 2\right) + 6 \\
 &= 8T\left(\left\lfloor \frac{k}{8} \right\rfloor\right) + 14 \\
 &\vdots \\
 &= 2^i T\left(\left\lfloor \frac{k}{2^i} \right\rfloor\right) + 2(2^i - 1) \\
 &= 2^{\log(k)} T\left(\left\lfloor \frac{k}{2^{\log(k)}} \right\rfloor\right) + 2(2^{\log(k)} - 1) \\
 &\approx kT(1) + 2k - 2 \\
 &= 3k - 2
 \end{aligned}$$

So $T(n) \in \Theta(n)$.

Answer of exercise 4.4.2

Question:

What happens to our running time for `BinarySearch` if the recurrence is as follows?

$$T(n) = \begin{cases} 1 & , n \leq 1 \\ 3T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 2 & , n > 1 \end{cases}$$

What is the running time of $T(n)$ now? (If you've already learned the Master Theorem, don't use it.)

Answer:

$$\begin{aligned}
 T(k) &= 3T\left(\left\lfloor \frac{k}{2} \right\rfloor\right) + 2 \\
 &= 3\left(3T\left(\left\lfloor \frac{k}{4} \right\rfloor\right) + 2\right) + 2 \\
 &= 9T\left(\left\lfloor \frac{k}{4} \right\rfloor\right) + 8 \\
 &= 9\left(3T\left(\left\lfloor \frac{k}{8} \right\rfloor\right) + 2\right) + 8 \\
 &= 27T\left(\left\lfloor \frac{k}{8} \right\rfloor\right) + 26 \\
 &\vdots \\
 &= 3^i T\left(\left\lfloor \frac{k}{2^i} \right\rfloor\right) + 2\left(\sum_{j=0}^{i-1} 3^j\right) \\
 &= 3^{\log(k)} T\left(\left\lfloor \frac{k}{2^{\log(k)}} \right\rfloor\right) + 2\left(\sum_{j=0}^{\log(k)-1} 3^j\right) \\
 &\approx 3^{\log(k)} T(1) + 2\left(\sum_{j=0}^{\log(k)-1} 3^j\right) \\
 &= (2^{\log(3)})^{\log(k)} + 2\left(\sum_{j=0}^{\log(k)-1} 3^j\right) \\
 &= k^{\log(3)} + 2\left(\sum_{j=0}^{\log(k)-1} 3^j\right) \\
 &\leq k^{\log(3)} + 2(2 \cdot 3^{\log(k)}/3) \\
 &= k^{\log(3)} + \frac{4}{3} \cdot k^{\log(3)} = \frac{7}{3} k^{\log(3)}
 \end{aligned}$$

So $T(n) \in \Theta(n^{\log(3)})$.

Answer of exercise 4.4.3

Answer Not Provided

Answer of exercise 5.0.0

Question:

Use the Master Theorem to check our math from the beginning of Section 4, that for:

$$T(n) = \begin{cases} 1 & , n \leq 1 \\ T(\lfloor \frac{n}{2} \rfloor) + 2 & , n > 1 \end{cases}$$

$$T(n) \in \Theta(\log(n))$$

Answer:

To use the Master Theorem, we need to find a , b , and d .
 $a = 1, b = 2, d = 0$

$$a \leq b^d$$

$$1 \leq 2^0$$

$$1 = 1, \text{ case 2: } \Theta(n^d \log(n))$$

$$\begin{aligned} \text{Thus, } T(n) &\in \Theta(n^d \log(n)) \\ &= \Theta(n^0 \log(n)) \\ &= \Theta(\log(n)) \checkmark \end{aligned}$$

Yes, $T(n) \in \Theta(\log(n))$

Answer of exercise 5.0.1

Answer Not Provided

Answer of exercise 5.0.2

Answer Not Provided

Answer of exercise 5.0.3

Answer Not Provided

Answer of exercise 5.0.4

Question:

Use the Master Theorem to solve for $T(n)$, where

$$T(n) = \begin{cases} 1 & , n \leq 1 \\ 10T(\lfloor \frac{n}{3} \rfloor) + n^2 + n + 1 & , n > 1 \end{cases}$$

Answer:

Master Theorem time! $a = 10, b = 3, d = 2$

$$a \square b^d$$

$$10 \square 3^2$$

$$10 > 9, \text{ case 3: } \Theta(n^{\log_b(a)})$$

$$\begin{aligned} \text{Thus, } T(n) &\in \Theta(n^{\log_b(a)}) \\ &= \Theta(n^{\log_3(10)}) \end{aligned}$$

We can't simplify that any more. If it helps to get an idea of what $\log_3(10)$ is, you can ask a calculator, but the answer should still be listed as above. So, $T(n) \in \Theta(n^{\log_3(10)})$. ($\log_3(10) < 2.1$)

Answer of exercise 5.0.5

Answer Not Provided

Answer of exercise 5.0.6

Answer Not Provided

Answer of exercise 5.0.7

Answer Not Provided

Answer of exercise 5.0.8

Answer Not Provided

Answer of exercise 5.1.0

Question:

Execute **Merge** on the two lists $[1, 5], [2, 2, 3]$. Show each change as an element is taken from one of the two lists to put into the merged list. (You can remove the elements from the two lists getting merged if you want, though that's not actually what's happening if they're arrays.)

Answer:

- $[] \leftarrow [1, 5], [2, 2, 3]$
- $[1] \leftarrow [5], [2, 2, 3]$
- $[1, 2] \leftarrow [5], [2, 3]$
- $[1, 2, 2] \leftarrow [5], [3]$
- $[1, 2, 2, 3] \leftarrow [5], []$
- $[1, 2, 2, 3, 5] \leftarrow [], []$

Answer of exercise 5.1.1

Answer Not Provided

Answer of exercise 5.1.2

Answer Not Provided

Answer of exercise 5.1.3

Question:

Execute **MergeSort** on the list $[4, 2, 1, 3]$ (using only one processor). Show each of the split and merge steps. Don't skip splits and merges, even on already-sorted lists!

Answer:

- Split $[4, 2, 1, 3] \rightarrow [4, 2], [1, 3]$
- Split $[4, 2] \rightarrow [4], [2]$
- Merge $[4], [2] \rightarrow [2, 4]$
- Split $[1, 3] \rightarrow [1], [3]$
- Merge $[1], [3] \rightarrow [1, 3]$
- Merge $[2, 4], [1, 3] \rightarrow [1, 2, 3, 4]$

Answer of exercise 5.1.4

Answer Not Provided

Answer of exercise 5.1.5

Answer Not Provided

Answer of exercise 5.1.6

Answer Not Provided

Answer of exercise 5.2.0

Question:

Execute **QuickSortBasic** on the list $[2, 4, 1, 3]$ (using only one processor). Instead of randomly generating a pivot, p , just assume you always choose 0. As with the **MergeSort** exercises in the prior section, show how you break up the array at each step, then copy them back together. (I also wrote the pivot between

the two split arrays. See the solution to this problem for a good example.)

Answer:

- Split $[2, 4, 1, 3] \rightarrow [1], 2, [4, 3]$
- Split $[4, 3] \rightarrow [3], 4, []$
- Recombine $[3], 4, [] \rightarrow [3, 4]$
- Recombine $[1], 2, [3, 4] \rightarrow [1, 2, 3, 4]$

Answer of exercise 5.2.1

Answer Not Provided

Answer of exercise 5.2.2

Answer Not Provided

Answer of exercise 5.2.3

Question:

Execute **QuickSort** (the advanced version) on the list $[2, 4, 1, 3]$ (using only one processor). Instead of randomly generating a pivot, p , just assume you always choose *low*. Show each of the swaps, and (symbolically) break up the lists when you make the recursive calls. (See the solution to this problem for a good example.)

Answer:

- $[2, 4, 1, 3]$
- $[1, 4, 2, 3]$
- $[1, 2, 4, 3]$

- $[1], 2, [4, 3]$
- $[1], 2, [3, 4]$
- $[1], 2, [3], 4$

Answer of exercise 5.2.4

Answer Not Provided

Answer of exercise 5.2.5

Answer Not Provided

Answer of exercise 5.2.6

Question:

We could consider the “average” case of `QuickSort` being the one where we split the array a quarter of the way down. That would make our recurrence relation be: (*AT* for Average Time)

$$AT(n) \in \begin{cases} c & , n \leq n_0 \\ AT(3n/4) + AT(n/4) + \Theta(n) & , n > n_0 \end{cases}$$

We could use this to pretty confidently say that $ET(n) \approx AT(n)$. Unfortunately, the formula for *AT* doesn’t match anything we can apply our version of the Master Theorem to. We could, however, overestimate and say that $ET(n)$ is in big-*O* of that overestimation. For that overestimation, we could pretend there are two recursive calls of size $3n/4$ instead of two calls of different sizes. As it turns out, that (sadly) doesn’t result in the $\Theta(n \log(n))$ that more advanced math tells us is correct for $ET(n)$.

In the opposite direction, what similar trick could we use to show a *lower* bound on the expected running time of $ET(n)$?

Do the math out to find the lower bound and express ET in terms of that.

Sadly, this lower bound doesn't tell us anything new. Why is that?

Answer:

Let $BACT(n)$ (short for Below Average Case) be:

$$BACT(n) \in \begin{cases} 1 & , n \leq 1 \\ 2BACT(n/4) + \Theta(n) & , n > 1 \end{cases}$$

This replaces the bigger recursive call of $AT(n)$ with another of the smaller one, so $BACT(n) \leq AT(n)$. It seems like a pretty significant underestimate, so it's safe to assume that $ET(n) \in \Omega(BACT)$.

In the formula for $BACT$, $a = 2$, $b = 4$, and $d = 1$

$$\begin{aligned} a &\square b^d \\ 2 &\square 4^1 \\ 2 &< 4 \end{aligned}$$

So we're in case 1: $\Theta(n^d) = \Theta(n^1) = \Theta(n)$.

Thus, $ET(n) \in \Omega(n)$.

Unfortunately, this is not a useful lower bound to use because it's lower than the absolute best-case running time of the algorithm at $\Theta(n \log(n))$. That best case time is a tighter (higher) lower bound, so it's more useful than what we discovered here.

Answer of exercise 5.2.7

Answer Not Provided

Answer of exercise 5.2.8

Question:

Modify the `QuickSortHelper` algorithm to run in parallel.

Answer:

There are two parts to consider:

- We can't easily parallelize the while loop that divides up the array.
- We *can* run the two recursive calls in parallel, though!

Algorithm: `QuickSortHelper`

Input: Array of numbers, A , integers low , $high$

Output: A , sorted from index low to $high$, inclusive

```
1: if  $low < high$  then
2:    $p \leftarrow_R [low, low + 1, \dots, high]$ 
3:   Swap  $A[p] \leftrightarrow A[low]$ 
4:    $i \leftarrow low$ 
5:    $j \leftarrow high$ 
6:    $p \leftarrow low$ 
7:   while  $i < j$  do
8:     if  $A[i] > A[j]$  then
9:       Swap  $A[i] \leftrightarrow A[j]$ 
10:    if  $p = i$  then
11:       $i \leftarrow i + 1$ 
12:       $p \leftarrow j$ 
13:    else
14:       $j \leftarrow j - 1$ 
15:       $p \leftarrow i$ 
16:    end if
17:  else
18:    if  $p = i$  then
19:       $j \leftarrow j - 1$ 
20:    else
```

```
21:          $i \leftarrow i + 1$ 
22:     end if
23: end if
24: end while
25: SimultDo
26:     QuickSortHelper( $A, low, i - 1$ )
27:     QuickSortHelper( $A, i + 1, high$ )
28: EndDo
29: end if
```

Answer of exercise 5.2.9

Answer Not Provided

Answer of exercise 5.2.10

Answer Not Provided

Answer of exercise 5.2.11

Answer Not Provided

Answer of exercise 6.1.0

Question:

- a) Write a presorting algorithm to compute the mode of an array of numbers. (The mode is the element that occurs the most number of times.)
- b) Analyze the running time of your algorithm.
- c) Compare your running time to the running time of the brute-force version of this algorithm. (It may help to describe the brute-force version in words; you do not need to write the algorithm out.) Is the presorting version faster?

Answer:

a) Algorithm: **ArrayMode**

Input: A , an array of numbers

Output: $mode$, the number in A that occurs the most times

```
1: MergeSort( $A$ )
2:  $mode \leftarrow A[0]$ 
3:  $modeCount \leftarrow 0$ 
4:  $x \leftarrow A[0]$ 
5:  $count \leftarrow 0$ 
6: for  $i \in [0, \dots, A.length - 1]$  do
7:    $y \leftarrow A[i]$ 
8:   if  $y = x$  then
9:      $count \leftarrow count + 1$ 
10:    if  $count > modeCount$  then
11:       $modeCount \leftarrow count$ 
12:       $mode \leftarrow x$ 
13:    end if
14:  else
15:     $x \leftarrow y$ 
16:     $count \leftarrow 1$ 
17:  end if
18: end for
```

- b) **MergeSort** takes $\Theta(n \log(n))$ time. The loop runs n times and has constant steps inside, so $T(n) \in \Theta(n \log(n)) + \Theta(n) = \Theta(n \log(n))$ time.
- c) The brute-force algorithm consists of a loop that iterates over each number in the array. For that number, it counts the number of times it exists in the array, by again iterating

over the array. That's a loop within a loop, so it takes $\Theta(n^2)$. Our presorting algorithm is faster.

Answer of exercise 6.1.1

Answer Not Provided

Answer of exercise 6.1.2

Answer Not Provided

Answer of exercise 6.1.3

Answer Not Provided

Answer of exercise 6.3.0

Question:

Consider representing AVL trees with 4-tuples: (value, height, leftSubtree, rightSubtree). We use $\emptyset$ to represent an empty tree.

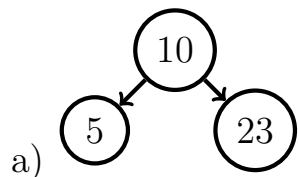
a) Draw the tree represented by the tuple

$$(10, h_1, (5, h_2, \emptyset, \emptyset), (23, h_3, \emptyset, \emptyset))$$

b) What is the value of each h_i ?

c) Is the tree balanced? If any trees or subtrees are unbalanced, which ones?

Answer:



b) • $h_1 = 1$

- $h_2 = 0$
- $h_3 = 0$

c) The tree and both subtrees are balanced.

Answer of exercise 6.3.1

Answer Not Provided

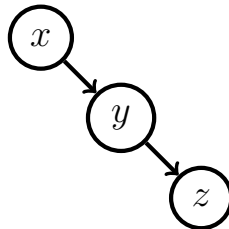
Answer of exercise 6.3.2

Answer Not Provided

Answer of exercise 6.3.3

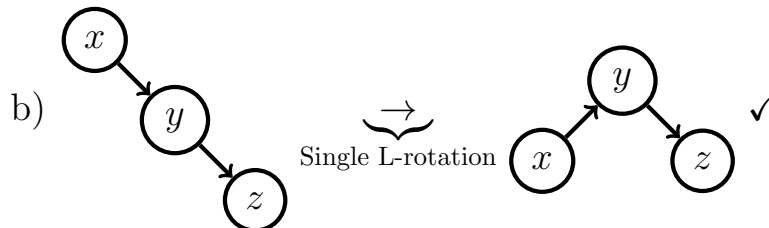
Question:

The following AVL tree just had an element added, and needs to be rebalanced. (a) Determine which element was just added, then (b) show and label the rotation used to perform the rebalancing.



Answer:

a) The tree rooted at x is the lowest unbalanced tree, so z must be the most-recently added node.



Answer of exercise 6.3.4

Answer Not Provided

Answer of exercise 6.3.5

Answer Not Provided

Answer of exercise 6.3.6

Question:

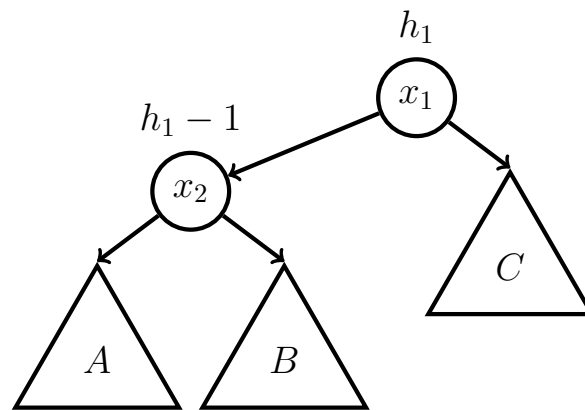
Consider representing AVL trees with 4-tuples: (value, height, leftSubtree, rightSubtree). We use $\emptyset$ to represent an empty tree. Perform a Single R-rotation on

$$(x_1, h_1, (x_2, h_1 - 1, A, B), C)$$

and write the resulting tree in 4-tuple-form in terms of A , B , C , x_1 , x_2 , and h_1 .

Answer:

$(x_1, h_1, (x_2, h_1 - 1, A, B), C)$ is the tree:

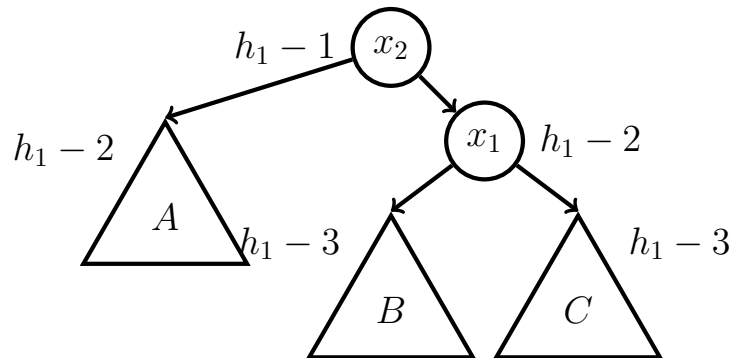


We can also determine the heights of the three subtrees:

- C must be $h_1 - 3$ in order to need the rotation.

- A must be taller than B (otherwise we'd be doing a double-LR rotation instead) so it must have height $h_1 - 2$
- B must then be $h_1 - 3$.

Performing the single-R rotation changes this to:



The final tree, in 4-tuple form, is: $(x_2, h_1 - 1, A, (x_1, h_1 - 2, B, C))$

Answer of exercise 6.3.7

Answer Not Provided

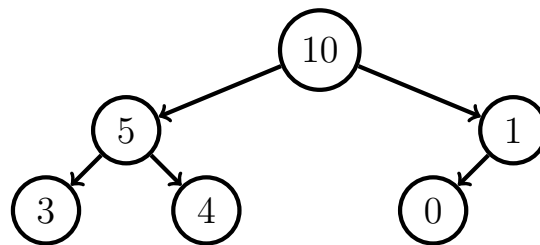
Answer of exercise 6.3.8

Answer Not Provided

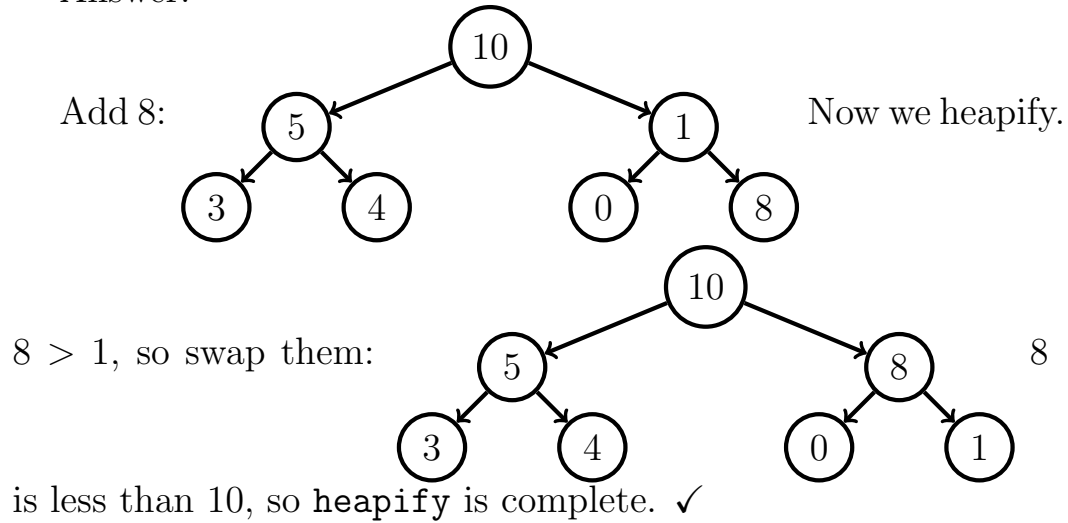
Answer of exercise 6.4.0

Question:

Add the number 8 to the following max-heap. Draw the entire heap with every change that is made during heapification.



Answer:



Answer of exercise 6.4.1

Answer Not Provided

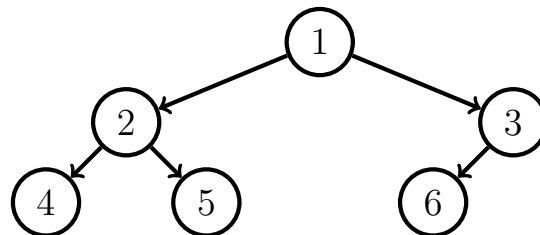
Answer of exercise 6.4.2

Answer Not Provided

Answer of exercise 6.4.3

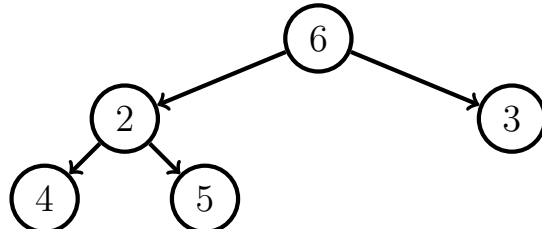
Question:

Remove the element from the root of the following min-heap. Draw the entire heap with every change that is made during reheapification.

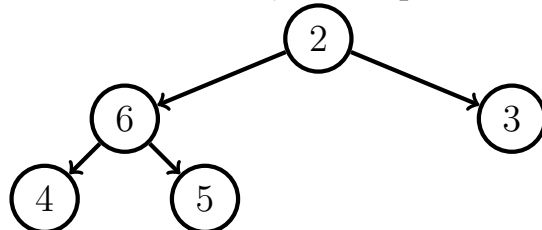


Answer:

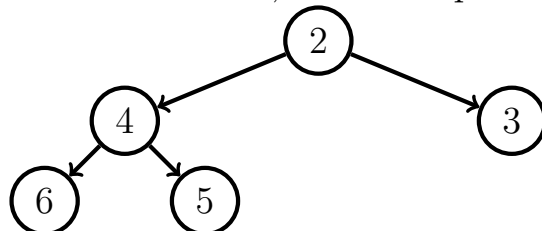
Remove the root and move the last element (6) up to that spot:



$2 < 3$ and $6 > 2$, so swap the 6 and the 2:



$4 < 5$ and $6 > 4$, so we swap the 6 and the 4:



6 doesn't have any further children, so reheapify is complete. ✓

Answer of exercise 6.4.4

Answer Not Provided

Answer of exercise 6.4.5

Answer Not Provided

Answer of exercise 6.4.6

Answer Not Provided

Answer of exercise 6.4.7

Answer Not Provided

Answer of exercise 6.4.8

Answer Not Provided

Answer of exercise 6.4.9

Answer Not Provided

Answer of exercise 6.4.10

Question:

Run `HeapSort` (the array version) on the list $[4, 3, 1, 2]$. Show all changes to the list and the array. (I'm expecting you to use a min-heap, but you can alternatively do it with a max-heap.)

Answer:

With a min-heap, represented as an array:

- Set up: $[4, 3, 1, 2], [\blacksquare, _, _, _] \leftarrow \text{heap}$
- `Heap.add`: $[_, 3, 1, 2] \rightarrow [\blacksquare, 4, _, _]$
- `Heap.add`: $[_, _, 1, 2] \rightarrow [\blacksquare, 4, 3, _] \rightarrow [\blacksquare, 3, 4, _]$
- `Heap.add`: $[_, _, _, 2] \rightarrow [\blacksquare, 3, 4, 1, _] \rightarrow [\blacksquare, 1, 4, 3, _]$
- `Heap.add`: $[_, _, _, _] \rightarrow [\blacksquare, 1, 4, 3, 2] \rightarrow [\blacksquare, 1, 2, 3, 4]$
- `Heap.remove`: $[1, _, _, _] \leftarrow [\blacksquare, 4, 2, 3, _] \rightarrow [\blacksquare, 2, 4, 3, _]$
- `Heap.remove`: $[1, 2, _, _] \leftarrow [\blacksquare, 3, 4, _, _]$
- `Heap.remove`: $[1, 2, 3, _] \leftarrow [\blacksquare, 4, _, _, _]$
- `Heap.remove`: $[1, 2, 3, 4] \leftarrow [\blacksquare, _, _, _, _]$

Answer of exercise 6.4.11

Answer Not Provided

Answer of exercise 6.4.12

Answer Not Provided

Answer of exercise 6.6.0

Question:

Use the GCD to determine $\text{LCM}(25, 60)$. Show all your work to calculate both the GCD and LCM.

Answer:

$$\begin{aligned}\text{GCD}(25, 60) &= \text{GCD}(60 \bmod 25, 25) \\ &= \text{GCD}(10, 25) \\ &= \text{GCD}(25 \bmod 10, 10) \\ &= \text{GCD}(5, 10) \\ &= \text{GCD}(10 \bmod 5, 5) \\ &= \text{GCD}(0, 5) \\ &= 5\end{aligned}$$

$$\begin{aligned}\text{Thus, } \text{LCM}(25, 60) &= \frac{25 \times 60}{\text{GCD}(25, 60)} \\ &= \frac{1500}{5} \\ &= 300\end{aligned}$$

Answer of exercise 6.6.1

Answer Not Provided

Answer of exercise 6.6.2

Answer Not Provided

Answer of exercise 6.6.3

Answer Not Provided

Answer of exercise 6.6.4

Question:

Solve the following linear program in two dimensions. Provide the location of the best point as well as the value of the objective function at that point. Do not use any automated solvers to find your answer. (Hint: plot the constraints)

Maximize x_0 subject to:

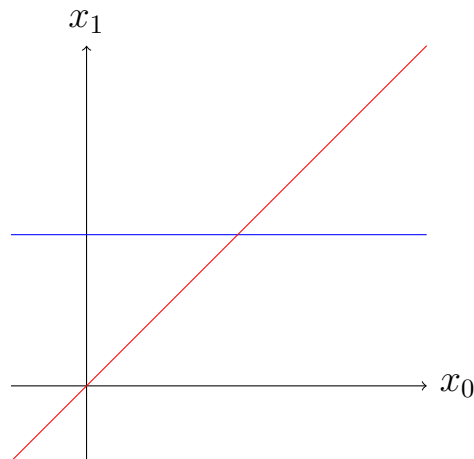
$$x_1 \leq 2$$

$$x_0 - x_1 \leq 0$$

and $x_0, x_1 \geq 0$

Answer:

Plot it out!



Our working space is above the red line and below the blue line. The objective function is maximizing in the direction of

the vector $(1, 0)$, which is the x-axis. Thus, the correct solution is the point furthest to the right, which is where the two lines intersect: $x_0 = 2, x_1 = 2$. The maximum value of the objective function there is 2.

Answer of exercise 6.6.5

Answer Not Provided

Answer of exercise 6.6.6

Answer Not Provided

Answer of exercise 6.6.7

Question:

Transform the following problem into a linear programming problem, then use technology to solve it. (I know that many online solvers are very flexible with their inputs, but you need to put it in the form described in the chapter.) Cite the site you used and how you used it!

Minimize $x_0 - x_1 + x_2$ subject to:

$$x_0 + 5 \geq x_1 + 2x_2$$

$$x_0 + x_1 \leq 10x_2$$

$$3x_1 \leq 10 - x_2$$

$$x_0 \geq -x_2$$

and $x_0, x_1, x_2 \geq 0$

Answer:

- Since we're asked to minimize a formula, we want to maximize its negative, so our objective function will be: $-x_0 + x_1 - x_2$.

- Our first constraint needs to be rewritten to fit the format:

$$\begin{aligned}x_0 + 5 &\geq x_1 + 2x_2 \text{ (flip)} \\x_1 + 2x_2 &\leq x_0 + 5 \\-x_0 + x_1 + 2x_2 &\leq 5\end{aligned}$$

- Fixing the second constraint:

$$\begin{aligned}x_0 + x_1 &\leq 10x_2 \\x_0 + x_1 - 10x_2 &\leq 0\end{aligned}$$

- Fixing the third constraint:

$$\begin{aligned}3x_1 &\leq 10 - x_2 \\3x_1 + 1x_2 &\leq 10\end{aligned}$$

- Fixing the fourth constraint:

$$\begin{aligned}x_0 &\geq -x_2 \text{ (flip)} \\-x_2 &\leq x_0 \\-x_0 - x_2 &\leq 0\end{aligned}$$

All together, the problem now has the form:

Maximize $-x_0 + x_1 - x_2$

subject to:

$$\begin{aligned}-1x_0 + 1x_1 + 2x_2 &\leq 5 \\1x_0 + 1x_1 - 10x_2 &\leq 0 \\0x_0 + 3x_1 + 1x_2 &\leq 10 \\-x_0 - x_2 &\leq 0\end{aligned}$$

and $x_0, x_1, x_2 \geq 0$

Plugging those in to <https://www.zweigmedia.com/RealWorld/simplex.html> yields the solution: $x_0 = 0, x_1 = \frac{100}{31} \approx 3.225806, x_2 = \frac{10}{31} \approx 0.3225806$.

Answer of exercise 6.6.8

Answer Not Provided

Answer of exercise 6.6.9

Answer Not Provided

Answer of exercise 7.3.0

Question:

Consider the hash function $h : [0, 1, 2, \dots, 1023] \rightarrow [0, 1, 2, \dots, 12]$ where $h(x) = x \bmod 13$. Let's create a set, S , implemented with a closed hash table, T , that uses h .

- a) Draw T after adding elements 1004, 561, 620, 81, 120, and 180, to S , in that order.
- b) How many cells are checked to determine that $400 \notin S$?
- c) How many cells are checked to determine that $327 \notin S$?
- d) How many cells are checked to determine that $120 \in S$?

Answer:

- a)
 - $1004 \bmod 13 = 3 \checkmark$
 - $561 \bmod 13 = 2 \checkmark$
 - $620 \bmod 13 = 9 \checkmark$
 - $81 \bmod 13 = 3 \rightarrow 4 \checkmark$
 - $120 \bmod 13 = 3 \rightarrow 4 \rightarrow 5 \checkmark$

- $180 \bmod 13 = 11$ ✓

		561	1004	81	120				620		180	
0	1	2	3	4	5	6	7	8	9	10	11	12

- b) $400 \bmod 13 = 10$, which is empty, so only one cell needs to be checked.
- c) $327 \bmod 13 = 2$, but cells 2, 3, 4, and 5 are occupied, so 5 cells need to be checked.
- d) $120 \bmod 13 = 3$, but cells 3 and 4 contain other values. Cell 5 contains 120, so 3 cells need to be checked.

Answer of exercise 7.3.1

Answer Not Provided

Answer of exercise 7.3.2

Answer Not Provided

Answer of exercise 7.3.3

Question:

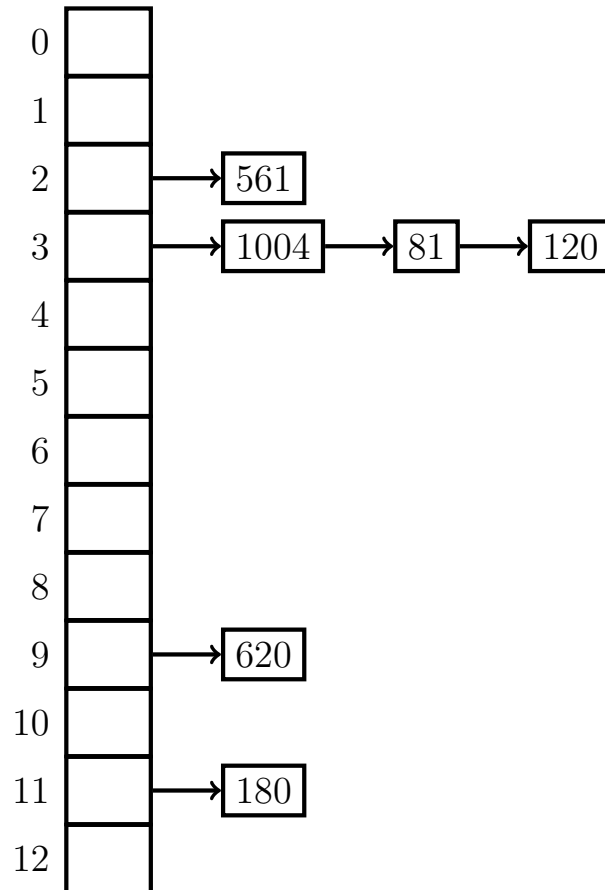
Consider the hash function $h : [0, 1, 2, \dots, 1023] \rightarrow [0, 1, 2, \dots, 12]$ where $h(x) = x \bmod 13$. Let's create a set, S , implemented with an open hash table, T , that uses h .

- a) Draw T after adding elements 1004, 561, 620, 81, 120, and 180, to S , in that order.
- b) How many tests on lists are made to determine that $400 \notin S$?
- c) How many tests on lists are made to determine that $327 \notin S$?

d) How many tests on lists are made to determine that $120 \in S$?

Answer:

- a)
- $1004 \bmod 13 = 3 \checkmark$
 - $561 \bmod 13 = 2 \checkmark$
 - $620 \bmod 13 = 9 \checkmark$
 - $81 \bmod 13 = 3 \checkmark$
 - $120 \bmod 13 = 3 \checkmark$
 - $180 \bmod 13 = 11 \checkmark$



- b) One check. $400 \bmod 13 = 10$, and the list at index 10 is empty.
- c) Two checks. $327 \bmod 13 = 2$, 327 isn't in the linked list starting at index 2, and that list has a length of 1.
- d) Three checks! $120 \bmod 13 = 3$ and 120 is the twoeth element in the list at index 3.

Answer of exercise 7.3.4

Answer Not Provided

Answer of exercise 7.3.5

Answer Not Provided

Answer of exercise 7.3.6

Answer Not Provided

Answer of exercise 8.0.0

Question:

Execute `DynamicProgrammingMakeChange([4, 3, 1], 6, new HashTable)`. Show each of the additions to the hash table in the correct order. You can include any other computational steps you want as long as you show at least the hash table additions.

Answer:

I showed each call that results in a new element added to h .
 (I indented each inner call to make it more clear.)
`DynamicProgrammingMakeChange([4, 3, 1], 6, h)`
 `DynamicProgrammingMakeChange([4, 3, 1], 2, h)`
 `DynamicProgrammingMakeChange([4, 3, 1], 1, h)`
 `DynamicProgrammingMakeChange([4, 3, 1], 0, h)`

```
        h.put(0, [0, 0, 0])
    h.put(1, [0, 0, 1])
    h.put(2, [0, 0, 2])
    DynamicProgrammingMakeChange([4, 3, 1], 3, h)
    h.put(3, [0, 1, 0])
    DynamicProgrammingMakeChange([4, 3, 1], 5, h)
        DynamicProgrammingMakeChange([4, 3, 1], 4, h)
            h.put(4, [1, 0, 0])
                h.put(5, [1, 0, 1])
                    h.put(6, [0, 2, 0])
```

Answer of exercise 8.0.1

Answer Not Provided

Answer of exercise 8.0.2

Answer Not Provided

Answer of exercise 8.0.3

Answer Not Provided

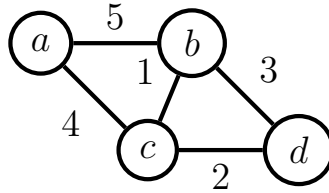
Answer of exercise 8.0.4

Answer Not Provided

Answer of exercise 9.1.0

Question:

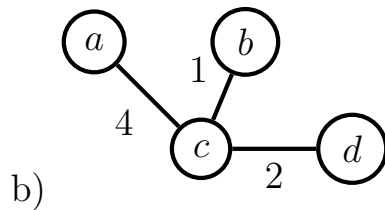
Use Prim's Algorithm to find the minimum spanning tree of the graph below, starting from vertex a :



- a) Show the state of V_T , T , and P each time T and V_T change and the new edges are added to P . You do not have to include duplicate edges in P . (I wrote out P as a sorted list.)
- b) Draw the final spanning tree with the edge weights. (Do not draw the unchosen edges.)

Answer:

- a)
- $V_T = \{a\}$; $T = \emptyset$;
 $P = [((a, c), 4), ((a, b), 5)]$
 - $V_T = \{a, c\}$; $T = \{(a, c)\}$;
 $P = [((c, b), 1), ((c, d), 2), ((a, b), 5)]$
 - $V_T = \{a, c, b\}$; $T = \{(a, c), (c, b)\}$;
 $P = [((c, d), 2), ((b, d), 3), ((a, b), 5)]$
 - $V_T = \{a, c, b, d\}$; $T = \{(a, c), (c, b), (c, d)\}$;
 $P = [((b, d), 3), ((a, b), 5)]$



Answer of exercise 9.1.1

Answer Not Provided

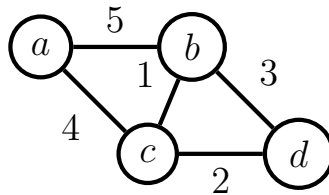
Answer of exercise 9.1.2

Answer Not Provided

Answer of exercise 9.2.0

Question:

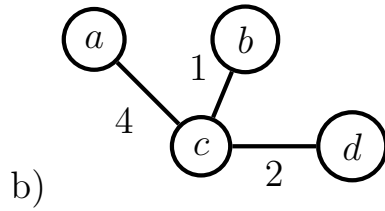
Use Kruskal's Algorithm to find the minimum spanning tree of the graph below.



- a) Show the state of T , S , and P each time T changes. (I wrote out P as a sorted list.)
- b) Draw the final spanning tree with the edge weights. (Do not draw the unchosen edges.)

Answer:

- a)
 - $T = \emptyset$; $S = \{\{a\}, \{b\}, \{c\}, \{d\}\}$;
 $P = [((b, c), 1), ((c, d), 2), ((b, d), 3), ((a, c), 4), ((a, b), 5)]$
 - $T = \{(b, c)\}$; $S = \{\{a\}, \{b, c\}, \{d\}\}$;
 $P = [((c, d), 2), ((b, d), 3), ((a, c), 4), ((a, b), 5)]$
 - $T = \{(b, c), (c, d)\}$; $S = \{\{a\}, \{b, c, d\}\}$;
 $P = [((b, d), 3), ((a, c), 4), ((a, b), 5)]$
 - $T = \{(b, c), (c, d), (a, c)\}$; $S = \{\{a, b, c, d\}\}$;
 $P = [((a, b), 5)]$



Answer of exercise 9.2.1

Answer Not Provided

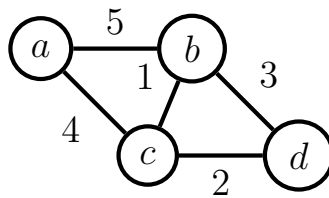
Answer of exercise 9.2.2

Answer Not Provided

Answer of exercise 9.3.0

Question:

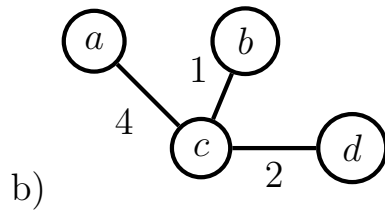
Use Borůvka's Algorithm to find the minimum spanning tree of the graph below.



- a) Show T , and $Trees$ initially and after every round of the main while loop. Also, for each iteration of the main while loop, show the edge (x, y) each tree selects during the second inner loop (**for** $tree \in Trees$ **do**).
- b) Draw the final spanning tree with the edge weights. (Do not draw the unchosen edges.)

Answer:

- a) • $T = \emptyset$; $Trees = \{\{a\}, \{b\}, \{c\}, \{d\}\}$; new edges by tree:
- $\{a\} : (a, c)$
 - $\{b\} : (b, c)$
 - $\{c\} : (b, c)$
 - $\{d\} : (c, d)$
- $T = \{(b, c), (c, d), (a, c)\}$; $Trees = \{\{a, b, c, d\}\}$



Answer of exercise 9.3.1

Answer Not Provided

Answer of exercise 9.3.2

Answer Not Provided

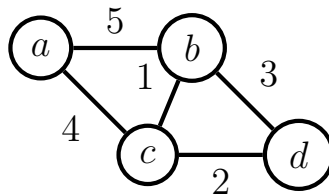
Answer of exercise 9.3.3

Answer Not Provided

Answer of exercise 9.4.0

Question:

Use Dijkstra's Shortest Path Algorithm to find the shortest path from a to d in the graph below.



- a) Show the state of P and list the distances of the nodes at the end of the first for loop inside the first while loop, and after that big while loop. (I wrote out P as a sorted list.) Break ties alphabetically by the name of the vertices.
- b) Show SP at the end.

Answer:

- a)
- $P = [((a, b), 5), ((a, c), 4)];$
 $a.dist = 0, b.dist = 5, c.dist = 4, d.dist = \infty$
 - $P = [((a, b), 5), ((c, b), 5), ((c, d), 6)];$
 $a.dist = 0, c.dist = 4, b.dist = 5, d.dist = 6$
 - $P = [((c, b), 5), ((c, d), 6), ((b, d), 8)];$
 $a.dist = 0, c.dist = 4, b.dist = 5, d.dist = 6$
 - $P = [((b, d), 8)];$
 $a.dist = 0, c.dist = 4, b.dist = 5, d.dist = 6$
- b) $SP = [a, c, d]$

Answer of exercise 9.4.1

Answer Not Provided

Answer of exercise 9.4.2

Answer Not Provided

Answer of exercise 11.3.0

Question:

Consider three computational problems, A , B , and C such that B is NP-hard and C is in PSPACE. Assume we find two reductions, $f_1: A \rightarrow B$ and $f_2: B \rightarrow C$.

- a) What's the strongest thing we can say about the computational complexity of A and why?
- b) What's the strongest thing we can say about the computational complexity of B and why?
- c) What's the strongest thing we can say about the computational complexity of C and why?
- d) Would any of this change our current understanding of complexity classes?

Answer:

- a) $A \in \text{PSPACE}$, because $C \in \text{PSPACE}$ and Easiness salmons the two reductions to reach A .
- b) $B \in \text{PSPACE}$ and B is NP-hard. B is NP-hard because we were given that and $B \in \text{PSPACE}$, because $C \in \text{PSPACE}$ and Easiness salmons a reduction.
- c) $C \in \text{PSPACE}$ and C is NP-hard. $C \in \text{PSPACE}$ because we were given that and C is NP-hard because B is NP-hard and hardness follows the reduction.
- d) None of these contradict our current understanding of complexity classes.

Answer of exercise 11.3.1

Answer Not Provided

Answer of exercise 11.3.2

Answer Not Provided

References

- [1] Alan Cobham. The intrinsic computational difficulty of functions. pages 24–30, 1965.
- [2] Jack Edmonds. Paths, trees, and flowers. *Canadian Journal of Mathematics*, 17:449–467, 1965.
- [3] Anany V. Levitin. *Introduction to the Design and Analysis of Algorithms (3rd Edition)*. Pearson Education, USA, 2012.